Tools preparation:

1. Sample datasets
   https://drive.google.com/file/d/1uB7pjzGoHMDTqITfR0dCgL-pVVqtnts2/view?usp=sharing

2. Download, install and verify installation of Python for Windows (v3.6 or above)
   https://www.python.org/downloads/

   *set python command in environment variables

   Check version

```
\> python --version
```

3. Download and install jupyter notebook

```
\> pip install jupyter
```

   Set jupyter notebook password

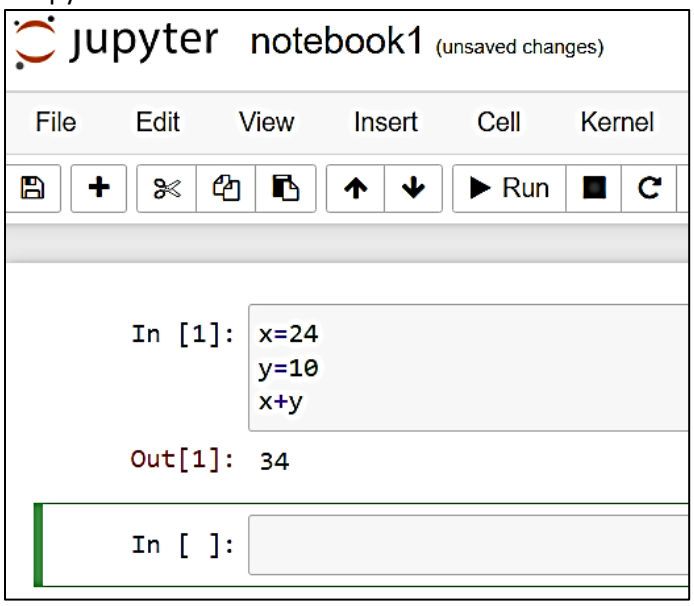
```
\> jupyter notebook password
```

   Launch jupyter notebook in browser

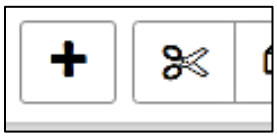
```
\> jupyter notebook
```

   *In jupyter notebook dashboard, create a new folder. Then create a new python notebook inside.
   *note that notebooks have '.ipynb' file extension

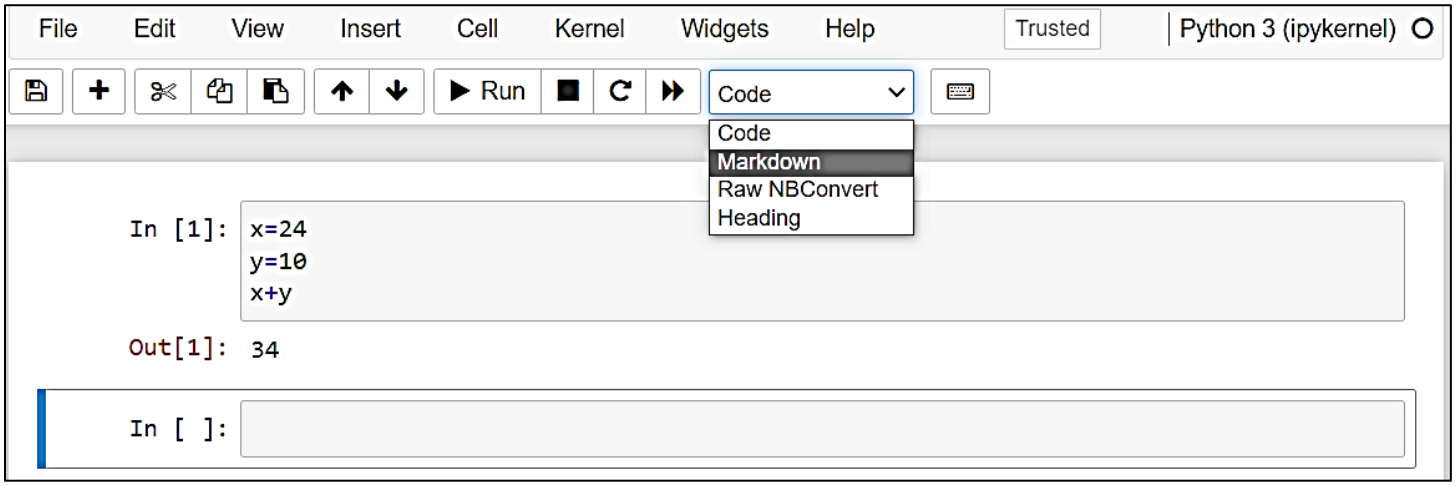
   (optional)
   Try a basic test and run for python



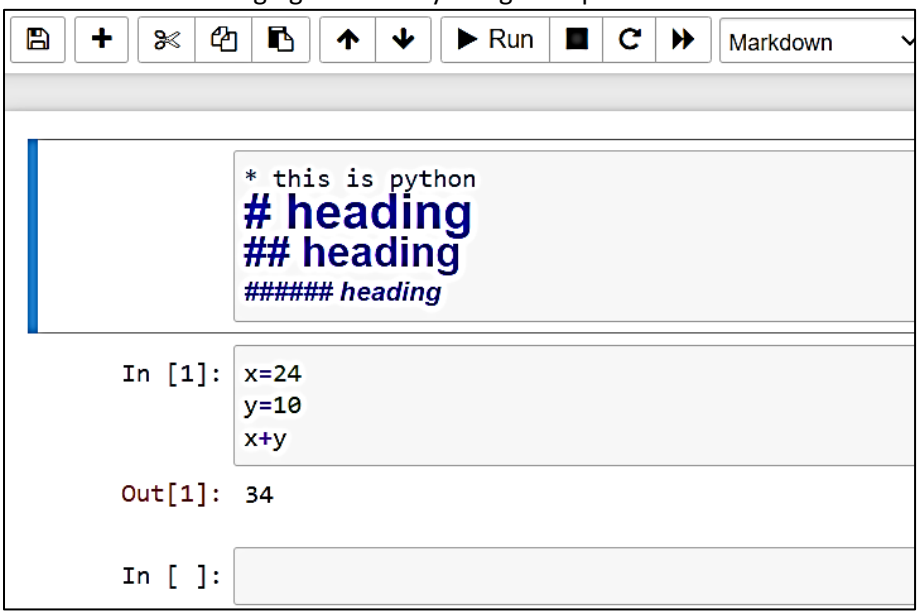
Add or remove cells



Create a markdown cell (comments / remarks / points / html codes to add font effects like **bold**)
Add a new cell → change the cell format to markdown



Try adding some content and arranging the cells by using the up and down arrow icons



In running some commands in the notebook, adding exclamation before the commands means it will be executed as a shell command and not as a notebook command

```
In [6]: !ls
        notebook1.ipynb

In [7]: ls
         Volume in drive C is Windows
         Volume Serial Number is 040E-0439

         Directory of C:\Users\core360\pythontests

        20/08/2022  04:18 PM    <DIR>          .
        20/08/2022  04:18 PM    <DIR>          ..
        20/08/2022  01:16 PM    <DIR>          .ipynb_checkpoints
        20/08/2022  04:18 PM             1,862 notebook1.ipynb
                       1 File(s)          1,862 bytes
                       3 Dir(s)  131,291,615,232 bytes free
```

You'll notice other shell commands also work:

```
In [11]: pwd
Out[11]: 'C:\\Users\\core360\\pythontests'

In [12]: !pwd
         /c/Users/core360/pythontests
```

Change directory:

```
In[1]: !cd /var/etc
```

Lesson 01 - Data Science Overview

Introduction to Data Science

The data science lifecycle involves various roles, tools, and processes, which enables analysts to present and perform actionable insights. Typically, a data science project undergoes the following stages:

- Data ingestion: The lifecycle begins with the data collection--both raw structured and unstructured data from all relevant sources using a variety of methods. These methods can include manual entry, web scraping, and real-time streaming data from systems and devices. Data sources can include structured data, such as customer data, along with unstructured data like log files, video, audio, pictures, the Internet of Things (IoT), social media, and more.

- Data storage and data processing: Since data can have different formats and structures, companies need to consider different storage systems based on the type of data that needs to be captured. Data management teams help to set standards around data storage and structure, which facilitate workflows around analytics, machine learning and deep learning models. This stage includes cleaning data, deduplicating, transforming and combining the data using ETL (extract, transform, load) jobs or other data integration technologies. This

data preparation is essential for promoting data quality before loading into a data warehouse, data lake, or other repository.

- Data analysis: Here, data scientists conduct an exploratory data analysis to examine biases, patterns, ranges, and distributions of values within the data. This data analytics exploration drives hypothesis generation for a/b testing. It also allows analysts to determine the data's relevance for use within modeling efforts for predictive analytics, machine learning, and/or deep learning. Depending on a model's accuracy, organizations can become reliant on these insights for business decision making, allowing them to drive more scalability.

- Communicate: Finally, insights are presented as reports and other data visualizations that make the insights—and their impact on business—easier for business analysts and other decision-makers to understand. A data science programming language such as R or Python includes components for generating visualizations; alternately, data scientists can use dedicated visualization tools.

It may be easy to confuse the terms "data science" and "business intelligence" (BI) because they both relate to an organization's data and analysis of that data, but they do differ in focus.

Business intelligence (BI) is typically an umbrella term for the technology that enables data preparation, data mining, data management, and data visualization. Business intelligence tools and processes allow end users to identify actionable information from raw data, facilitating data-driven decision-making within organizations across various industries. While data science tools overlap in much of this regard, business intelligence focuses more on data from the past, and the insights from BI tools are more descriptive in nature. It uses data to understand what happened before to inform a course of action. BI is geared toward static (unchanging) data that is usually structured. While data science uses descriptive data, it typically utilizes it to determine predictive variables, which are then used to categorize data or to make forecasts

Different Sectors Using Data Science

Almost all industries can benefit from data science and analytics. However, below are some industries that are better poised to make use of data science and analytics.

- Retailers need to correctly anticipate what their customers want and then provide those things. If they don't do this, they will likely be left behind the competition. Big data and analytics provide retailers the insights they need to keep their customers happy and returning to their stores.

- The medical industry is using big data and analytics in a big way to improve health in a variety of ways. For instance, the use of wearable trackers to provide important information to physicians who can make use of the data to provide better care to their patients. Wearable trackers also provide information like whether the patient is taking his/her medication and following the right treatment plan.

- The banking industry is generally not looked at as being one that uses technology a lot. However, this is slowly changing as bankers are beginning to increasingly use technology to drive their decision-making. For instance, the Bank of America uses natural language processing and predictive analytics to create a virtual assistant called Erica to help customers view information on upcoming bills or view transaction histories. The assistant will eventually study their customers' banking habits and suggest relevant financial advice at appropriate times.

- Construction companies track everything from the average time needed to complete tasks to materials-based expenses and everything in between. Big data is now being used in a big way in the construction industry to drive better decision-making.

- There is always a need for people to reach their destinations on time and data science and analytics can be used by transportation providers, both public and private, to increase the chances of successful journeys. For instance, most Travel Road Guide Apps and uses statistical data to map customer journeys, manage unexpected circumstances, and provide people with personalized transport details.

- Consumers now expect rich media in different formats as and when they want it on a variety of devices. Collecting, analyzing, and utilizing these consumer insights is now a challenge that data science is stepping in to tackle. Data science is being used to leverage social media and mobile content and understand real-time, media content usage patterns. With data science techniques, companies can better create content for different target audiences, measure content performance, and recommend on-demand content.

- One challenge in the education industry where data science and analytics can help is to incorporate data from different vendors and sources and use them on platforms not designed for varying data. Schools have developed a learning and management system that can track when a student logs into the system, the overall progress of the student, and how much time is spent on different pages, among other things. Big data can also be used to measure teachers' effectiveness by fine-tuning teachers' performance by measuring against subject matter, student numbers, student aspirations, student demographics, and many other variables.

- The manufacturing industry also generates huge amounts of data that has so far gone untapped. The increasing demand and supply of natural resources, such as oil, minerals, gas, metals, agricultural products, etc. has led to the generation of huge amounts of data that is complex, difficult to handle, and a prime candidate for big data analytics.

- In the Government, big data has many applications in the public services field. Places where big data is/can be used include in financial market analysis, health-related research, environmental protection, energy exploration, and fraud detection. One specific example is the use of big data analytics by the Social Security System (SSS) to analyze large numbers of social disability claims that come in as unstructured data. Analytics is being used to rapidly process medical information and detect fraudulent or suspicious claims. Another example is the use of data science techniques by the Food and Drug Administration (FDA) to identify and analyze patterns related to food-related diseases and illnesses.

- The energy and utilities industry generates and will continue to generate huge amounts of data that can be analyzed using big data analytics. For instance, nowadays, smart readers allow data to be collected every 15 minutes or so as compared to how it was previously when it was once a day. This data can be used to better study the consumption of utilities, which in turn allows for better control of utility use and improved customer feedback. The use of big data by utility companies also allows for improved asset and workforce management and is useful for identifying and correcting errors as soon as possible.

Purpose and Components of Python

Why Python for Data Science?

What distinguishes python is its popularity among Data Scientists. Many of the open source tools used in ETL / ML Modelling / Visualization – which may sit behind seamless APIs – leverage python in some capacity. Knowing the language allows you to learn from the code others have written.

Some Pros:

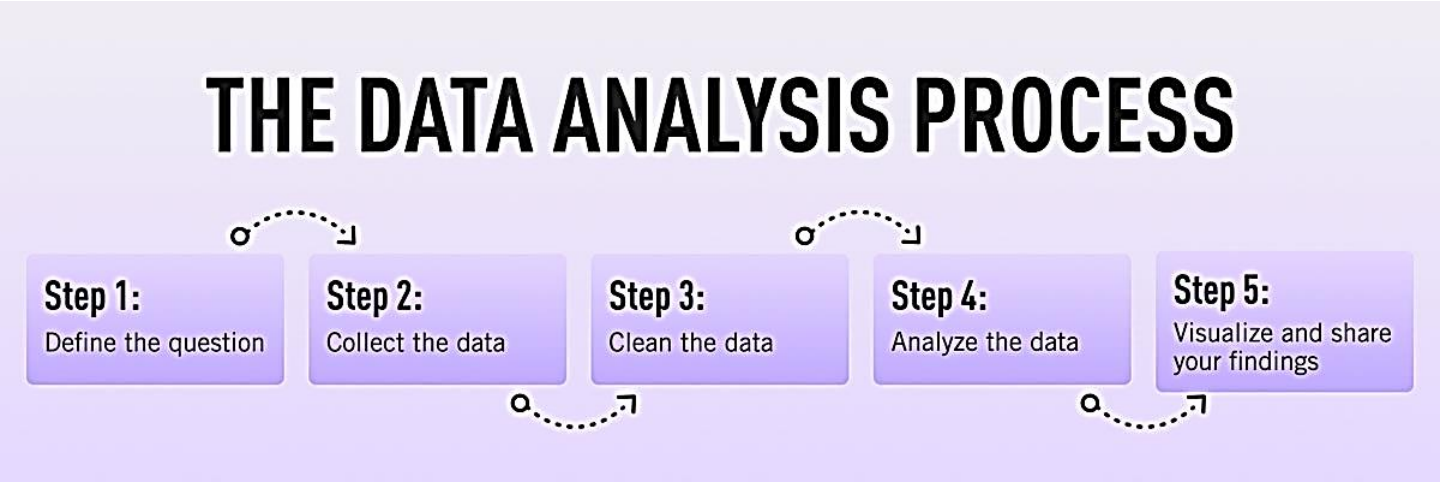
- Python is one of the most commonly used language for data-science has a lot of library available for that use case, from Scikit-Learn, XGBoost, PySpark, Tensorflow, Pytorch …
- It is an interactive language, so you can see the feedback of the different operation sequentially
- It tends to be more easily integrated into production systems than languages such as R, and using it might make it easier to work with engineers as a common language, and makes it easier to learn some of the adjacent topic, from Cloud topics, to data-engineering, to setup web apis ..

Some Cons:

- It isn't as fast as some other languages such as Scala when using native functions
- There is in my opinion an overall lack of a good data-science IDE such as RStudio
- It isn't a quick MVP data application framework such as Shiny on R
- In database computation support isn't as extensive as R (SQLServer, Teradata and Oracle all support it)

Lesson 02 - Data Analytics Overview

Data Analytics Process



1. Step one: Defining the question
   - ✓ define your objective or in data analytics jargon, this is sometimes called the 'problem statement'.
   - ✓ Come up with a hypothesis and figuring how to test it.
   - ✓ What business problem am I trying to solve? A data analyst's job is to understand the business and its goals in enough depth that they can frame the problem the right way.
   - ✓ Determine which sources of data will best help you solve it. This is where your business acumen comes in again. For instance, perhaps you've noticed that the sales process for new clients is very slick, but that the production team is inefficient. Knowing this, you could hypothesize that the sales process wins lots of new

clients, but the subsequent customer experience is lacking. Could this be why customers don't come back? Which sources of data will help you answer this question?

- ✓ Defining your objective is mostly about soft skills, business knowledge, and lateral thinking. But you'll also need to keep track of business metrics and key performance indicators (KPIs). Monthly reports can allow you to track problem points in the business.
- ✓ Some KPI dashboards come with a fee, like Databox and DashThis. However, you'll also find open-source software like Grafana, Freeboard, and Dashbuilder. These are great for producing simple dashboards, both at the beginning and the end of the data analysis process.

2. Step two: Collecting the data

- ✓ Create a strategy for collecting and aggregating the appropriate data.
- ✓ Determining which data you need.
  - quantitative (numeric) data, e.g. sales figures
  - qualitative (descriptive) data, such as customer reviews.

- ✓ All data fit into one of three categories: first-party, second-party, and third-party data. Let's explore each one.
  - First-party data are data that you, or your company, have directly collected from customers. It might come in the form of transactional tracking data or information from your company's customer relationship management (CRM) system. Whatever its source, first-party data is usually structured and organized in a clear, defined way. Other sources of first-party data might include customer satisfaction surveys, focus groups, interviews, or direct observation.

  - Second-party data is the first-party data of other organizations. This might be available directly from the company or through a private marketplace. The main benefit of second-party data is that they are usually structured, and although they will be less relevant than first-party data, they also tend to be quite reliable. Examples of second-party data include website, app or social media activity, like online purchase histories, or shipping data.

  - Third-party data is data that has been collected and aggregated from numerous sources by a third-party organization. Often (though not always) third-party data contains a vast amount of unstructured data points (big data). Many organizations collect big data to create industry reports or to conduct market research. The research and advisory firm Gartner is a good real-world example of an organization that collects big data and sells it on to other companies. Open data repositories and government portals are also sources of third-party data.

- ✓ One thing you'll need, regardless of industry or area of expertise, is a data management platform (DMP). A DMP is a piece of software that allows you to identify and aggregate data from numerous sources, before manipulating them, segmenting them, and so on. There are many DMPs available. Some well-known enterprise DMPs include Salesforce DMP, SAS, and the data integration platform, Xplenty. If you want to play around, you can also try some open-source platforms like Pimcore or D:Swarm.

3. Step three: Cleaning the data

- ✓ Get data ready for analysis. This means cleaning, or 'scrubbing' it, and is crucial in making sure that you're working with high-quality data.

- ✓ Key data cleaning tasks include:
  - Removing major errors, duplicates, and outliers—all of which are inevitable problems when aggregating data from numerous sources.
  - Removing unwanted data points—extracting irrelevant observations that have no bearing on your intended analysis.

- Bringing structure to your data—general 'housekeeping', i.e. fixing typos or layout issues, which will help you map and manipulate your data more easily.
- Filling in major gaps—as you're tidying up, you might notice that important data are missing. Once you've identified gaps, you can go about filling them.

- ✓ A good data analyst will spend around 70-90% of their time cleaning their data. This might sound excessive. But focusing on the wrong data points (or analyzing erroneous data) will severely impact your results
- ✓ Another thing many data analysts do (alongside cleaning data) is to carry out an exploratory analysis. This helps identify initial trends and characteristics, and can even refine your hypothesis.
- ✓ Cleaning datasets manually—especially large ones—can be daunting. Luckily, there are many tools available to streamline the process. Open-source tools, such as OpenRefine, are excellent for basic data cleaning, as well as high-level exploration. However, free tools offer limited functionality for very large datasets.
- ✓ Python libraries (e.g. Pandas) and some R packages are better suited for heavy data scrubbing. You will, of course, need to be familiar with the languages.
- ✓ Alternatively, enterprise tools are also available. For example, Data Ladder, which is one of the highest-rated data-matching tools in the industry. There are many more.

4. Step four: Analyzing the data
   - ✓ The type of data analysis you carry out largely depends on what your goal is.
   - ✓ There are many techniques available. Univariate or bivariate analysis, time-series analysis, and regression analysis are just a few you might have heard of.

   - ✓ Broadly speaking, all types of data analysis fit into one of the following four categories.
     - Descriptive analysis identifies what has already happened. It is a common first step that companies carry out before proceeding with deeper explorations. As an example, an online learning platform might use descriptive analytics to analyze course completion rates for their customers. Or they might identify how many users access their products during a particular period.

     - Diagnostic analytics focuses on understanding why something has happened. 'Which factors are negatively impacting the customer experience?' A diagnostic analysis would help answer this. For instance, it could help the company draw correlations between the issue (struggling to gain repeat business) and factors that might be causing it (e.g. project costs, speed of delivery, customer sector, etc.)

     - Predictive analysis allows you to identify future trends based on historical data. In business, predictive analysis is commonly used to forecast future growth, for example. But it doesn't stop there. Predictive analysis has grown increasingly sophisticated in recent years. The speedy evolution of machine learning allows organizations to make surprisingly accurate forecasts. Take the insurance industry. Insurance providers commonly use past data to predict which customer groups are more likely to get into accidents. As a result, they'll hike up customer insurance premiums for those groups. Likewise, the retail industry often uses transaction data to predict where future trends lie, or to determine seasonal buying habits to inform their strategies.

     - Prescriptive analysis allows you to make recommendations for the future. This is the final step in the analytics part of the process. It's also the most complex. This is because it incorporates aspects of all the other analyses we've described. A great example of prescriptive analytics is the algorithms that guide Google's self-driving cars. Every second, these algorithms make countless decisions based on past and present data, ensuring a smooth, safe ride. Prescriptive analytics also helps companies decide on new products or areas of business to invest in.

5. Step five: Sharing your results

- ✓ This involves interpreting the outcomes, and presenting them in a manner that's digestible for all types of audiences. Since you'll often present information to decision-makers, it's very important that the insights you present are 100% clear and unambiguous. Data analysts commonly use reports, dashboards, and interactive visualizations to support their findings.

- ✓ How you interpret and present results will often influence the direction of a business. Depending on what you share, your organization might decide to restructure, to launch a high-risk product, or even to close an entire division. That's why it's very important to provide all the evidence that you've gathered, and not to cherry-pick data. Ensuring that you cover everything in a clear, concise way will prove that your conclusions are scientifically sound and based on the facts. On the flip side, it's important to highlight any gaps in the data or to flag any insights that might be open to interpretation.

- ✓ There are tons of data visualization tools available, suited to different experience levels. Popular tools requiring little or no coding skills include Google Charts, Tableau, Datawrapper, and Infogram.

- ✓ If you're familiar with Python and R, there are also many data visualization libraries and packages available. For instance, check out the Python libraries Plotly, Seaborn, and Matplotlib.

Summary

In this post, we've covered the main steps of the data analytics process. These core steps can be amended, re-ordered and re-used as you deem fit, but they underpin every data analyst's work:

a. Define the question—What business problem are you trying to solve? Frame it as a question to help you focus on finding a clear answer.
b. Collect data—Create a strategy for collecting data. Which data sources are most likely to help you solve your business problem?
c. Clean the data—Explore, scrub, tidy, de-dupe, and structure your data as needed. Do whatever you have to! But don't rush…take your time!
d. Analyze the data—Carry out various analyses to obtain insights. Focus on the four types of data analysis: descriptive, diagnostic, predictive, and prescriptive.
e. Share your results—How best can you share your insights and recommendations? A combination of visualization tools and communication is key.

Exploratory Data Analysis (EDA)

EDA is often used to:

- ✓ Discover patterns
- ✓ Spot anomalies
- ✓ Test hypothesis
- ✓ Summary statistics
- ✓ Graphical representations

EDA-Quantitative Technique

Many of the quantitative techniques fall into two broad categories: Interval estimation and Hypothesis tests

Interval Estimates

It is common in statistics to estimate a parameter from a sample of data. The value of the parameter using all of the possible data, not just the sample data, is called the population parameter or true value of the parameter. An estimate of the true parameter value is made using the sample data. This is called a point estimate or a sample estimate.

For example, the most commonly used measure of location is the mean. The population, or true, mean is the sum of all the members of the given population divided by the number of members in the population. As it is typically impractical to measure every member of the population, a random sample is drawn from the population. The sample mean is calculated by summing the values in the sample and dividing by the number of values in the sample. This sample mean is then used as the point estimate of the population mean.

Hypothesis Tests

Hypothesis tests also address the uncertainty of the sample estimate. However, instead of providing an interval, a hypothesis test attempts to refute a specific claim about a population parameter based on the sample data. For example, the hypothesis might be one of the following:

- the population mean is equal to 10
- the population standard deviation is equal to 5
- the means from two populations are equal
- the standard deviations from 5 populations are equal

To reject a hypothesis is to conclude that it is false. However, to accept a hypothesis does not mean that it is true, only that we do not have evidence to believe otherwise. Thus hypothesis tests are usually stated in terms of both a condition that is doubted (null hypothesis) and a condition that is believed (alternative hypothesis).

EDA - Graphical Technique

Graphical Techniques with Formulas:
https://www.itl.nist.gov/div898/handbook/eda/section3/eda33.htm

Data Profiling

Data profiling is the process of examining the data available by performing

1. descriptive analysis
2. informative summaries about the data

Data profiling help us make a thorough assessment of data quality and helps us understand the

- Content
- Structure
- Relationship

Activity: Data Profiling

1. Install the Pandas Profiling Library

```
\> pip install pandas-profiling
```

2. Prepare the dataset (“Iris.csv”)
3. Back to notebook, import and call reference to the library, read the dataset, display top 5 rows

```
In [16]: import pandas_profiling  #import pandas_profiling library
         import pandas as pd      #import pandas for reading dataset

In [17]: # reading the dataset

         data = pd.read_csv("Iris.csv")
```

4. Display rows

```
In [19]: # check top 5 rows
         data.head()
```

Out[19]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [20]: # check last 5 rows
         data.tail()
```

Out[20]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

```
In [21]: # check a random row
         data.sample()
```

Out[21]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
104	105	6.5	3.0	5.8	2.2	Iris-virginica

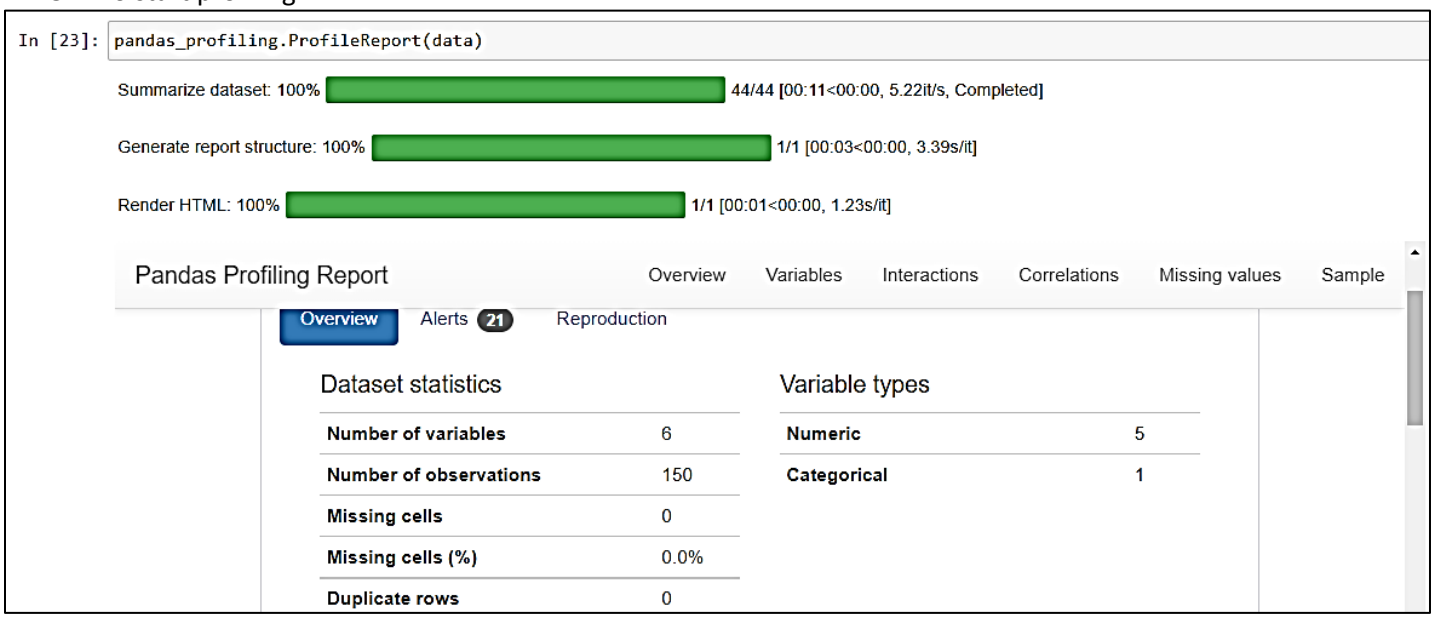
```
In [22]: # check multiple random row
         data.sample(3)
```

Out[22]:

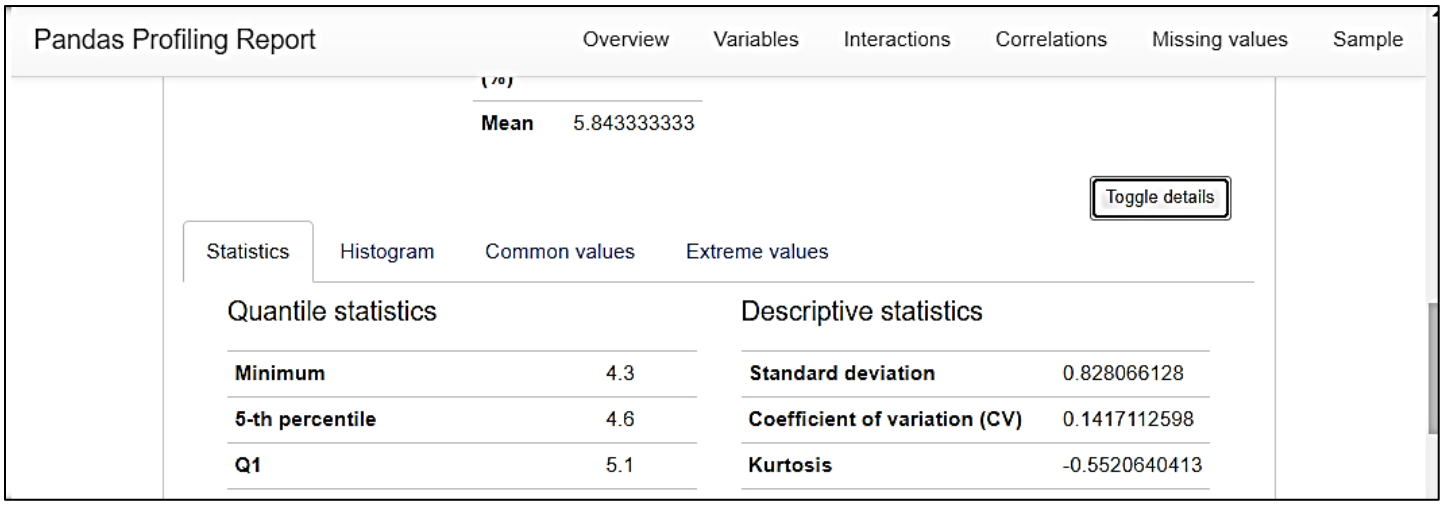
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
108	109	6.7	2.5	5.8	1.8	Iris-virginica
135	136	7.7	3.0	6.1	2.3	Iris-virginica
9	10	4.9	3.1	1.5	0.1	Iris-setosa

*observe the 'Species' column. Here it is the target column. The Target Column is a variable whole values are to modeled and predicted by learning other variables present in the data set.

5. To start profiling



*take time in analyzing the report.



*navigate to the Variables section and notice how the columns are subjected to multiple quantitative approached like min-max-mean-distinct-missing-zeroes, click on 'toggle details' button to see more outputs like statistics, histogram, common values and extreme values

*Correlation is defined as the degree of relationship between two or more variables. It is also called the simple correlation. The degree of relationship between two or more variables is called multi correlation. when two or more variables are said to be highly correlated it means that they have a strong relationship such that a given rise or fall in one variable will lead to a direct change in the other variable or variables. good examples of highly correlated variables are price and quantity, wage rate and output, tax and income.

Saving profile reports to file

```
# generate html report based on our panda profiling
profile1 = pandas_profiling.ProfileReport(data)
profile1.to_file("your_report.html")

# generate json report based on our panda profiling
profile1 = pandas_profiling.ProfileReport(data)
profile1.to_file("your_report.json")
```

Setting Title and defined plot

```
pandas_profiling.ProfileReport(data,title="Iris Profiling Report 1")
```

Using minimal reports (only includes Overview and Variables Pages

```
pandas_profiling.ProfileReport(data, title="Iris Profiling Report Minimal", minimal=True )
```

Analyze target data

1. Let us check the value count of each species (target column is categorical)

```
In [31]: #check value count of species
         data['Species'].value_counts()

Out[31]: Iris-setosa        50
         Iris-versicolor    50
         Iris-virginica     50
         Name: Species, dtype: int64
```

2. Show statistical data

```
In [32]: #check statistical data
         data.describe()
```

Out[32]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

Summarizing Data

describe() function:
*show descriptive statistics for numerical data only

```
import pandas_profiling
import pandas as pd

data = pd.read_csv("Iris.csv")
data.describe()
```

Output:

- **Count** returns the number of observations in each column
- **Mean** returns the mean of all the observations of the variable
- **Standard Deviation** returns the standard deviation is the average amount of variability in your dataset. It tells you, on average, how far each value lies from the mean. A high standard deviation means that values are generally far from the mean, while a low standard deviation indicates that values are clustered close to the mean.

  In normal distributions, data is symmetrically distributed with no skew. Most values cluster around a central region, with values tapering off as they go further away from the center. The standard deviation tells you how spread out from the center of the distribution your data is on average.

- **Min** returns the minimum value present in each column
- **Max** returns the maximum value present in each column
- **25 percentile** returns the percentage of observations falling below 25%
- **50 percentile** returns the percentage of observations falling below 50%
- **75 percentile** returns the percentage of observations falling below 75%

```
#check statistical data for numerical (non-categorical) columns
data.describe()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
#check statistical data for categorical (string) columns
data.describe(include='object')
```

	Species
count	150
unique	3
top	Iris-setosa
freq	50

Exploring the Dabl library

Data Analysis Baseline Library (Dabl)
*use for exploratory visualizations and preprocessing

1. Create a new notebook
2. We will be using the ***diabetes.csv*** dataset
3. Install Dabl

```
\> pip install dabl
```

4. Import packages

```
import pandas as pd
import dabl
```

5. Import the dataset and try reading records

```
data = pd.read_csv('diabetes.csv')
data.head()
```

6. Try cleaning the dataset

```
data_clean = dabl.clean(data, target_col='Outcome',verbose=1)
data_clean.head()
```

```
Detected feature types:
continuous      7
dirty_float     0
low_card_int    1
categorical     1
date            0
free_string     0
useless         0
dtype: int64
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	2	138	62	35	0	33.6	0.127	47	1
1	0	84	82	31	125	38.2	0.233	23	0
2	0	145	0	0	0	44.2	0.630	31	1
3	0	135	68	42	250	42.3	0.365	24	1
4	1	139	62	41	480	40.7	0.536	21	0

Note:

- Continuous: columns which are having continuous values and high cardinality.
- dirty float: float variable that takes string values.
- low card int: columns which contain integer type values and have low cardinality.
- categorical: columns containing pandas categorical values in an int, float or string type.
- date: columns which contain dates.
- free string: columns which contain multiple string values.
- useless: constant or integer values that do not come under any of the category.

7. We can also change the type to meet our needs and requirements. For example, the column named Pregnancies is labelled neither as continuous nor as categorical and since the values in the column are single integer values we can make them into categorical values.

```
data_clean = dabl.clean(data, type_hints={"Pregnancies": "categorical"})
```

and remove/reset type_hints to default

```
data_clean = dabl.clean(data,type_hints=None)
```

8. How to determine which column(s) is continuous, low card int, categorical, etc…? We can use **detect_types()** function

```
types = dabl.detect_types(data_clean)
types
```

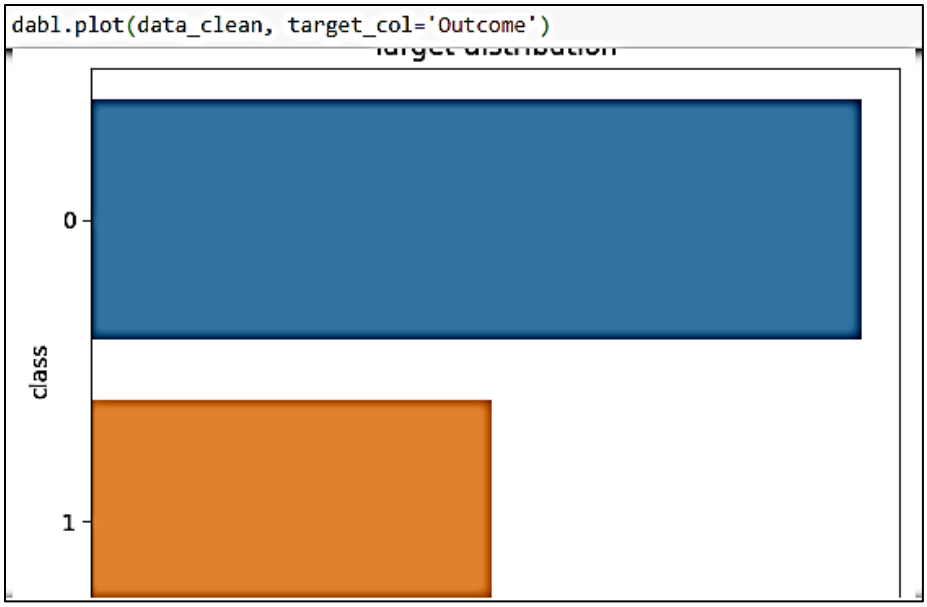
	continuous	dirty_float	low_card_int	categorical	date	free_string	useless
Pregnancies	False	False	True	False	False	False	False
Glucose	True	False	False	False	False	False	False
BloodPressure	True	False	False	False	False	False	False
SkinThickness	True	False	False	False	False	False	False
Insulin	True	False	False	False	False	False	False
BMI	True	False	False	False	False	False	False
DiabetesPedigreeFunction	True	False	False	False	False	False	False
Age	True	False	False	False	False	False	False
Outcome	False	False	False	True	False	False	False

9. Perform Univariate and Bivariate analysis using dabl (to be discussed in more detail in a later section)

```
dabl.plot(data_clean, target_col='Outcome')
```

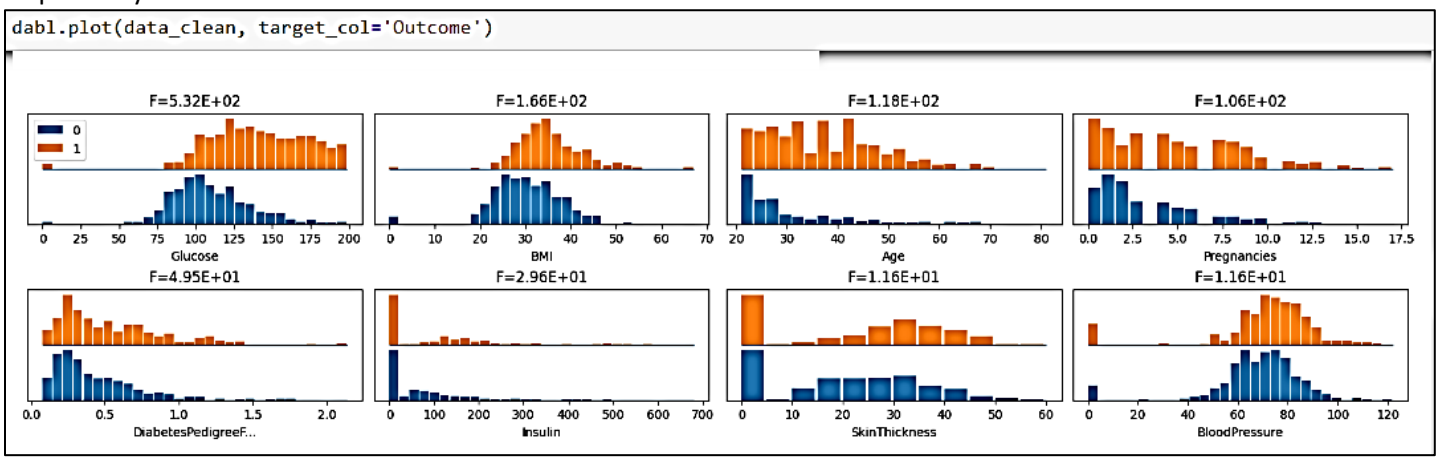
Dabl first automatically identifies and drops any outliers present in the dataset. It then identifies what type of data is present in the target (whether is categorical or continuous) and then displays the appropriate graph. Since ours is a categorical target, the output is a bar graph containing the count of 0s and 1s. Dabl also calculates and displays Linear discriminant analysis scores for the training set.

```
dabl.plot(data_clean, target_col='Outcome')
```



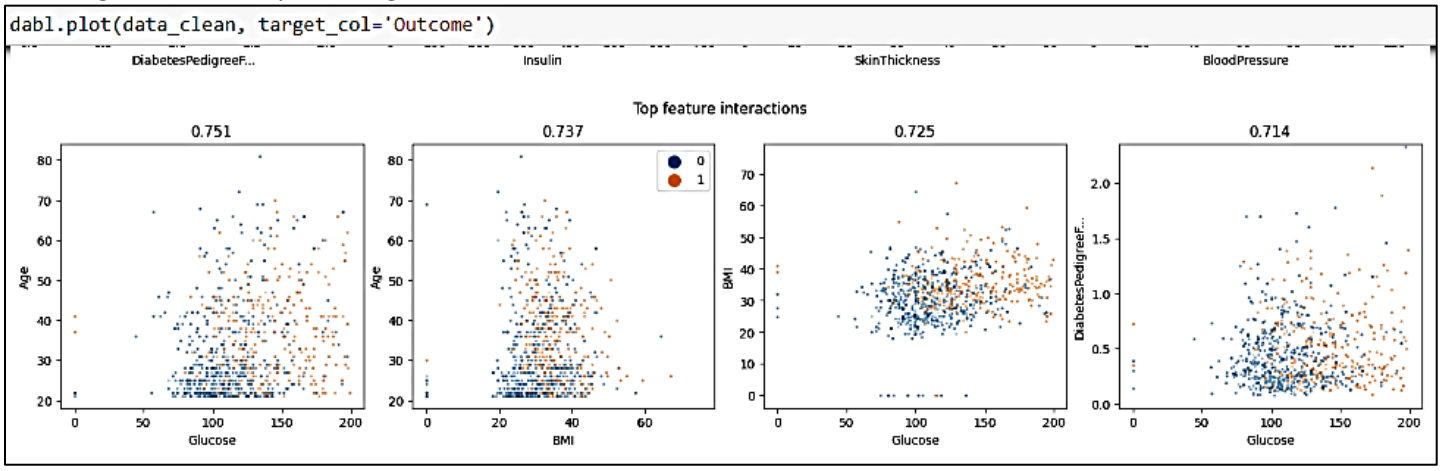
The next graph is to identify the distribution of each feature against our target. As you can see below, each feature is plotted as a histogram against our target and the number of features that lead to 1 and 0 are shown in orange and blue respectively.

```
dabl.plot(data_clean, target_col='Outcome')
```

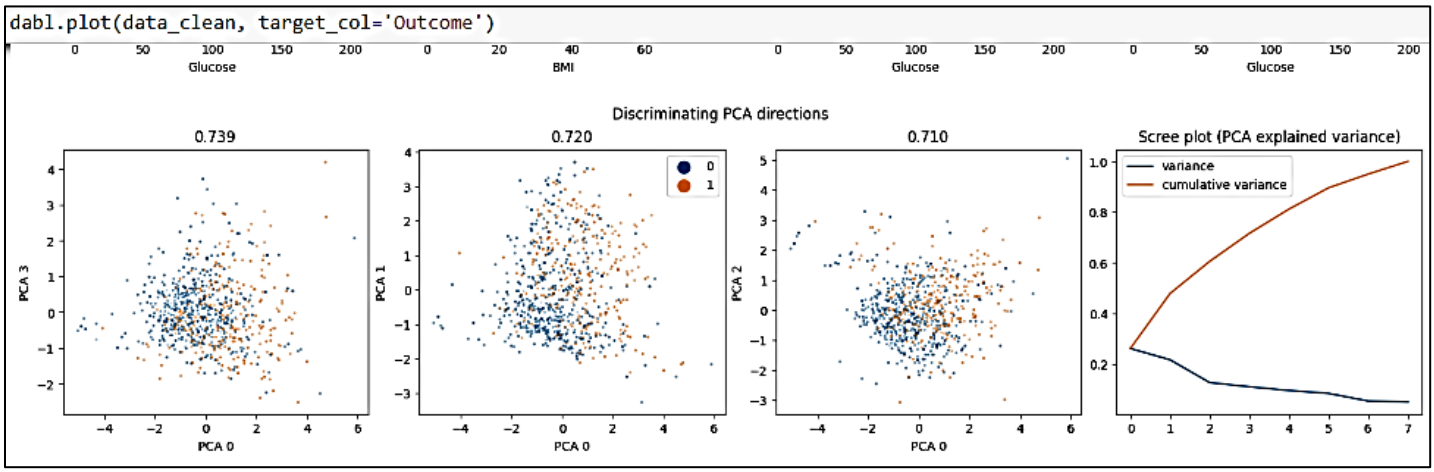


The next graph is the scatter plot of the different combinations of data that exists in the dataset. For example, the feature glucose will be plotted against all the other columns and the distribution.

```
dabl.plot(data_clean, target_col='Outcome')
```



In order to increase the efficiency and speed of training, dabl automatically performs PCA on the dataset and also shows the distribution to us. The next graph is the discriminant PCA graph for all the features in the dataset. It also displays the variance and cumulative variance for the dataset.

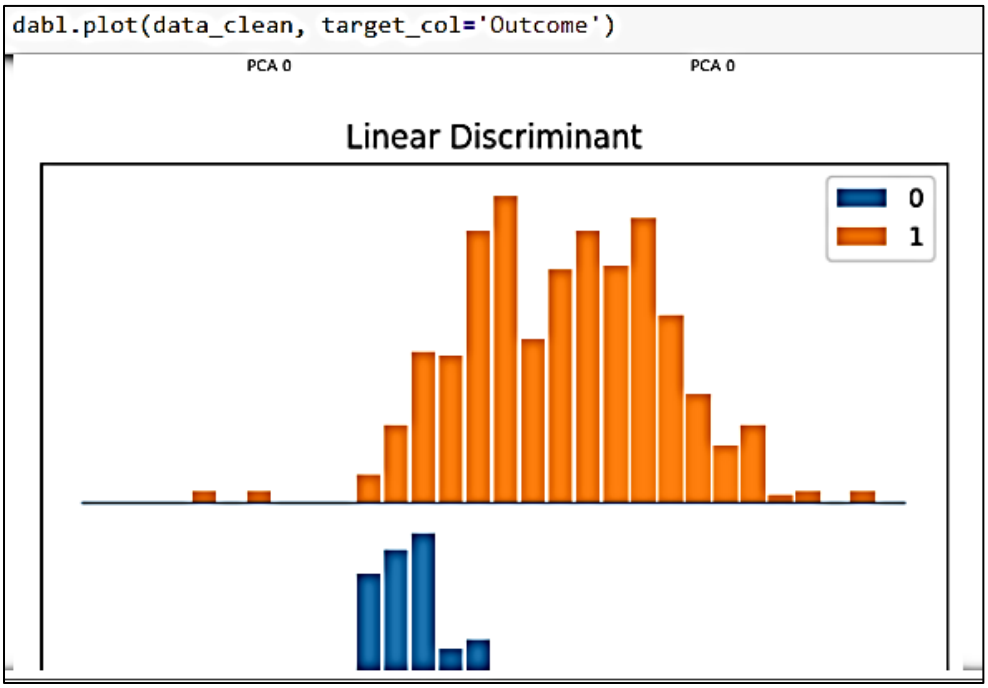


Note:

Principal Component Analysis, or PCA, is a dimensionality-reduction method that is often used to reduce the dimensionality of large data sets, by transforming a large set of variables into a smaller one that still contains most of the information in the large set.

Reducing the number of variables of a data set naturally comes at the expense of accuracy, but the trick in dimensionality reduction is to trade a little accuracy for simplicity. Because smaller data sets are easier to explore and visualize and make analyzing data much easier and faster for machine learning algorithms without extraneous variables to process.

The final graph displayed here is the linear discriminant analysis which is done by combining the features against the target.



10. Model development (*optional)

    Dabl intends to speed up the process of model training and provides a low code environment to train models. It takes up very little time and memory to train models using dabl. But, as mentioned earlier, this is a recently developed library and provides basic methods for machine learning training. Here I will be using a simple classifier model to train the diabetes dataset.

```
classifier = dabl.SimpleClassifier(random_state=0)
x = data_clean.drop('Outcome', axis=1)
y = data_clean.Outcome
classifier.fit(x, y)
```

Exploring the Sweetviz library

Sweetviz library is an open-source python library that generates beautiful, high-density visualizations. SweetViz aims to generate a quick analysis of target characteristics, training vs testing data, and other such data characterization tasks.

1. Install sweetviz library

```
\> pip install sweetviz
```

2. Sweetviz common functions

```
analyze() :- To analyze a single dataframe.
compare() :- To compare two data frames usually training and testing dataset.
compare_intra() :- To compare the two subsets of the same dataframe.
```

3. For the activity, we will use penguins_cleaned.csv dataset so upload them.
4. Import libraries

```
import sweetviz as sv
import pandas as pd
```

5. Read the dataset

```
#read dataset
penguins = pd.read_csv('penguins_cleaned.csv')
```

6. Here we will be trying to discover facts around the species of penguins from our dataset. So we will be splitting the dataset into X and Y, where Y will contain all the information related to the penguin species and X will contain the other information.

```
# Separating X and y
X = penguins.drop('species', axis=1)
y = penguins['species']
```

7. Displaying data from X

```
#displaying data from X
X
```

```
#displaying data from X
X
```

	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Torgersen	39.1	18.7	181	3750	male
1	Torgersen	39.5	17.4	186	3800	female
2	Torgersen	40.3	18.0	195	3250	female
3	Torgersen	36.7	19.3	193	3450	female
4	Torgersen	39.3	20.6	190	3650	male
...	...	...	...	...	...	...
328	Dream	55.8	19.8	207	4000	male
329	Dream	43.5	18.1	202	3400	female
330	Dream	49.6	18.2	193	3775	male
331	Dream	50.8	19.0	210	4100	male
332	Dream	50.2	18.7	198	3775	female

333 rows × 6 columns

8. Displaying data from y

```
#displaying data from Y
y
```

```
0         Adelie
1         Adelie
2         Adelie
3         Adelie
4         Adelie
         ...
328    Chinstrap
329    Chinstrap
330    Chinstrap
331    Chinstrap
332    Chinstrap
Name: species, Length: 333, dtype: object
```

9. Review the entire dataset

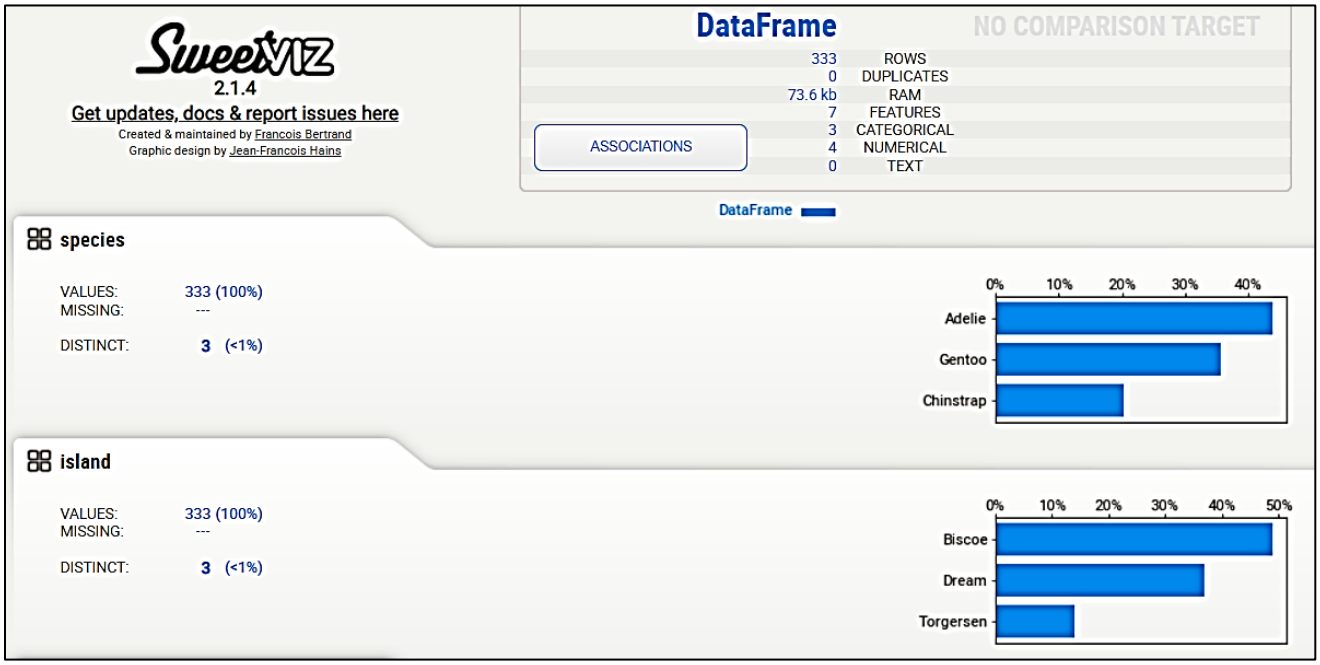
```
#review the entire dataset
penguins
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181	3750	male
1	Adelie	Torgersen	39.5	17.4	186	3800	female
2	Adelie	Torgersen	40.3	18.0	195	3250	female
3	Adelie	Torgersen	36.7	19.3	193	3450	female
4	Adelie	Torgersen	39.3	20.6	190	3650	male
...	...	...	...	...	...	...	...
328	Chinstrap	Dream	55.8	19.8	207	4000	male
329	Chinstrap	Dream	43.5	18.1	202	3400	female
330	Chinstrap	Dream	49.6	18.2	193	3775	male
331	Chinstrap	Dream	50.8	19.0	210	4100	male
332	Chinstrap	Dream	50.2	18.7	198	3775	female

333 rows × 7 columns

10. Perform EDA to dataset

```
#creating a EDA report
analyze_report = sv.analyze(penguins)
analyze_report.show_html('analyze.html')
#analyze_report.show_html('analyze.html', open_browser=False)
```



11. Now Lets Move Ahead and create a comparison report of our Train versus Test Dataset:

```
#splitting into train and test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
#displaying Train set
X_train
```

	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
38	Dream	44.1	19.7	196	4400	male
243	Biscoe	51.1	16.5	225	5250	male
118	Torgersen	35.2	15.9	186	3050	female
90	Dream	38.1	18.6	190	3700	female
303	Dream	46.9	16.6	192	2700	female
...	...	...	...	...	...	...
176	Biscoe	42.8	14.2	209	4700	female
30	Dream	39.2	21.1	196	4150	male
1	Torgersen	39.5	17.4	186	3800	female
83	Dream	38.9	18.8	190	3600	female
254	Biscoe	49.8	15.9	229	5950	male

266 rows × 6 columns

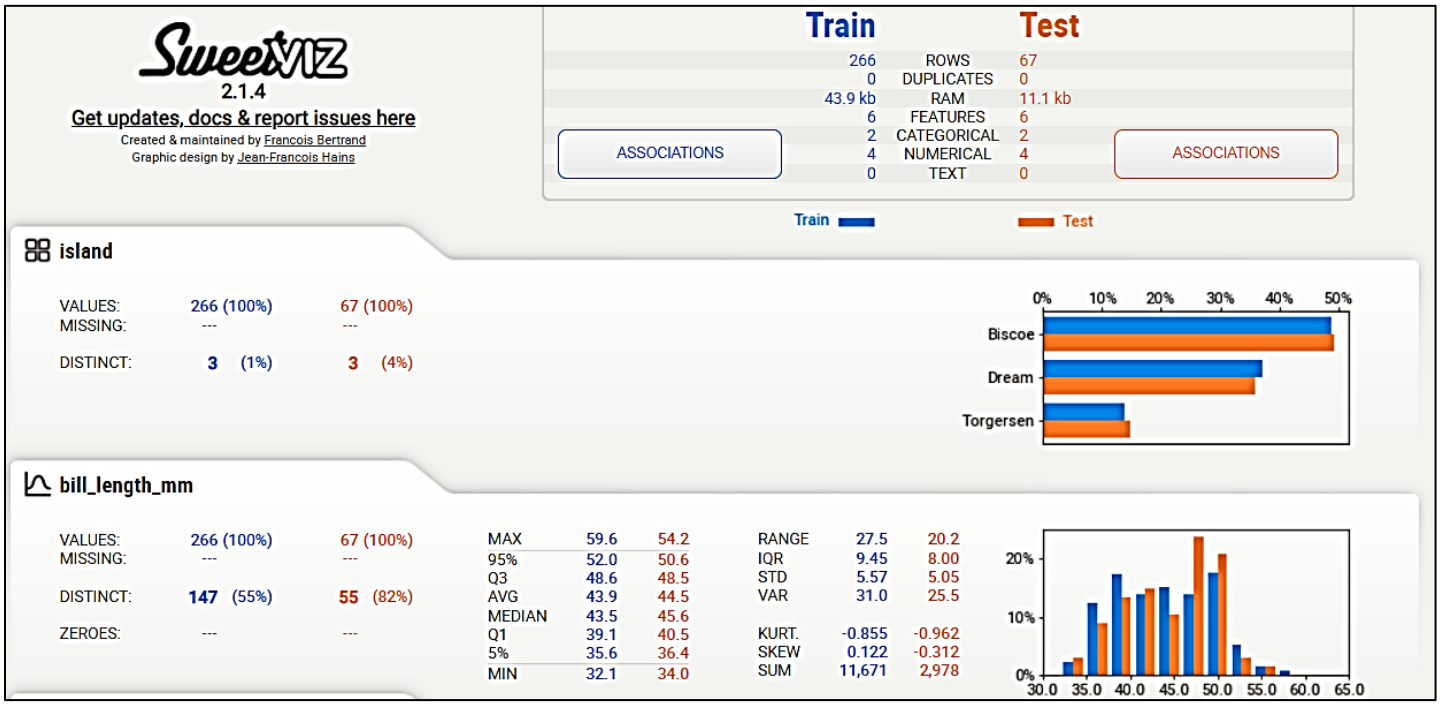
```
#displaying Test set
X_test
```

	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
193	Biscoe	44.9	13.3	213	5100	female
273	Dream	46.0	18.9	195	4150	female
115	Torgersen	37.7	19.8	198	3500	male
311	Dream	50.1	17.9	190	3400	female
44	Biscoe	39.6	17.7	186	3500	female
...	...	...	...	...	...	...
216	Biscoe	48.2	15.6	221	5100	male
119	Torgersen	40.6	19.0	199	4000	male
260	Biscoe	47.2	13.7	214	4925	female
171	Biscoe	46.1	15.1	215	5100	male
37	Dream	36.0	18.5	186	3100	female

67 rows × 6 columns

12. Creating the comparison report between Train versus Test,

```
#compare test vs train
compare_report = sv.compare([X_train, 'Train'], [X_test, 'Test'])
compare_report.show_html('compare.html')
#compare_report.show_html('compare.html', open_browser=False)
```



Data Analytics Conclusion or Predictions

Your conclusion will tie together all of the work you have done.

First, summarize the entire process in 2-4 sentences. What were you trying to show with your experiment? Refer back to your hypothesis. Does your data support your hypothesis, or not? If your data does not support your hypothesis, then explain why not.

Use your background research to help explain your results.

Did you find a connection between your independent variable and your dependent variable. For example, as the time got longer, did your results increase or decrease? Or were they not consistent? Was your procedure effective?

Finish your conclusion with 1-2 sentences explaining how you would improve this project if you were to do it again. Would you conduct more trials? Would you change the amount of your independent variable? Did your results give you ideas of another question to expand this project?

Start with an Outline
If you start writing without having a clear idea of what your data analysis report is going to include, it may get messy. Start the report by writing an outline first. Plan the structure and contents of each section first to make sure you've covered everything.

Make a Selection of Vital KPIs
Don't overwhelm the audience by including every single metric there is. You can discuss your whole dashboard in a meeting with your team, but if you're creating data analytics reports for other departments or the executives, it's best to focus on the most relevant KPIs that demonstrate the data important for the overall business performance.

Pick the Right Charts for Appealing Design
If you're showing historical data – for instance, how you've performed now compared to last month – it's best to use timelines or graphs. For other data, pie charts or tables may be more suitable. Make sure you use the right data visualization to display your data accurately and in an easy-to-understand manner.

Use a Narrative
Just exporting your data into a spreadsheet isn't a data analytics report. The fact that you're dealing with data may sound too technical, but actually, your report should tell a story about your performance. What happened on a specific day? Did your organic traffic increase or suddenly drop? Why? And more. There are a lot of questions to answer and you can put all the responses together in a coherent, understandable narrative.

Organize the Information
Before you start writing or building your dashboard, choose how you're going to organize your data. Are you going to talk about the most relevant and general ones first? It may be the best way to start the report – the best practices typically involve starting with more general information and then diving into details if necessary.

Include a Summary
Some people in your audience won't have the time to read the whole report, but they'll want to know about your findings. Besides, a summary at the beginning of your data analytics report will help the reader get familiar with the topic and the goal of the report. And a quick note: although the summary should be placed at the beginning, you usually write it when you're done with the report. When you have the whole picture, it's easier to extract the key points that you'll include in the summary.

Careful with Your Recommendations
Your communication skills may be critical in data analytics reports. Know that some of the results probably won't be satisfactory, which means that someone's strategy failed. Make sure you're objective in your recommendations and that you're not looking for someone to blame. Don't criticize, but give suggestions on how things can be improved. Being solution-oriented is much more important and helpful for the business.

Double-Check Everything
The whole point of using data analytics tools and data, in general, is to achieve as much accuracy as possible. Avoid manual mistakes by proofreading your report when you finish, and if possible, give it to another person so they can confirm everything's in place.

Use Interactive Dashboards
Using the right tools is just as important as the contents of your data analysis. The way you present it can make or break a good report, regardless of how valuable the data is. That said, choose a great reporting tool that can automatically

update your data and display it in a visually appealing manner. Make sure it offers a streamlined dashboard that you can also customize depending on the purpose of the report.

Data Types for Plotting

Quantitative versus Qualitative Data
The distinction between quantitative and qualitative data is the most fundamental way to divide types of data. Is the characteristic something you can objectively measure with numbers or not?

- Quantitative: The information is recorded as numbers and represents an objective measurement or a count. Temperature, weight, and a count of transactions are all quantitative data. Analysts also refer to this type as numerical data.

- Qualitative: The information represents characteristics that you do not measure with numbers. Instead, the observations fall within a countable number of groups. In fact, this type of variable can capture information that isn't easily measured and can be subjective. Taste, eye color, architectural style, and marital status are all types of qualitative variables.

Types of Quantitative Data: Continuous and Discrete
When you can represent the information you're gathering with numbers, you are collecting quantitative data. This class encompasses two categories.

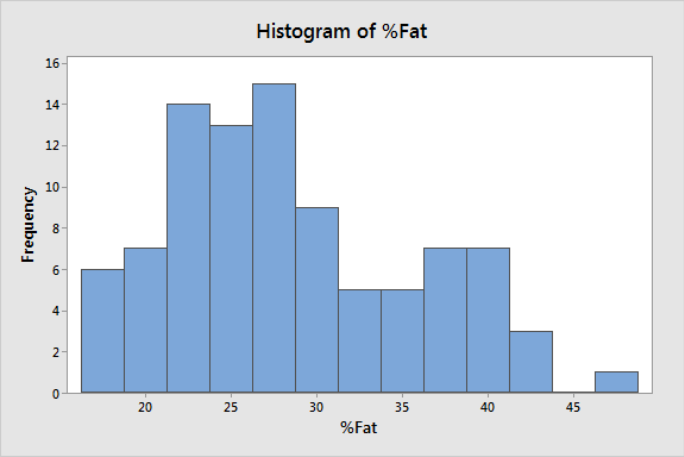
Continuous data

Continuous variables can take on any numeric value, and it can be meaningfully divided into smaller increments, including fractional and decimal values. There are an infinite number of possible values between any two values. Typically, you measure continuous variables on a scale. For example, when you measure height, weight, and temperature, you have continuous data.

With continuous variables, you can assess measures of central tendency and variability, such as the mean, median, distribution, range, and standard deviation. For example, the mean height in the U.S. is 5 feet 9 inches for men and 5 feet 4 inches for women.

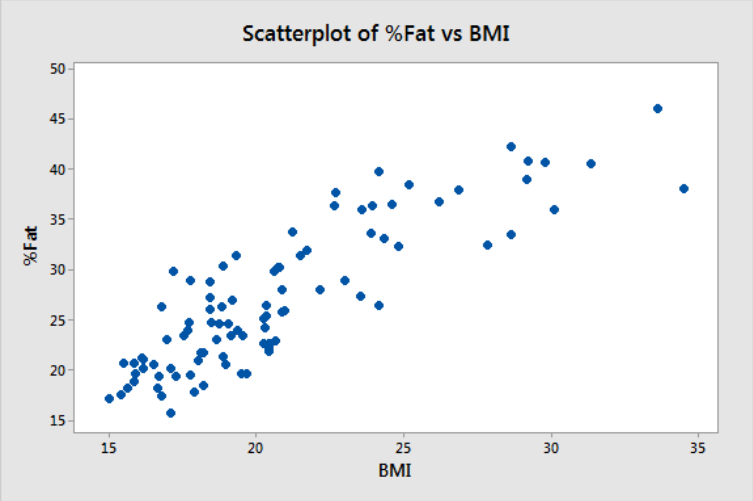
How to graph continuous data

Histograms are a standard way to graph continuous variables because they show the distribution of the values. The histogram below helps you determine whether the distribution of body fat percentage values for adolescent girls are symmetric or skewed; understand the range of values; and identify where the most common values fall.

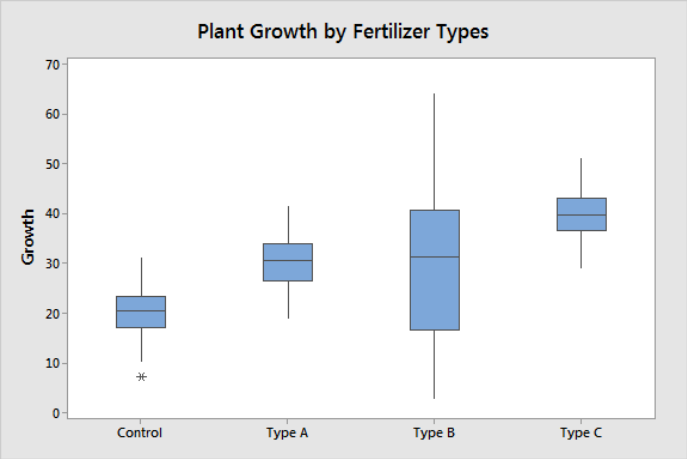


%Fat
23.9
28.8
32.4
25.8
22.5

When you have two continuous variables, you can graph them using a scatterplot. The scatterplot shows how the body fat percentage tends to rise as BMI increases. Use correlation to assess the strength of this relationship or regression analysis to derive the equation for the line that provides the best fit for these data.



When you have continuous variables that are divided into groups, you can use a boxplot to display the central tendency and spread of each group. Fertilizer Type C is associated with the highest plant growth while Type B produces the greatest variability.

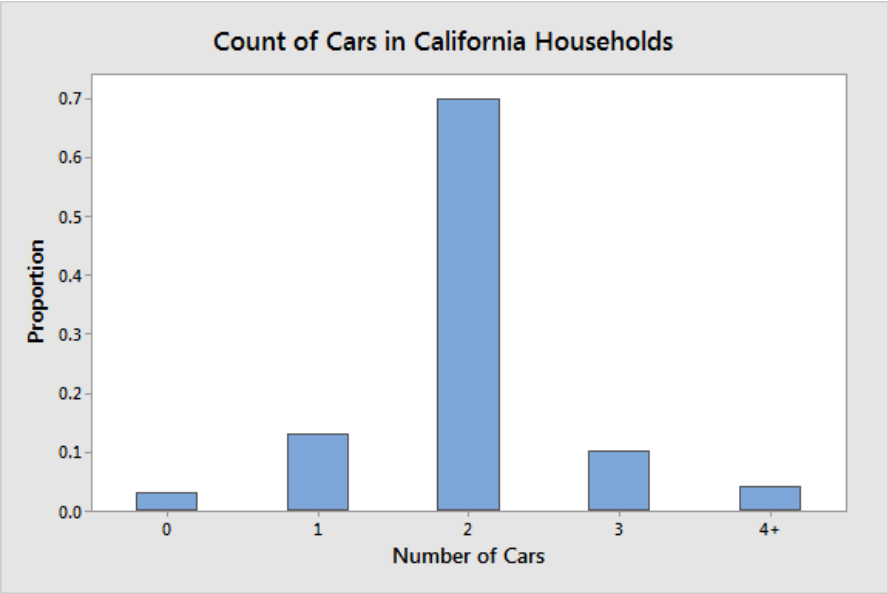


Discrete data

Discrete quantitative data are a count of the presence of a characteristic, result, item, or activity. These measures cannot be meaningfully divided into smaller increments. For example, a single household can have 1 or 2 cars, but it cannot have 1.6. There are a finite number of possible values that you can record for an observation.

With discrete variables, you can calculate and assess a rate of occurrence or a summary of the count, such as the mean, sum, and standard deviation. For example, U.S. households had an average of 2.11 vehicles in 2014.

Bar charts are a standard way to graph discrete variables. Each bar represents a distinct value, and the height represents its proportion in the entire sample.



Car Count
2
1
2
2
3

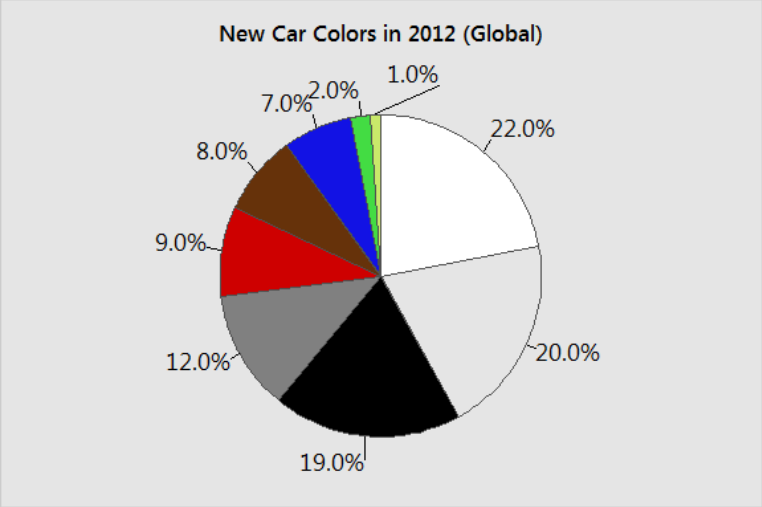
<u>Qualitative Data: Categorical, Binary, and Ordinal</u>

When you record information that categorizes your observations, you are collecting qualitative data. There are three types of qualitative variables—categorical, binary, and ordinal. With these data types, you're often interested in the proportions of each category. Consequently, bar charts and pie charts are conventional methods for graphing qualitative variables because they are useful for displaying the relative percentage of each group out of the entire sample.

In cases where you have a choice about recording a characteristic as a continuous or qualitative variable, the best practice is to record the continuous data because you can learn so much more.

Categorical data

Categorical data have values that you can put into a countable number of distinct groups based on a characteristic. For a categorical variable, you can assign categories, but the categories have no natural order. Analysts also refer to categorical data as both attribute and nominal variables. For example, college major is a categorical variable that can have values such as psychology, political science, engineering, biology, etc.

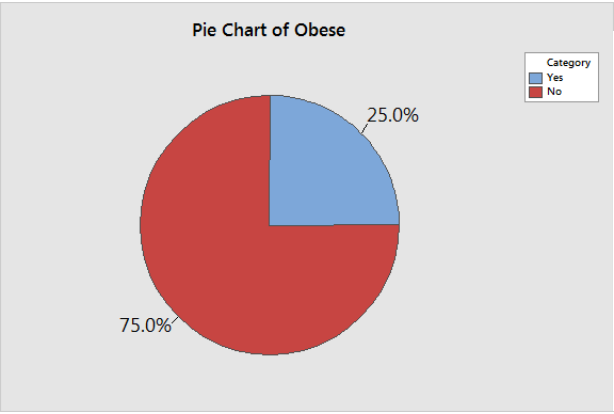


Color
White
Silver
Black
Gray
Red

Binary data

Binary data can have only two values. If you can place an observation into only two categories, you have a binary variable. Statisticians also refer to binary data as both dichotomous and indicator variables. For example, pass/fail, male/female, and the presence/absence of a characteristic are all binary data.

Binary variables are helpful for calculating proportions or percentages, such as the proportion of defective products in a sample. You just take the number of faulty products and divide by the sample size.



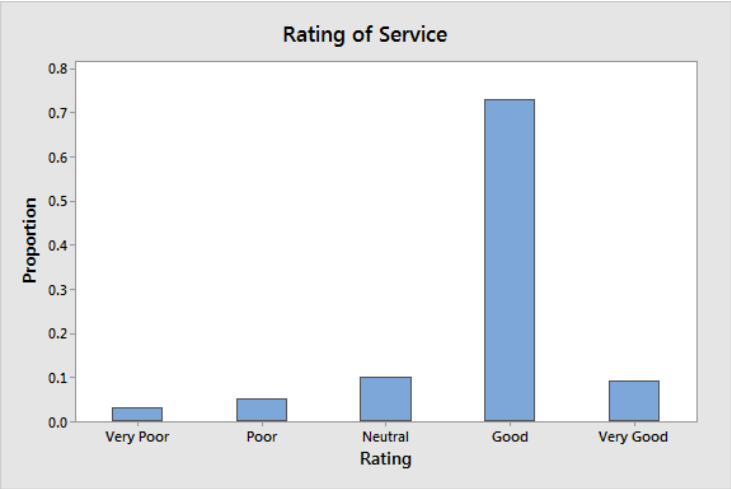
Obese
Yes
No
No
Yes
No

Ordinal data

Ordinal data have at least three categories, and the categories have a natural order. Examples of ordinal variables include overall status (poor to excellent), agreement (strongly disagree to strongly agree), and rank (such as sporting teams).

Analysts often consider ordinal variables to have a combination of qualitative and quantitative properties. Analysts often represent ordinal variables using numbers, such as a 1-5 Likert scale that measures satisfaction. In number form, you can calculate average scores as with quantitative variables. However, the numbers have limited usefulness because the differences between ranks might not be constant.

For example, first, second, and third in a race are ordinal data. The difference in time between first and second place might not be the same the difference between second and third place.



Rating
Very Poor
Poor
Neutral
Good
Very Good

Lesson 03 - Data Analytics Overview

Introduction to Statistics

Statistics and Probability are the branches of Mathematics concerned with the laws governing random events. It includes collection, analysis, interpretation, and display of numerical data.

Common applications of statistics and probability :

- ✓ Risk assessments
- ✓ Buying behaviors
- ✓ Weather Forecasting
- ✓ Politics
- ✓ Insurance
- ✓ Quantum Physics
- ✓ Making predictions (likelihood of winning the Lottery)
- ✓ Heavily used in Machine Learning and Artificial Intelligence

Notes on probability:

- Probability can be expressed from range of 0-100% (0 to 1)
- Probability = number of ways it can happen / total number of outcomes
- Example: getting a “6” from a dice roll

Number of ways it can happen: 1 there is only one “6” on the 6 faces of a dice
Total number of outcomes: 6
Probability = 1/6 or 0.166 or 16.6%

- Example: getting a king in a deck of cards

Number of ways it can happen: 4 there 4 kings in a deck
Total number of outcomes: 52 there are 52 cards in a deck
Probability = 4/52 or 1/13 or 0.0769 or 7.7%

- Some terms in probability

Experiment	planned operation carried out in a controlled environment
Outcome	result of an experiment
Sample Space	set of all possible outcomes
Equally likely	each outcome occurs with equal probability
"or" event	outcome is in A or B or is in both A and B

Example:
(3 sets)
A={1,2,3,4,5}
B={4,5,6,7,8}
A or B={1,2,3,4,5,6,7,8}

4 and 5 is not listed twice in any of the sets

Independent and Dependent Events

Independent Events – each event is not affected by other events (ex: dice throw)

Dependent Events – events affected by previous or other events

Example:

2 blue and 3 red marbles in a bag
Chance of getting blue is 2/5

But after taking one out, the chances changes!

Concept of Replacement
If we replace/return the marbles we take from the bag each time, then chances do not change and the events are independent.

Conditional Probability

Conditional Probability is probability of one event occurring with some relationship to another event.

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Where:
P(A|B) – the conditional probability; the probability of event A occurring given that event B has already occurred
P(A ∩ B) – the joint probability of events A and B; the probability that both events A and B occur
P(B) – the probability of event B

The formula above is applied to the calculation of the conditional probability of events that are neither independent nor mutually exclusive.

Bayes' Theorem

Another way of calculating conditional probability is by using the Bayes' theorem. The theorem can be used to determine the conditional probability of event A, given that event B has occurred, by knowing the conditional probability of event B, given the event A has occurred, as well as the individual probabilities of events A and B. Mathematically, the Bayes' theorem can be denoted in the following way:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Example 1:

P(Fire) -how often we see dangerous fire
P(Smoke) -how often we see smoke
P(Fire|Smoke) -how often there is dangerous fire when when we see smoke
P(Smoke|Fire) -how often we can see smoke when there is dangerous fire

Dangerous fires are rare (1%)
Smokes are common (10%) due to barbeques
90% of dangerous fire makes smoke

Let us discover probability of dangerous fire when there is smoke:
P(Fire|Smoke) = P(Fire) P(Smoke|Fire)P(Smoke)
P(Fire|Smoke) = 1% x 90%10%
P(Fire|Smoke) = 9%

Example 2:

You are planning a picnic today, but the morning is cloudy

- ✓ Oh no! 50% of all rainy days start off cloudy!
- ✓ But cloudy mornings are common (about 40% of days start cloudy)
- ✓ And this is usually a dry month (only 3 of 30 days tend to be rainy, or 10%)
- ✓ What is the chance of rain during the day?

We will use Rain to mean rain during the day, and Cloud to mean cloudy morning.

The chance of Rain given Cloud is written P(Rain|Cloud)

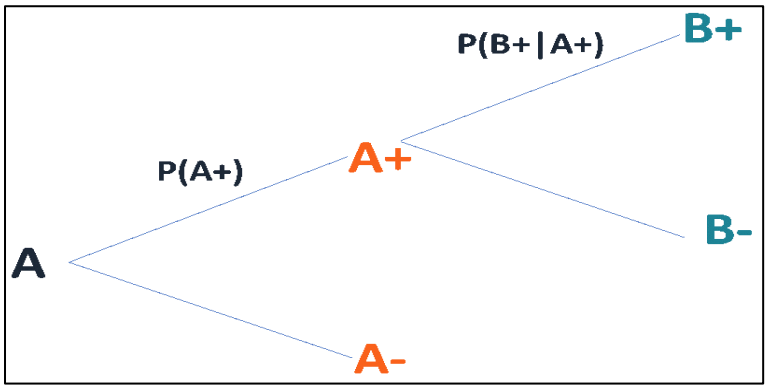
So let's put that in the formula:

P(Rain|Cloud) = P(Rain) P(Cloud|Rain)P(Cloud)

P(Rain) is Probability of Rain = 10%
P(Cloud|Rain) is Probability of Cloud, given that Rain happens = 50%
P(Cloud) is Probability of Cloud = 40%
P(Rain|Cloud) = 0.1 x 0.5/0.4 = .125

Or a 12.5% chance of rain. Not too bad, let's have a picnic!

Finally, conditional probabilities can be found using a tree diagram. In the tree diagram, the probabilities in each branch are conditional.



Conditional Probability for Independent Events

Two events are independent if the probability of the outcome of one event does not influence the probability of the outcome of another event. Due to this reason, the conditional probability of two independent events A and B is:

P(A|B) = P(A) P(B|A) = P(B)

Conditional Probability for Mutually Exclusive Events

In probability theory, mutually exclusive events are events that cannot occur simultaneously. In other words, if one event has already occurred, another event cannot occur. Thus, the conditional probability of mutually exclusive events is always zero.

P(A|B) = 0 P(B|A) = 0

Statistical and Non-statistical Analysis

- Statistical questions require collection of data and the answers have variability (there is more than 1 possible way to answer the question). A statistical question is a question that can be answered by collecting data that vary like “How old are the students in my school?” is a statistical question.

- Non-Statistical questions have answers with no variability (there is only 1 answer). For example, “How old am I?”

Example 1: Deciding Whether a Question Is Statistical Determine whether the following is a statistical question: How tall are you? Answer To answer this question, we just have to record one height. There will be no variability in the data which means that it is not a statistical question.

Example 2: Deciding Whether a Question Is Statistical
Determine whether the following is a statistical question: What are the heights of the players of your favorite soccer team?

Answer
The answer to this question would be a list of the heights of the players on the team. We expect that there will be variability in the data we collect because not everyone will be the same height. Therefore, this is a statistical question.

Example 3: Deciding Whether a Question Is Statistical
Determine whether the following is a statistical question: In general, how tall are humans?

Answer
An answer to this question could be the mean of the heights of a sample of humans. Therefore, to answer it, we need to collect the data which tells us the heights of different people. Since people are of different heights, there will be variability in the data we collect. Hence, it is a statistical question.

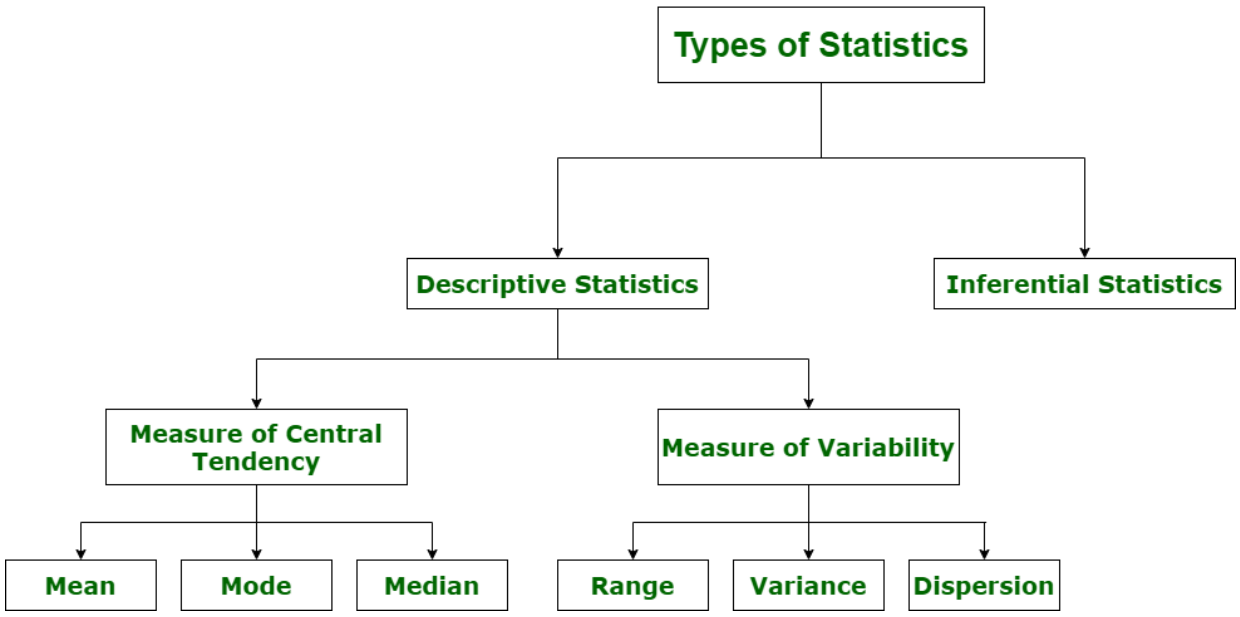
Example 4: Deciding Whether a Question Is Statistical
Determine whether the following is a statistical question: What is your favorite subject?

Answer
The answer to this question is a single subject; we do not have to collect data with variability to answer the question. Therefore, it is not a statistical question.

Example 5: Deciding Whether a Question Is Statistical
Determine whether the following is a statistical question: How do students travel to school?

Answer
To answer this question, we would have to collect data about how students travel to school: by car, bike, or bus for example. Since we expect the students' answers to vary, this is an example of a statistical question.

Major Categories of Statistics



1. Descriptive Statistics :

Descriptive statistics uses data that provides a description of the population either through numerical calculation or graph or table. It provides a graphical summary of data. It is simply used for summarizing objects, etc. There are two categories in this as following below.

(a). Measure of central tendency –

Measure of central tendency is also known as summary statistics that is used to represents the center point or a particular value of a data set or sample set.

In statistics, there are three common measures of central tendency as shown below:

(i) Mean :

It is measure of average of all value in a sample set.

For example,

Cars	Mileage	Cylinder
Swift	21.3	3
Verna	20.8	2
Santro	19	5

$$\text{Mean (m)} = \frac{\text{Sum of all the terms}}{\text{Total no. of terms}}$$

$$m = \frac{21.3 + 20.8 + 19}{3}$$

$$= 20.366$$

(ii) Median :

It is measure of central value of a sample set. In these, data set is ordered from lowest to highest value and then finds exact middle.

For example,

Cars	Mileage	Cylinder
Swift	21.3	3
Verna	20.8	2
Santro	19	5
i 20	15	4

Ordering the set from lowest to highest = 15 19 20.8 21.3

$$\text{Median} = \frac{19 + 20.8}{2}$$

Median = 23.5

(iii) Mode :

It is value most frequently arrived in sample set. The value repeated most of time in central set is actually mode.

For example,

2	3	4	2	4	6	4	7	7	4	2	4

Mode = 4

(b). Measure of Variability –
Measure of Variability is also known as measure of dispersion and used to describe variability in a sample or population. In statistics, there are three common measures of variability as shown below:

(i) Range :
It is given measure of how to spread apart values in sample set or data set.

Range = Maximum value - Minimum value

(ii) Variance :
It simply describes how much a random variable defers from expected value and it is also computed as square of deviation.

$S2= \sum ni=1 [(xi - \bar{x})2 \div n]$

In these formula, n represent total data points, $\bar{x}$ represent mean of data points and xi represent individual data points.

(iii) Dispersion :
It is measure of dispersion of set of data from its mean.

$\sigma= \sqrt{} (1\div n) \sum ni=1 (xi - \mu)2$

2. Inferential Statistics :
Inferential Statistics makes inference and prediction about population based on a sample of data taken from population. It generalizes a large dataset and applies probabilities to draw a conclusion. It is simply used for explaining meaning of descriptive stats. It is simply used to analyze, interpret result, and draw conclusion. Inferential Statistics is mainly related to and associated with hypothesis testing whose main target is to reject null hypothesis.

Hypothesis testing is a type of inferential procedure that takes help of sample data to evaluate and assess credibility of a hypothesis about a population. Inferential statistics are generally used to determine how strong relationship is within sample. But it is very difficult to obtain a population list and draw a random sample.

Inferential statistics can be done with help of various steps as given below:

1. Obtain and start with a theory.
2. Generate a research hypothesis.
3. Operationalize or use variables
4. Identify or find out population to which we can apply study material.
5. Generate or form a null hypothesis for these population.
6. Collect and gather a sample of children from population and simply run study.
7. Then, perform all tests of statistical to clarify if obtained characteristics of sample are sufficiently different from what would be expected under null hypothesis so that we can be able to find and reject null hypothesis.

Types of inferential statistics –
Various types of inferential statistics are used widely nowadays and are very easy to interpret. These are given below:

- One sample test of difference/One sample hypothesis test
- Confidence Interval
- Contingency Tables and Chi-Square Statistic
- T-test or Anova
- Pearson Correlation
- Bi-variate Regression
- Multi-variate Regression

Statistical Analysis Considerations

Pros:
1. Less time consuming: Because it is secondary data it is usually cheap and is less time consuming because someone else has compiled it.

2. Patterns and correlations are clear and visible: Statistical data is data that has already been analyzed and therefore the patterns and correlations have already been done and are clear and visible.

3. Taken from large samples so the generalizability is high: Statistical data is data that has been gathered from very large data samples. This means that the generalization is higher.

4. Can be used and re-used to check different variables: Statistical data is data that can be used and reused. It does not require one time used as the same data can be used to make a different decision.

5. Can be imitated: Statistical data can be imitated to check changes that increase the reliability and representativeness of the data.

6. Fast: Statistical data is data that can be analyzed relatively quickly and with much ease as compared to other forms of data.
Standardization: Statistical information is collected in a standardized way which gives the data meaning.

7. Straight forward: Statistical data is usually straightforward to analyze. It is data that has already been synthesized and therefore very little analysis is required.

8. Reliable: They are often required and respected by decision-makers within the institution and beyond eg funders, government. This makes them reliable and accurate.

9. Quality data: They support qualitative data obtained from questionnaires, interviews, etc with 'hard facts.

10. Benchmarking: Statistical data is useful for benchmarking purposes. They can be used to make comparisons and set new standards and targets within the organization or in a project.

Cons:
1. Unverified: The researcher cannot check the validity and can't find a mechanism for a causation theory only drawing patterns and correlations from the data. This means the researcher has limited options to verify the validity and authenticity of the data.

2. Can be misinterpreted: Statistical data is often secondary data which means that it can be easily be misinterpreted. This leaves the researcher vulnerable to distortion of information without the ability to make confirmations.

3. It can be manipulated: Statistical data is open to abuse it can be manipulated and phrased to show the point the researcher wants to show. This makes the data wanting in objectivity and more subjective in nature.

4. Because this is often secondary data it is hard to access and check: Statistical data is mostly secondary data that can only be accessed. It may be quite difficult to check and verify the data because the primary source of the data is unavailable.

5. It is not appropriate: Statistical data is not an appropriate method to understand issues in great depth and identify ways to solve the problems highlighted. This is because the data is collected from primary sources by an independent researcher.

6. Not ideal for evaluation: They are not suitable to evaluate user opinions, needs or satisfaction with services because they are subjective. The researcher cannot rely on statistics to measure the happiness or satisfaction of the client.

7. It is time-consuming: It may be time-consuming to arrange methods of data collection eg contact vendors, liaising with IT departments. This is because the data collection methods used in the primary research depend on the subjective perspective of the researcher.

8. Performance management: Statistical data cannot be used to measure performance management in an organization because it is outdated.

9. Decision-making: While statistical data can be used to make future inferences, it cannot be relied upon to make decisions in an organizational setting.

10. Comparison: Statistical data cannot be used to make comparisons with current data or future data because the methods of data collection and data analysis may not be known.

Population and Sample

- Population –It is actually a collection of set of individuals or objects or events whose properties are to be analyzed.
- Sample –It is the subset of a population.

Statistical Analysis Process

Step 1: Write your hypotheses and plan your research design
Step 2: Collect data from a sample
Step 3: Summarize your data with descriptive statistics
Step 4: Test hypotheses or make estimates with inferential statistics
Step 5: Interpret your results

Step 1: Write your hypotheses and plan your research design

To collect valid data for statistical analysis, you first need to specify your hypotheses and plan out your research design.

Writing statistical hypotheses

The goal of research is often to investigate a relationship between variables within a population. You start with a prediction, and use statistical analysis to test that prediction.

A statistical hypothesis is a formal way of writing a prediction about a population. Every research prediction is rephrased into null and alternative hypotheses that can be tested using sample data.

While the null hypothesis always predicts no effect or no relationship between variables, the alternative hypothesis states your research prediction of an effect or relationship.

Example: Statistical hypotheses to test an effect

- Null hypothesis: A 5-minute meditation exercise will have no effect on math test scores in teenagers.
- Alternative hypothesis: A 5-minute meditation exercise will improve math test scores in teenagers.

Example: Statistical hypotheses to test a correlation

- Null hypothesis: Parental income and GPA have no relationship with each other in college students.
- Alternative hypothesis: Parental income and GPA are positively correlated in college students.

Planning your research design

A research design is your overall strategy for data collection and analysis. It determines the statistical tests you can use to test your hypothesis later on.

- In an experimental design, you can assess a cause-and-effect relationship (e.g., the effect of meditation on test scores) using statistical tests of comparison or regression.
- In a correlational design, you can explore relationships between variables (e.g., parental income and GPA) without any assumption of causality using correlation coefficients and significance tests.
- In a descriptive design, you can study the characteristics of a population or phenomenon (e.g., the prevalence of anxiety in U.S. college students) using statistical tests to draw inferences from sample data.

Measuring variables

When planning a research design, you should operationalize your variables and decide exactly how you will measure them.

For statistical analysis, it's important to consider the level of measurement of your variables, which tells you what kind of data they contain:

- Categorical data represents groupings. These may be nominal (e.g., gender) or ordinal (e.g. level of language ability).
- Quantitative data represents amounts. These may be on an interval scale (e.g. test score) or a ratio scale (e.g. age).

Many variables can be measured at different levels of precision. For example, age data can be quantitative (8 years old) or categorical (young). If a variable is coded numerically (e.g., level of agreement from 1–5), it doesn't automatically mean that it's quantitative instead of categorical.

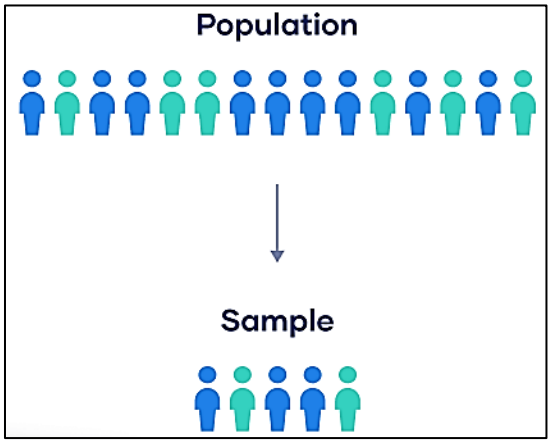
Identifying the measurement level is important for choosing appropriate statistics and hypothesis tests. For example, you can calculate a mean score with quantitative data, but not with categorical data.

Step 2: Collect data from a sample

Population vs sample

In most cases, it's too difficult or expensive to collect data from every member of the population you're interested in studying. Instead, you'll collect data from a sample.

Statistical analysis allows you to apply your findings beyond your own sample as long as you use appropriate sampling procedures. You should aim for a sample that is representative of the population.



Sampling for statistical analysis

There are two main approaches to selecting a sample.

- Probability sampling: every member of the population has a chance of being selected for the study through random selection.
- Non-probability sampling: some members of the population are more likely than others to be selected for the study because of criteria such as convenience or voluntary self-selection.

In theory, for highly generalizable findings, you should use a probability sampling method. Random selection reduces sampling bias and ensures that data from your sample is actually typical of the population. Parametric tests can be used to make strong statistical inferences when data are collected using probability sampling.

But in practice, it's rarely possible to gather the ideal sample. While non-probability samples are more likely to be biased, they are much easier to recruit and collect data from. Non-parametric tests are more appropriate for non-probability samples, but they result in weaker inferences about the population.

If you want to use parametric tests for non-probability samples, you have to make the case that:

- your sample is representative of the population you're generalizing your findings to.
- your sample lacks systematic bias.

Keep in mind that external validity means that you can only generalize your conclusions to others who share the characteristics of your sample. For instance, results from Western, Educated, Industrialized, Rich and Democratic samples (e.g., college students in the US) aren't automatically applicable to all non-WEIRD populations.

If you apply parametric tests to data from non-probability samples, be sure to elaborate on the limitations of how far your results can be generalized in your discussion section.

Step 3: Summarize your data with descriptive statistics

Once you've collected all of your data, you can inspect them and calculate descriptive statistics that summarize them.

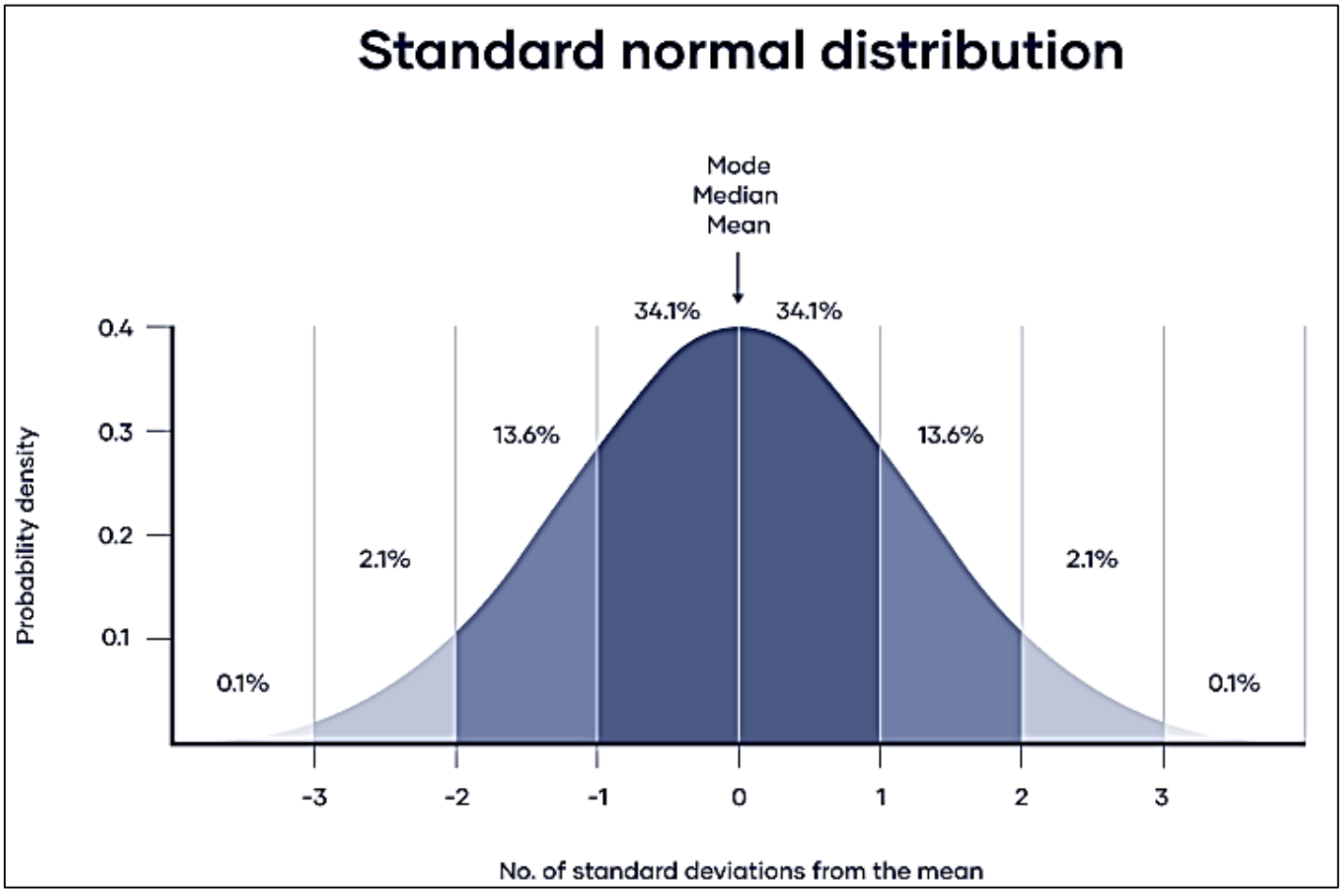
Inspect your data

There are various ways to inspect your data, including the following:

- Organizing data from each variable in frequency distribution tables.
- Displaying data from a key variable in a bar chart to view the distribution of responses.
- Visualizing the relationship between two variables using a scatter plot.

By visualizing your data in tables and graphs, you can assess whether your data follow a skewed or normal distribution and whether there are any outliers or missing data.

A normal distribution means that your data are symmetrically distributed around a center where most values lie, with the values tapering off at the tail ends.



In contrast, a skewed distribution is asymmetric and has more values on one end than the other. The shape of the distribution is important to keep in mind because only some descriptive statistics should be used with skewed distributions.

Extreme outliers can also produce misleading statistics, so you may need a systematic approach to dealing with these values.

Calculate measures of central tendency

Measures of central tendency describe where most of the values in a data set lie. Three main measures of central tendency are often reported:

- Mode: the most popular response or value in the data set.
- Median: the value in the exact middle of the data set when ordered from low to high.
- Mean: the sum of all values divided by the number of values.

However, depending on the shape of the distribution and level of measurement, only one or two of these measures may be appropriate. For example, many demographic characteristics can only be described using the mode or proportions, while a variable like reaction time may not have a mode at all.

Calculate measures of variability

Measures of variability tell you how spread out the values in a data set are. Four main measures of variability are often reported:

- Range: the highest value minus the lowest value of the data set.
- Interquartile range: the range of the middle half of the data set.
- Standard deviation: the average distance between each value in your data set and the mean.
- Variance: the square of the standard deviation.

Step 4: Test hypotheses or make estimates with inferential statistics

A number that describes a sample is called a statistic, while a number describing a population is called a parameter. Using inferential statistics, you can make conclusions about population parameters based on sample statistics.

Researchers often use two main methods (simultaneously) to make inferences in statistics.

- Estimation: calculating population parameters based on sample statistics.
- Hypothesis testing: a formal process for testing research predictions about the population using samples.

Estimation

You can make two types of estimates of population parameters from sample statistics:

- A point estimate: a value that represents your best guess of the exact parameter.
- An interval estimate: a range of values that represent your best guess of where the parameter lies.

Hypothesis testing

Using data from a sample, you can test hypotheses about relationships between variables in the population. Hypothesis testing starts with the assumption that the null hypothesis is true in the population, and you use statistical tests to assess whether the null hypothesis can be rejected or not.

Statistical tests determine where your sample data would lie on an expected distribution of sample data if the null hypothesis were true. These tests give two main outputs:

- A test statistic tells you how much your data differs from the null hypothesis of the test.
- A p value tells you the likelihood of obtaining your results if the null hypothesis is actually true in the population.

Statistical tests come in three main varieties:

- Comparison tests assess group differences in outcomes.
- Regression tests assess cause-and-effect relationships between variables.
- Correlation tests assess relationships between variables without assuming causation.

Step 5: Interpret your results

The final step of statistical analysis is interpreting your results.

Statistical significance

In hypothesis testing, statistical significance is the main criterion for forming conclusions. You compare your p value to a set significance level (usually 0.05) to decide whether your results are statistically significant or non-significant.

Statistically significant results are considered unlikely to have arisen solely due to chance. There is only a very low chance of such a result occurring if the null hypothesis is true in the population.

Lesson 04 - Python Environment Setup and Essentials

Anaconda

Python Basics

- Variable naming conventions
- Data Types and Type Casting

```
int(), str(), float()
```

- Scope of variables
- Arithmetic and Assignment Operators
- Comparison, Logical and Bitwise Operators
- String Formatting

```
first_name='Rose'
last_name='Mary'
print('Hello! I am {} {}'.format(first_name,last_name))
print('Hello! I am {0} {1}'.format(first_name,last_name))
print('Hello! I am {0} {1}. {1} is my surname'.format(first_name,last_name))
# Hello! I am Rose Mary. Mary is my surname
```

```
f_name='Karan'
age=40
print(f'My name is {f_name}.I am {age} years old')
```

- String Functions

capitalize()	isnumeric()	replace("old","new")
isalnum()	len()	strip()
isalpha()	lower()	
isdigit()	upper()	

```
#join() method
s1=['Ram','Shyam','Mohan']
x=" ".join(s1)
print(x)
```

```
#len() method
s='Python Programming Language'
len(s)
```

```
st='Lionel Messi'
st.count('e')
```

```
#find() method
s='Ronaldo'
s.find('u')
```

```
#split() method
s='Data+Science'
s.split('+')
```

```
#strip() method
s='  Tom Cruise '
s.strip()
```

- User Input

```
x=input()
input('What is your name:')
x=int(input('Enter a number:' ))
y=float(input('Enter your marks:' ))
```

Data Types with Python

Number

#assign

```
num1=24
```

#delete reference

```
del num1,num2,num3
```

#control float decimal places

```
num=12.345678
print("the number is %.2f" %(num))
```

#other notable functions

```
import math
num=math.ceil(12.345678)      #smallest integer not less than the given (13)
num=math.floor(12.345678)     #largest integer not greater than the given (12)
num=min(12,24,46,8)           #gets smallest from options(8)
num=max(12,24,46,8)           #gets biggest from options(46)
```

```
import random
r=random.random() * 10
```

```
r=random.randrange(1,100)
```

Lists (mutable / editable)

```
a=['apple','banana','cherry']
a
```

```
a[1]
```

```
List1.append(600)
List1.remove(600)
List1.insert(0,"first")
List1.pop()                    #remove last entry
List1.pop(2)                   #remove entry at position
B=max(List1)
S=min(List1)
```

```
#check for membership
if 3 in [1,2,3]:
    print("yes")
```

Tuples (immutable / uneditable)

```
b=(4,8,10,0,8,6)
b
```

```
b[2]
```

```
b[2]=5
---------------------------------------------------------------------------
TypeError                          Traceback (most recent call last)
<ipython-input-14-9241502132fc> in <module>
----> 1 b[2]=5

TypeError: 'tuple' object does not support item assignment
```

```
B=max(Tup1)
S=min(Tup1)

if 3 in (1,2,3):
    print("yes")
```

Sets (unordered, unchangeable but allows adding items, no duplicates)

```
#create set
s={'apple','banana','cherry'}
```

```
#loop over set
for i in s:
   print(i)
```

#trying to change a set value results in error

```
s[0]='pineapple'
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-6-a2d825ae5cb2> in <module>
----> 1 s[0]='pineapple'

TypeError: 'set' object does not support item assignment
```

```
#add item
s.add('pineapple')
print(s)
```

```
#add multiple items
s.update({'orange','watermelon'})
print(s)
```

```
#remove item
s.discard('mango')
```

```
#create a set and remove last item
a={'apple','banana','cherry','pineapple'}
a.pop()
```

```
#clear the entire set
a.clear()
```

Dictionaries

create dictionary and display items

```
d={"brand":"Ford","model":"Mustang","year":1964}
d
```

```
thisdict=dict(brand="Ford",model="Mustang",year=1964)
thisdict
```

#output value of a given key

```
print(d.get("brand"))
```

```
d['brand']
```

#Assigning new value to given key

```
d["brand"]="Maruti"
print(d)
```

```
d["color"]="Red"
print(d)
```

copy dictionary to another variable

```
dict1={1:"Ram",2:"Shyam",3:"Lakhan"}
dict2=dict1.copy()
print(dict2)
```

#display all items

```
dict1.items()
```

#display all keys

```
dict1.keys()
```

#remove a key-value pair by supplying a key

```
dict1.pop(1)
print(dict1)
```

#remove first key-value pair

```
dict1.popitem()
dict1
```

#add a new key-value pair

```
dict1.update({4:"Sita"})
dict1
```

#clearing all key-value pairs

```
dict1={1:"Ram",2:"Shyam",3:"Lakhan"}
print(dict1)

dict1.clear()
print(dict1)
```

#displaying values only

```
dict1={1:"Ram",2:"Shyam",3:"Lakhan"}
dict1.values()
```

Basic Operators and Functions

Python comparison operators

Equal	==
Not equal	!=
Greater than	>
Less than	<
Greater than or equal	>=
Less than or equal	<=

Condition statement

```
Num1 = 10
Num2 = 20
Total = Num1 + Num2
```

```
if Total<50:
	print("less than 50")
elif Total>50:
	print("greater than 50")
else:
	print("equals 50")
```

```
#test for string membership
Title="python pentesting"
if "test" in Title:
	print("member")
else:
	print("not a member")
```

Loops

```
counter = 1
while counter<=10:
	print(counter)
	counter+=1
```

```
#sample1: year loop
#sample2: computation loop
#sample3: ip loop
	toc = 0
	foc = 0

	while toc<=254:

		while foc<=254:
			print("160.12.%d.%d") %(toc,foc)
			foc+=1
		foc=0		#reset foc to 0
		toc+=1

	print('done')
```

Break

```
counter = 1
while counter<=10:
	print(counter)
	counter+=1
	if counter==5:
		break;
```

Exception handling

```
try:
	#risky code here
except:
	sys.exit("your custom error message here")
```

Functions

#function without parameters

```
def line2():
        print("line2")

print("line3")
line2()          #call a function
```

#function with parameter and a return value

```
def addme(x,y):
        z=x+y
        return z

print(addme(10,20))
```

#call another script to use its functions

```
#[script1.py]
def addme(x,y):
        z=x+y
        return z

#[script2.py]
import script1
from script1 import *
print(addme(10,40))
```

Regular Expressions

Metacharacters for regular expressions

#import the regular expression package

```
import re
```

square brackets [] used to find a match in the string

```
s="ac"
re.search('[abc]',s)
```

period (.) matches any single character except newline \n

```
s="ac"
re.search('.',s)
```

caret ^ used to check if string starts with a specific character

```
s='mumbai'
x = re.search('^m',s)
print(x)
```

dollar ($) used to check if string ends with a specific character

```
a = re.search('a$','Makati')
print(a)
b = re.search('a$','Manila')
print(b)
```

star or asterisk (*) used to check string for matches zero or more occurrences to left of it

```
a = re.search('ma*','man')
print(a)
b = re.search('ma*','woman')
print(b)
```

plus (+) one or more occurrence to

```
a = re.search('ma+','man')
print(a)
b = re.search('ma+','woman')
print(b)
```

check for exact number of occurrence

```
a=re.search('a{2}', 'abc')
print(a)
b = re.search('a{2}', 'aab')
print(b)
```

vertical bar or pipe (|) matches string using an “or” expression

```
a=re.search('a|b', 'mango')
print(a)
b = re.search('a|b', 'cherry')
print(b)
```

parenthesis () matches the string inside the parenthesis

```
a=re.search('(a)m', 'amm')
print(a)
b = re.search('(a)m', 'abm')
print(b)
```

backslash used to escape special characters including regular expression special characters

```
a=re.search('\$a', '$abc')
print(a)
```

Built-in Functions for Regular Expressions

#import regular expression package

```
import re
```

#returns a list of all occurrence

```
st = "Philippines is my country"
x=re.findall('pp',st)
print(x)
[output] ['nd']
```

#search for the first occurrence

```
st = "Philippines is my country"
x=re.search('Manila',st)
print(x)
[output] None
```

split string using space (\s)

```
s="I love watching movies"
x = re.split('\s',s)
print(x)
[output] ['I', 'love', 'watching', 'movies']
```

substitute character with another

```
s="I love watching movies"
x = re.sub('\s','2',s)
print(x)
[output] I2love2watching2movies
```

Sets for regular exoression

#import regular expression package

```
import re
```

#find occurrence of a, r and n in the string

```
s="I love watching movies"
x = re.findall('[arn]',s)
print(x)
[output] ['a', 'n']
```

#find occurrence of letters a to n in the string

```
s="I love watching movies"
x = re.findall('[a-n]',s)
print(x)
['I', 'e', 'a', 'c', 'h', 'i', 'n', 'g', 'm', 'i', 'e']
```

#find occurrence of letters except a, r and n in the string

```
s="I love watching movies"
x = re.findall('[^arn]',s)
print(x)
['I', ' ', 'l', 'o', 'v', 'e', ' ', 'w', 't', 'c', 'h', 'i', 'g', ' ', 'm', 'o', 'v', 'i', 'e', 's']
```

#find occurrence of number 0 to 9

```
s="I love watching movies after 8pm"
x = re.findall('[0-9]',s)
print(x)
['8']
```

#find occurrence of lower and uppercase alphabet letters

```
s="I love watching movies"
x = re.findall('[a-zA-Z]',s)
print(x)
```

#find occurrence of a special character (ex “plus +” symbol)

```
s="I love watching movies"
x = re.findall('[+]',s)
print(x)
```

Lesson 05 - Mathematical Computing with Python (NumPy)

Introduction to Numpy

NumPy aims to provide an array object up to 50x faster than traditional python lists. It provides a lot of supporting functions working with NumPy arrays easier.

Import the package

```
import numpy as np
```

creating a 1D Array

```
ar=np.array([1,4,6,3,7])
ar
[output] array([1, 4, 6, 3, 7])

ar1=np.array((2,3,4,5))
ar1
[output] array([2, 3, 4, 5])
```

create a 2-D Array

```
arr=np.array([[1,2,4], [5,6,8]])
arr
[output] array([[1, 2, 4], [5, 6, 8]])
```

create a 3-D Array

```
arr=np.array([[[1,2,3], [5,6,7]], [[1,2,3], [5,6,7]]])
arr
[output] array([[[1, 2, 3],
                 [5, 6, 7]],
                 [[1, 2, 3],
                 [5, 6, 7]]])
```

#accessing elements

```
ar=np.array([1,4,6,3,7])
ar[2]
[output] 6

arr=np.array([[1,2,4], [5,6,8]])
arr[0,1]
[output] 2
```

Use negative indexing to access an array from the end.

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('Last element from 2nd dim: ', arr[1, -1])
```

Mathematical Functions of NumPy

We can perform

- ✓ Trigonometric Functions
- ✓ Logarithmic Functions
- ✓ Inverse Functions
- ✓ Exponential Functions
- ✓ Statistical Functions

Unary operations

get cosine values

```
arr=np.array([1,3,5,2,6])
np.cos(arr)
[output] array([ 0.54030231, -0.9899925 ,  0.28366219, -0.41614684,  0.96017029])
```

get logarithmic values

```
np.log(arr)
[output] array([0.        , 1.09861229, 1.60943791, 0.69314718, 1.79175947])
```

get the mean, median and standard deviations

```
np.mean(arr)
[output] 3.4
```

```
np.median(arr)
[output] 3.0
```

```
np.std(arr)
1.8547236990991407
```

return exponential values in form of an array

```
np.exp(arr)
array([  2.71828183,  20.08553692, 148.4131591 ,   7.3890561 ,      403.42879349])
```

Binary Operations

create sample arrays

```
arr1=np.array([2,4,6,8,2])
arr2=np.array([3,6,1,5,2])
```

#addition

```
arr1+arr2
[output] array([ 5, 10,  7, 13,  4])
```

#subtraction

```
arr1-arr2
[output] array([ 5, 10,  7, 13,  4])
```

#multiplication

```
arr1*arr2
[output] array([ 6, 24,  6, 40,  4])
```

More NumPy Built-In Functions

import package

```
import numpy as np
```

create a 1D array that are all 1s

```
arr=np.ones(2)
arr
[output] array([1., 1.])
```

create 2D array with all values 1s

```
np.ones((2,2))
[output] array([[1., 1.],   [1., 1.]])
```

create a 1D array that are all 0s

```
np.zeros(2)
{output} array([0., 0.])
```

create 2D array with all values 0s

```
np.zeros((2,2))
[output] array([[0., 0.],     [0., 0.]])
```

returns an integer that tells us how many dimensions the array have.

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

An array can have any number of dimensions.
When the array is created, you can define the number of dimensions by using the ndmin argument.

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

Slicing Arrays

- ✓ Slicing in python means taking elements from one given index to another given index.
- ✓ We pass slice instead of index like this: [start:end].
- ✓ We can also define the step, like this: [start:end:step].
- ✓ If we don't pass start its considered 0
- ✓ If we don't pass end its considered length of array in that dimension
- ✓ If we don't pass step its considered 1

Slice elements from index 1 to index 5 from the following array:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
```

Slice elements from index 4 to the end of the array:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[4:])
```

Slice elements from the beginning to index 4 (not included):

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[:4])
```

Use the minus operator to refer to an index from the end:
Slice from the index 3 from the end to index 1 from the end:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[-3:-1])
```

Use the step value to determine the step of the slicing:
Return every other element from index 1 to index 5:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5:2])
```

Return every other element from the entire array:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[::2])
```

Slicing 2-D Arrays
Note: Remember that second element has index 1.
From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[1, 1:4])
```

NumPy Array Copy vs View

The main difference between a copy and a view of an array is that the copy is a new array, and the view is just a view of the original array.

The copy owns the data and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.

The view does not own the data and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

Make a copy, change the original array, and display both arrays:
The copy SHOULD NOT be affected by the changes made to the original array.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42
print(arr)
print(x)
```

Make a view, change the original array, and display both arrays:
The view SHOULD be affected by the changes made to the original array.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42
print(arr)
print(x)
```

Make a view, change the view, and display both arrays:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
x[0] = 31
print(arr)
print(x)
```

Print the value of the base attribute to check if an array owns it's data or not:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
y = arr.view()
print(x.base)
print(y.base)
```

Shape of an Array

The shape of an array is the number of elements in each dimension.

NumPy arrays have an attribute called shape
Shape returns a tuple with each index having the number of corresponding elements.

Print the shape of a 2-D array:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)

# The example above returns (2, 4), which means that the array has 2 dimensions, where the first dimension has 2 elements and the second has 4.
```

Create an array with 5 dimensions using ndmin using a vector with values 1,2,3,4 and verify that last dimension has value 4:

```
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('shape of array :', arr.shape)
```

Reshaping arrays

By reshaping we can add or remove dimensions or change number of elements in each dimension.

```
# Convert the following 1-D array with 12 elements into a 2-D array.
# The outermost dimension will have 4 arrays, each with 3 elements:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print(newarr)
```

```
# Convert the following 1-D array with 12 elements into a 3-D array.
# The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(2, 3, 2)
print(newarr)
```

Iterating

```
# Iterate on the elements of the following 1-D array:
import numpy as np
arr = np.array([1, 2, 3])
for x in arr:
  print(x)
```

```
# Iterate on the elements of the following 2-D array:
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
  print(x)
```

```
# Iterate on each scalar element of the 2-D array:
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
  for y in x:
    print(y)
```

```
# Iterate on the elements of the following 3-D array:
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
  print(x)
```

Iterate down to the scalars:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
  for y in x:
    for z in y:
      print(z)
```

Iterating Arrays Using nditer()

In basic for loops, iterating through each scalar of an array we need to use n for loops which can be difficult to write for arrays with very high dimensionality.

Iterate through the following 3-D array:

```
import numpy as np
arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
for x in np.nditer(arr):
  print(x)
```

Enumerate on following 1D arrays elements:

```
import numpy as np
arr = np.array([1, 2, 3])
for idx, x in np.ndenumerate(arr):
  print(idx, x)
```

Enumerate on following 2D array's elements:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for idx, x in np.ndenumerate(arr):
  print(idx, x)
```

Join

Join two arrays

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))
print(arr)
```

Join two 2-D arrays along rows (axis=1):

```
import numpy as np
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])
arr = np.concatenate((arr1, arr2), axis=1)
print(arr)
```

We can concatenate two 1-D arrays along the second axis
this would result in putting them one over the other, ie. stacking.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.stack((arr1, arr2), axis=1)
print(arr)
```

NumPy provides a helper function: hstack() to stack along rows.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.hstack((arr1, arr2))
print(arr)
```

NumPy provides a helper function: vstack() to stack along columns.

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.vstack((arr1, arr2))
print(arr)
```

Splitting

Split the array in 3 parts:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr)
```

Access the splitted arrays:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr[0])
print(newarr[1])
print(newarr[2])
```

Split the 2-D array into three 2-D arrays.

```
import numpy as np
arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
newarr = np.array_split(arr, 3)
print(newarr)
```

Searching

Find the indexes where the value is 4:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x)
```

Find the indexes where the values are even:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
x = np.where(arr%2 == 0)
print(x)
```

Sorting

Sort the array:

```
import numpy as np
arr = np.array([3, 2, 0, 1])
print(np.sort(arr))
```

Sort the array alphabetically:

```
import numpy as np
arr = np.array(['banana', 'cherry', 'apple'])
print(np.sort(arr))
```

Sort a 2-D array:

```
import numpy as np
arr = np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(arr))
```

Data Distribution

Data Distribution is a list of all possible values, and how often each value occurs. The random module offer methods that returns randomly generated data distributions.

- We can generate random numbers based on defined probabilities using the choice() method of the random module.
- The choice() method allows us to specify the probability for each value.
- he probability is set by a number between 0 and 1, where 0 means that the value will never occur and 1 means that the value will always occur.

#Generate a 1-D array containing 100 values, where each value has to be 3, 5, 7 or 9.
The probability for the value to be 3 is set to be 0.1
The probability for the value to be 5 is set to be 0.3
The probability for the value to be 7 is set to be 0.6
The probability for the value to be 9 is set to be 0

```
from numpy import random
x = random.choice([3, 5, 7, 9], p=[0.1, 0.3, 0.6, 0.0], size=(100))
print(x)
```

- The sum of all probability numbers should be 1.
- Even if you run the example above 100 times, the value 9 will never occur.
- You can return arrays of any shape and size by specifying the shape in the size parameter.

Same example as above, but return a 2-D array with 3 rows, each containing 5 values.

```
from numpy import random
x = random.choice([3, 5, 7, 9], p=[0.1, 0.3, 0.6, 0.0], size=(3, 5))
print(x)
```

Lesson 07 - Data Manipulation with Pandas

Introduction to Pandas

Pandas is a high-level data manipulation tool used in machine learning and data science.

To install pandas (comes installed with Jupyter)

```
\> pip install pandas
```

To use, import in your script

```
import pandas as pd
```

Reading Datasets

By calling read_csv(), read_excel() and other datasets, you create a DataFrame, which is the main data structure used in pandas.

Reading CSV files

```
data=pd.read_csv('Iris.csv')
data.head()
```

Reading Excel File

```
data1=pd.read_excel('example_excel.xls')
data1.head()
```

Plotting Datasets using Pandas

import package

```
import pandas as pd
```

read and test-display some data

```
data=pd.read_csv('Iris.csv')
data.head()
```

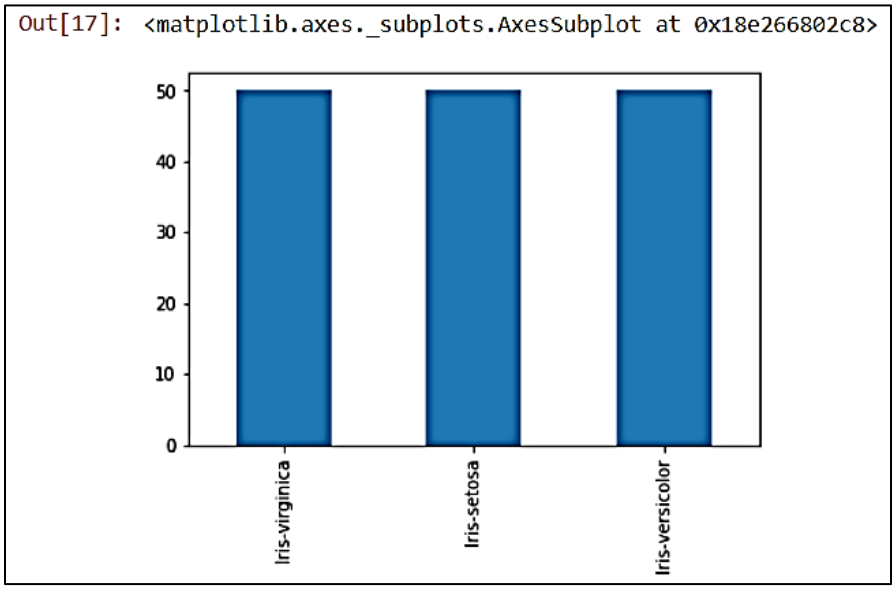
Note:

- ✓ The value_counts() function is used to return the value for different classes present in the column of a dataset.
- ✓ Plot a Pie Chart when we have 4-5 distinct values in a column, a Bar Chart when we have more than 5 distinct values and a line chart if we have many distinct values

create a bar plot

```
data['Species'].value_counts().plot(kind='bar')
```

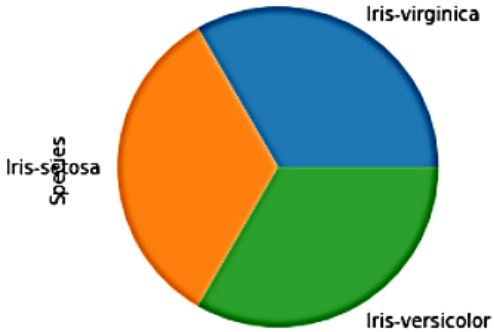
```
Out[17]: <matplotlib.axes._subplots.AxesSubplot at 0x18e266802c8>
```



create a pie chart

```
data['Species'].value_counts().plot(kind='pie')
```

```
Out[18]: <matplotlib.axes._subplots.AxesSubplot at 0x18e266ed408>
```



create a line chart

```
data['SepalLengthCm'].value_counts().plot(kind='line')
```

Understanding DataFrame

Activity:

First, download the data by passing the download URL to pandas.read_csv():

```
import pandas as pd
download_url = (
"https://raw.githubusercontent.com/fivethirtyeight/"
"data/master/college-majors/recent-grads.csv"
)
```

```
df = pd.read_csv(download_url)
```

```
type(df)
Out[4]: pandas.core.frame.DataFrame
```

Now that you have a DataFrame, you can take a look at the data.
First, you should configure the display.max.columns option to make sure pandas doesn't hide any columns.
Then you can view the first few rows of data with .head():

```
pd.set_option("display.max.columns", None)
df.head()
```

Your dataset contains some columns related to the earnings of graduates in each major:

- "Median" is the median earnings of full-time, year-round workers.
- "P25th" is the 25th percentile of earnings.
- "P75th" is the 75th percentile of earnings.
- "Rank" is the major's rank by median earnings.

Now you're ready to make your first plot with .plot():

```
df.plot(x="Rank", y=["P25th", "Median", "P75th"])
```

note: .plot() returns a line graph containing data from every row in the DataFrame.
The x-axis values represent the rank of each institution
The "P25th", "Median", and "P75th" values are plotted on the y-axis.

Looking at the plot, you can make the following observations:

- ✓ The median income decreases as rank decreases. This is expected because the rank is determined by the median income.
- ✓ Some majors have large gaps between the 25th and 75th percentiles. People with these degrees may earn significantly less or significantly more than the median income.
- ✓ Other majors have very small gaps between the 25th and 75th percentiles. People with these degrees earn salaries very close to the median income.

Plot Types

.plot() has several optional parameters. Most notably, the kind parameter accepts eleven different string values and determines which kind of plot you'll create:

- ✓ "area" is for area plots.
- ✓ "bar" is for vertical bar charts.
- ✓ "barh" is for horizontal bar charts.
- ✓ "box" is for box plots.
- ✓ "hexbin" is for hexbin plots.
- ✓ "hist" is for histograms.
- ✓ "kde" is for kernel density estimate charts.
- ✓ "density" is an alias for "kde".
- ✓ "line" is for line graphs.
- ✓ "pie" is for pie charts.
- ✓ "scatter" is for scatter plots.

The default value is "line". Line graphs, like the one you created above, provide a good overview of your data. You can use them to detect general trends. They rarely provide sophisticated insight, but they can give you clues as to where to zoom in.

If you don't provide a parameter to .plot(), then it creates a line plot with the index on the x-axis and all the numeric columns on the y-axis. While this is a useful default for datasets with only a few columns, for the college majors dataset and its several numeric columns, it looks like quite a mess.

Distributions and Histograms

DataFrame is not the only class in pandas with a .plot() method. As so often happens in pandas, the Series object provides similar functionality.

You can get each column of a DataFrame as a Series object. Here's an example using the "Median" column of the DataFrame you created from the college major data:

```
median_column = df["Median"]
```

```
type(median_column)
Out[13]: pandas.core.series.Series
```

```
median_column.plot(kind="hist")
```

The histogram shows the data grouped into ten bins ranging from $20,000 to $120,000, and each bin has a width of $10,000. The histogram has a different shape than the normal distribution, which has a symmetric bell shape with a peak in the middle. The histogram of the median data, however, peaks on the left below $40,000. The tail stretches far to the right and suggests that there are indeed fields whose majors can expect significantly higher earnings.

Sorting for plotting

Contrary to the first overview, you only want to compare a few data points, but you want to see more details about them. For this, a bar plot is an excellent tool.

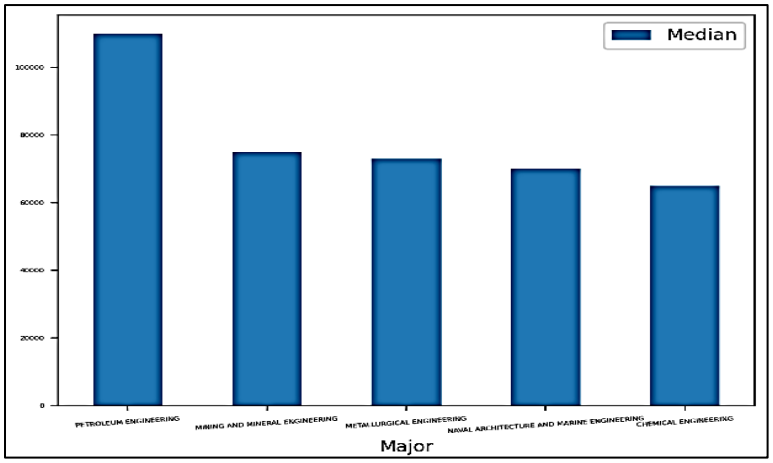
select the five majors with the highest median earnings.

```
top_5 = df.sort_values(by="Median", ascending=False).head()
```

create a bar plot that shows only the majors with these top five median salaries:

Notice that you use the rot and fontsize parameters to rotate and size the labels of the x-axis so that they're visible.

```
top_5.plot(x="Major", y="Median", kind="bar", rot=5, fontsize=4)
```



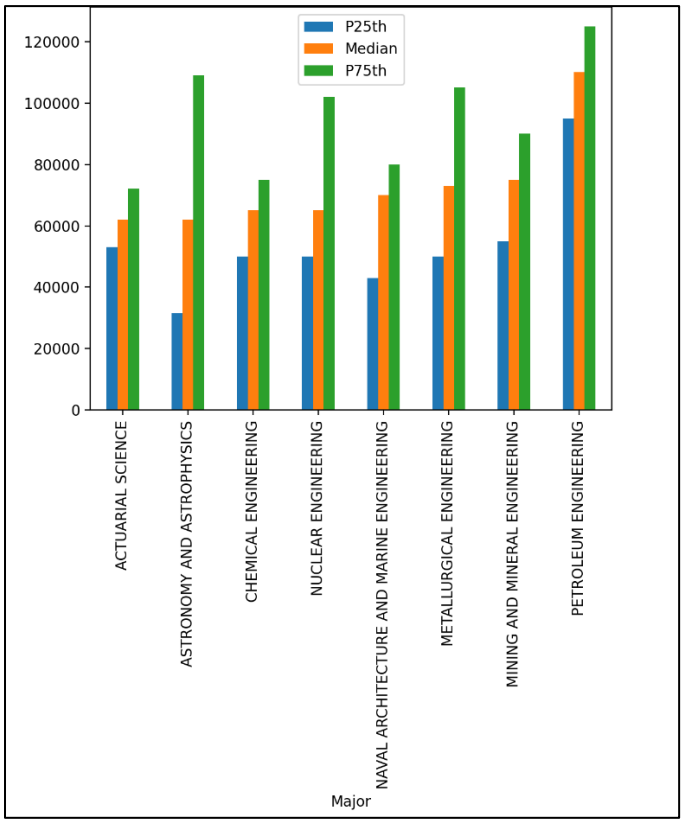
This plot shows that the median salary of petroleum engineering majors is more than $20,000 higher than the rest. The earnings for the second- through fourth-place majors are relatively close to one another.

If you have a data point with a much higher or lower value than the rest, then you'll probably want to investigate a bit further. For example, you can look at the columns that contain related data.

Let's investigate all majors whose median salary is above $60,000. First, you need to filter these majors with the mask df[df["Median"] > 60000]. Then you can create another bar plot showing all three earnings columns:

```
top_medians = df[df["Median"] > 60000].sort_values("Median")
```

```
top_medians.plot(x="Major", y=["P25th", "Median", "P75th"], kind="bar")
```

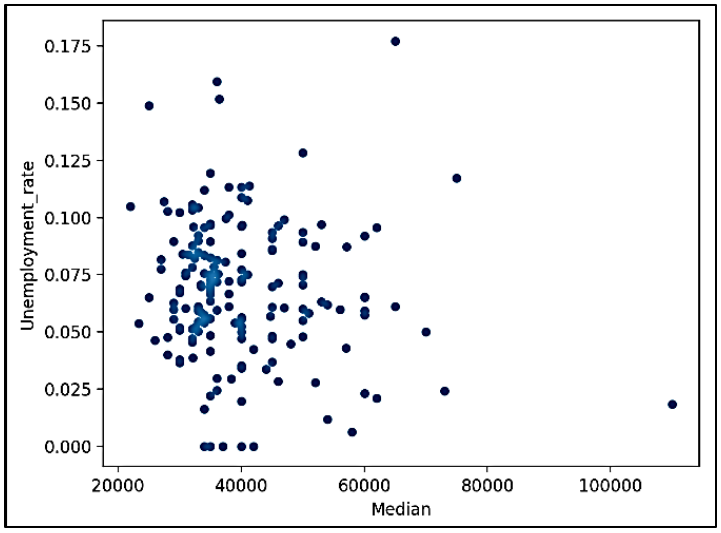


The 25th and 75th percentile confirm what you've seen above: petroleum engineering majors were by far the best paid recent graduates.

Check for Correlation

Often you want to see whether two columns of a dataset are connected. If you pick a major with higher median earnings, do you also have a lower chance of unemployment? As a first step, create a scatter plot with those two columns:

```
df.plot(x="Median", y="Unemployment_rate", kind="scatter")
```



A quick glance at this figure shows that there's no significant correlation between the earnings and unemployment rate.

While a scatter plot is an excellent tool for getting a first impression about possible correlation, it certainly isn't definitive proof of a connection. For an overview of the correlations between different columns, you can use .corr(). If you suspect a correlation between two values, then you have several tools at your disposal to verify your hunch and measure how strong the correlation is.

Keep in mind, though, that even if a correlation exists between two values, it still doesn't mean that a change in one would result in a change in the other. In other words, correlation does not imply causation.

Analyze Categorical Data

Grouping

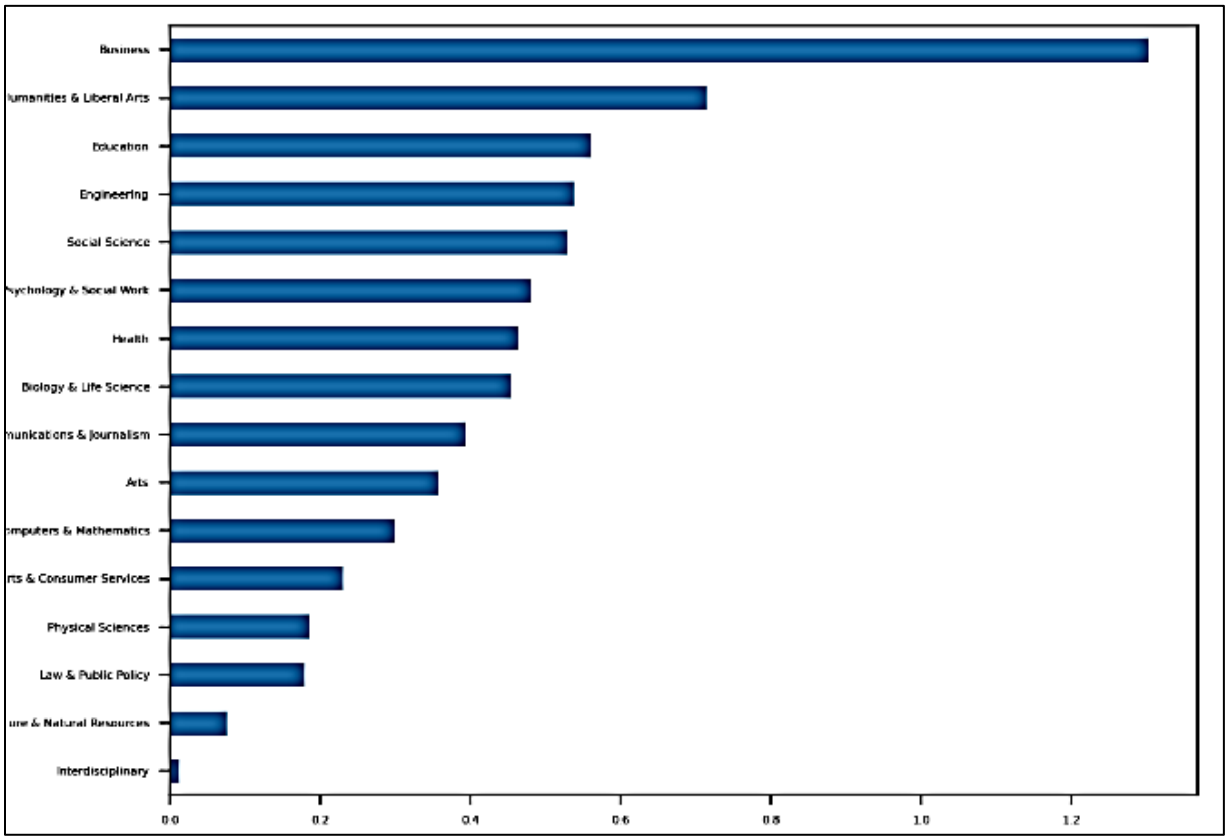
A basic usage of categories is grouping and aggregation. You can use .groupby() to determine how popular each of the categories in the college major dataset are:

```
In [20]: cat_totals = df.groupby("Major_category")["Total"].sum().sort_values()

In [21]: cat_totals
Out[21]:
Major_category
Interdisciplinary                        12296.0
Agriculture & Natural Resources          75620.0
Law & Public Policy                     179107.0
Physical Sciences                       185479.0
Industrial Arts & Consumer Services     229792.0
Computers & Mathematics                 299008.0
Arts                                    357130.0
Communications & Journalism             392601.0
Biology & Life Science                  453862.0
Health                                  463230.0
Psychology & Social Work                481007.0
Social Science                          529966.0
Engineering                             537583.0
Education                               559129.0
Humanities & Liberal Arts               713468.0
Business                               1302376.0
Name: Total, dtype: float64
```

With .groupby(), you create a DataFrameGroupBy object. With .sum(), you create a Series.

horizontal bar plot showing all the category totals in cat_totals:

```
cat_totals.plot(kind="barh", fontsize=4)
```



Determining Ratios

Vertical and horizontal bar charts are often a good choice if you want to see the difference between your categories. If you're interested in ratios, then pie plots are an excellent tool. However, since cat_totals contains a few smaller categories, creating a pie plot with cat_totals.plot(kind="pie") will produce several tiny slices with overlapping labels .

To address this problem, you can lump the smaller categories into a single group. Merge all categories with a total under 100,000 into a category called "Other", then create a pie plot:

```
small_cat_totals = cat_totals[cat_totals < 100_000]
```

```
big_cat_totals = cat_totals[cat_totals > 100_000]
```

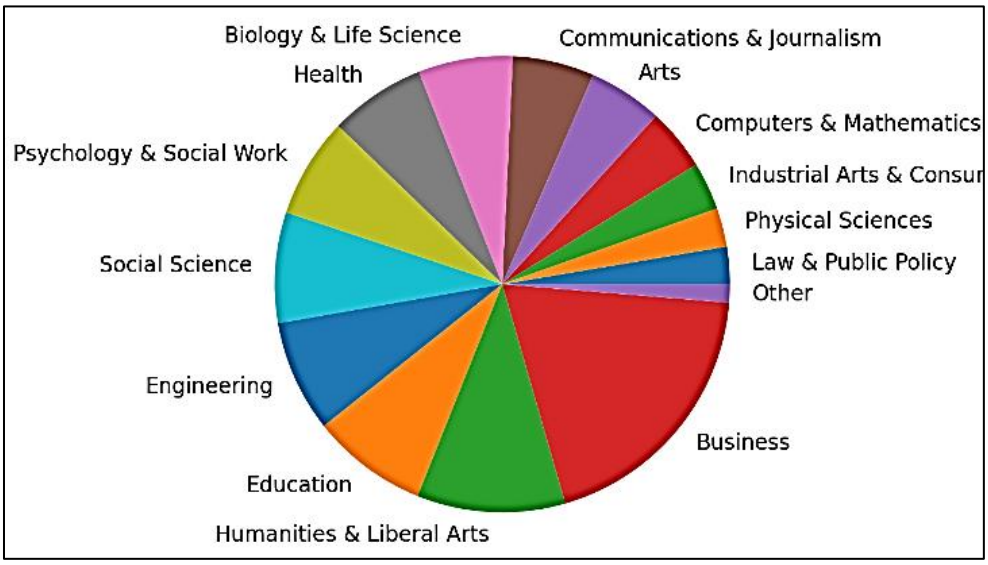
Adding a new item "Other" with the sum of the small categories

```
small_sums = pd.Series([small_cat_totals.sum()], index=["Other"])
```

```
big_cat_totals = big_cat_totals.append(small_sums)
```

```
big_cat_totals.plot(kind="pie", label="")
```

Notice that you include the argument label="". By default, pandas adds a label with the column name. That often makes sense, but in this case it would only add noise.

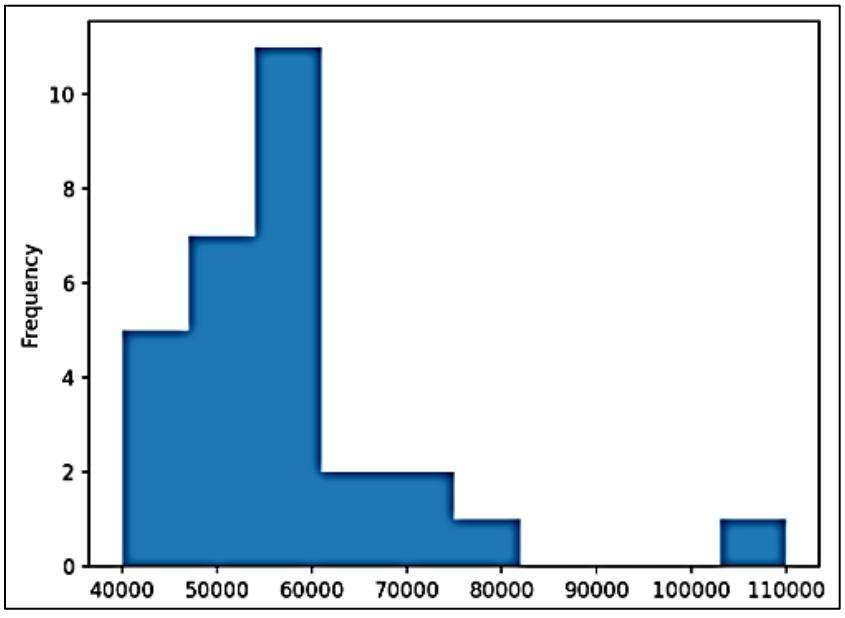


Zooming in on Categories

Sometimes you also want to verify whether a certain categorization makes sense. Are the members of a category more similar to one other than they are to the rest of the dataset? Again, a distribution is a good tool to get a first overview. Generally, we expect the distribution of a category to be similar to the normal distribution but have a smaller range.

Create a histogram plot showing the distribution of the median earnings for the engineering majors:

```
df[df["Major_category"] == "Engineering"]["Median"].plot(kind="hist")
```



You'll get a histogram that you can compare to the histogram of all majors from the beginning

The range of the major median earnings is somewhat smaller, starting at $40,000. The distribution is closer to normal, although its peak is still on the left. So, even if you've decided to pick a major in the engineering category, it would be wise to dive deeper and analyze your options more thoroughly.

View and Select Data Demo

Indexing

importing the necessary libraries

```
import pandas as pd
import numpy as np
```

creating a time series using the date_range method from pandas

```
time_series = pd.date_range('1/1/2020', periods=20)

df = pd.DataFrame(np.random.randn(20, 5),          # 20 refers to the number of rows
                                                   # 5 refers to the number of columns
        index=time_series,
        columns=['Column 1', 'Column 2', 'Column 3', 'Column 4', 'Column 5'])
```

lets check the dataframe that we have just created

```
df
```

	Column 1	Column 2	Column 3	Column 4	Column 5
2020-01-01	-0.506672	-0.223504	1.374367	-0.673617	-1.594992
2020-01-02	-0.288263	-0.308458	0.118085	0.552600	0.057227
2020-01-03	0.208609	0.340054	1.385632	-1.391342	1.447169
2020-01-04	0.755634	-0.584098	1.462399	0.169597	0.646324
2020-01-05	-2.071256	-0.095079	0.967895	-0.553546	-1.772507
2020-01-06	-0.553698	1.113536	-0.208415	-0.971505	0.448935
2020-01-07	2.179848	1.115860	-0.742047	0.039219	1.084622
2020-01-08	1.773190	-0.763939	0.731432	0.760961	-1.826588
2020-01-09	-1.410691	-0.435762	-0.312730	-0.420567	1.313196
2020-01-10	-2.198756	-0.140123	-0.278568	1.416964	-2.059844
2020-01-11	-0.367993	1.378564	-1.204453	-0.473113	-0.794758
2020-01-12	1.143618	0.460143	0.367623	0.319697	0.575761
2020-01-13	0.316275	0.946194	1.169399	-0.609133	0.136591
2020-01-14	-1.233144	0.239735	-1.435708	0.684846	1.075830
2020-01-15	1.369500	1.410637	0.092005	-0.044152	-0.193836
2020-01-16	-0.822112	-0.209220	-1.553531	1.470538	-0.962117
2020-01-17	-0.101630	-0.394767	0.098769	-1.416084	1.107761
2020-01-18	-1.573445	1.087323	-1.315079	0.293028	-0.680757
2020-01-19	-0.658735	0.197481	-0.258119	-1.348261	-0.546297
2020-01-20	0.400129	0.635901	-0.002345	-1.364261	-0.305020

```
df['Column 1']
```

```
2020-01-01   -0.506672
2020-01-02   -0.288263
2020-01-03    0.208609
2020-01-04    0.755634
2020-01-05   -2.071256
2020-01-06   -0.553698
2020-01-07    2.179848
2020-01-08    1.773190
2020-01-09   -1.410691
```

```
df[['Column 1', 'Column 2', 'Column 3']]
```

	Column 1	Column 2	Column 3
2020-01-01	-0.506672	-0.223504	1.374367
2020-01-02	-0.288263	-0.308458	0.118085
2020-01-03	0.208609	0.340054	1.385632
2020-01-04	0.755634	-0.584098	1.462399
2020-01-05	-2.071256	-0.095079	0.967895
2020-01-06	-0.553698	1.113536	-0.208415
2020-01-07	2.179848	1.115860	-0.742047
2020-01-08	1.773190	-0.763939	0.731432
2020-01-09	-1.410691	-0.435762	-0.312730
2020-01-10	-2.198756	-0.140123	-0.278568
2020-01-11	-0.367993	1.378564	-1.204453
2020-01-12	1.143618	0.460143	0.367623

timeseries specific indexing

```
col1 = df['Column 1']
col1[time_series[3]]
[output] 0.755634442895147
```

Slicing

*access sequences

iloc attribute access a group of rows and columns by index location
print first 5 rows of column 1 and column 2

```
df.iloc[:5, 0:2]
```

	Column 1	Column 2
2020-01-01	-0.506672	-0.223504
2020-01-02	-0.288263	-0.308458
2020-01-03	0.208609	0.340054
2020-01-04	0.755634	-0.584098
2020-01-05	-2.071256	-0.095079

Striding

Print all columns, with a 3 step stride by adding additional colon and specifying the step.

```
df[::3]
```

	Column 1	Column 2	Column 3	Column 4	Column 5
2020-01-01	-0.506672	-0.223504	1.374367	-0.673617	-1.594992
2020-01-04	0.755634	-0.584098	1.462399	0.169597	0.646324
2020-01-07	2.179848	1.115860	-0.742047	0.039219	1.084622
2020-01-10	-2.198756	-0.140123	-0.278568	1.416964	-2.059844
2020-01-13	0.316275	0.946194	1.169399	-0.609133	0.136591
2020-01-16	-0.822112	-0.209220	-1.553531	1.470538	-0.962117
2020-01-19	-0.658735	0.197481	-0.258119	-1.348261	-0.546297

Filtering

```
df[(df['Column 3'] < 0)]
```

	Column 1	Column 2	Column 3	Column 4	Column 5
2020-01-06	-0.553698	1.113536	-0.208415	-0.971505	0.448935
2020-01-07	2.179848	1.115860	-0.742047	0.039219	1.084622
2020-01-09	-1.410691	-0.435762	-0.312730	-0.420567	1.313196
2020-01-10	-2.198756	-0.140123	-0.278568	1.416964	-2.059844
2020-01-11	-0.367993	1.378564	-1.204453	-0.473113	-0.794758
2020-01-14	-1.233144	0.239735	-1.435708	0.684846	1.075830
2020-01-16	-0.822112	-0.209220	-1.553531	1.470538	-0.962117
2020-01-18	-1.573445	1.087323	-1.315079	0.293028	-0.680757
2020-01-19	-0.658735	0.197481	-0.258119	-1.348261	-0.546297
2020-01-20	0.400129	0.635901	-0.002345	-1.364261	-0.305020

```
df[(df['Column 1'] < 0) & (df['Column 2'] > 0)][['Column 4', 'Column 5']]
```

	Column 4	Column 5
2020-01-06	-0.971505	0.448935
2020-01-11	-0.473113	-0.794758
2020-01-14	0.684846	1.075830
2020-01-18	0.293028	-0.680757
2020-01-19	-1.348261	-0.546297

Missing Values

Notes:

- A lot of time are spent in Data Cleaning which includes tasks like cleaning missing values
- Missing values occur when there is no data or value stored for the variable in an observation
- This is quite common and can have significant effect to conclusion
- Examples of when there might be missing values:
    - Incomplete survey forms fields skipped by participants
    - Accidental or data entry errors
    - Pending information
    - Deleted outdated data

Delete missing values?

Notes:

- Sometimes deleting records with missing values is the way to go.
- But we may be left with small dataset and a small dataset implies an inaccurate machine learning model.
- We can delete the columns or rows where the percentage of missing values is very high
    - If the column has around 30-40% (or more than that) then we can delete such columns
    - If the dataset has more than 50 columns, we can delete rows if the row has more than 10 values missing, hence 20% or more missing values from rows we can delete rows.
- These are just recommendations and can vary from scenarios
- Check before deleting the columns:
    - If a column has high correlation with the target column
    - If the removal of a row or column can lead to information loss

Filling up missing values

- Business logic
  Ex: missing n-star ratings for products
  -here we do not input 0, will lead to very low rating
  -we can use negative numbers like -1 to -999
- Statistical
  Ex. Mean / Median / Mode

Activity: Interpolate Missing Values in Pandas

Suppose we have the following pandas DataFrame that shows the total sales made by a store during 15 consecutive days:

```
import pandas as pd
import numpy as np
```

#create DataFrame

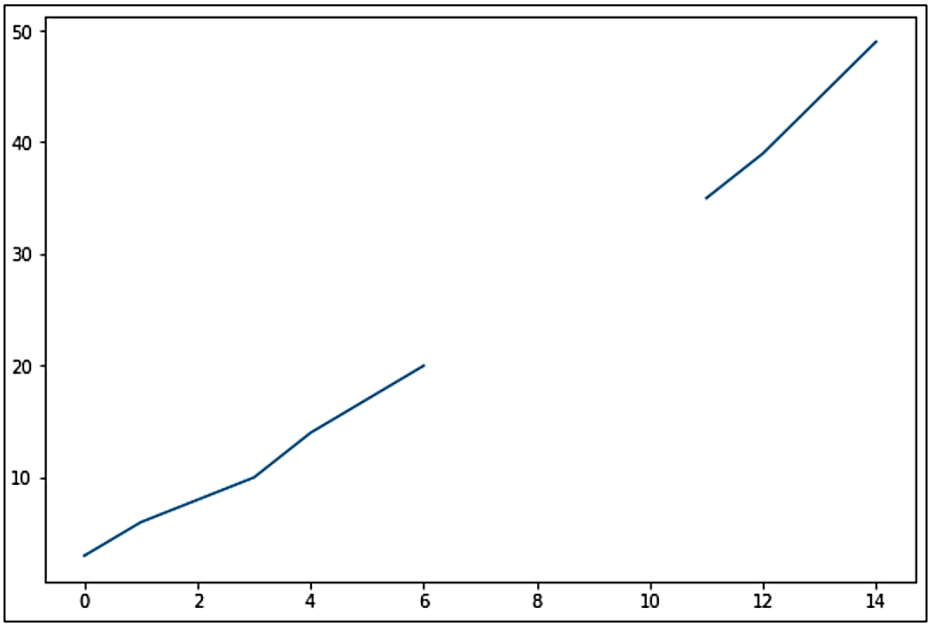
```
df = pd.DataFrame({'day': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
                   'sales': [3, 6, 8, 10, 14, 17, 20, np.nan, np.nan, np.nan, np.nan, 35, 39, 44, 49]})
```

Notice that we're missing sales numbers for four days in the data frame.

#create line chart to visualize sales

```
df['sales'].plot()
```

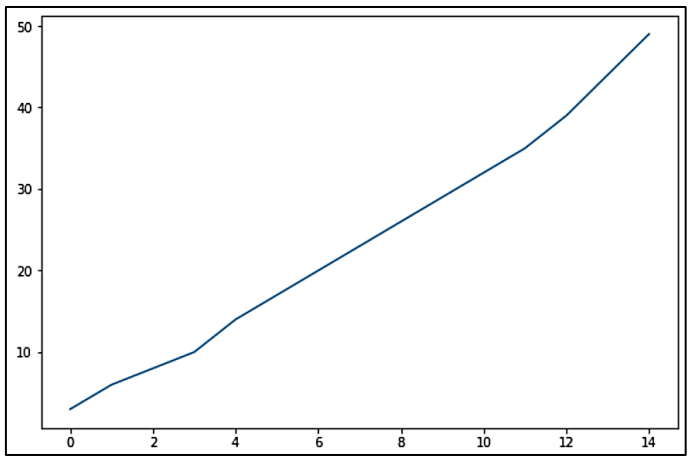
	day	sales
0	1	3.0
1	2	6.0
2	3	8.0
3	4	10.0
4	5	14.0
5	6	17.0
6	7	20.0
7	8	NaN
8	9	NaN
9	10	NaN
10	11	NaN
11	12	35.0
12	13	39.0
13	14	44.0
14	15	49.0



#interpolate missing values in 'sales' column

```
df['sales'] = df['sales'].interpolate()
df
```

	day	sales
0	1	3.0
1	2	6.0
2	3	8.0
3	4	10.0
4	5	14.0
5	6	17.0
6	7	20.0
7	8	23.0
8	9	26.0
9	10	29.0
10	11	32.0
11	12	35.0
12	13	39.0
13	14	44.0
14	15	49.0



Activity: Imputing Data with Pandas

Wine Magazine Dataset: https://www.kaggle.com/datasets/christopheiv/winemagdata130k

import libraries and read dataset

```
import pandas as pd
df = pd.read_csv("winemag-data-130k-v2.csv")
```

display some data, observe columns and missing values

```
df.head()
```

Display missing value information like the number of non-null values in each column:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129971 entries, 0 to 129970
Data columns (total 14 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   Unnamed: 0             129971 non-null  int64
 1   country                129908 non-null  object
 2   description            129971 non-null  object
 3   designation            92506 non-null   object
 4   points                 129971 non-null  int64
 5   price                  120975 non-null  float64
 6   province               129908 non-null  object
 7   region_1               108724 non-null  object
 8   region_2               50511 non-null   object
 9   taster_name            103727 non-null  object
 10  taster_twitter_handle  98758 non-null   object
 11  title                  129971 non-null  object
 12  variety                129970 non-null  object
 13  winery                 129971 non-null  object
dtypes: float64(1), int64(2), object(11)
memory usage: 13.9+ MB
```

calculate the number of missing values in each column:

```
df.isnull().sum()
```

```
Unnamed: 0                  0
country                    63
description                 0
designation             37465
points                      0
price                    8996
province                   63
region_1                21247
region_2                79460
taster_name             26244
taster_twitter_handle   31213
title                       0
variety                     1
winery                      0
dtype: int64
```

The 'price' column contains 8996 missing values.
We can replace these missing values using the '.fillna()' method.
For example, let's fill in the missing values with the mean price:

```
df['price'].fillna(df['price'].mean(), inplace = True)
df.isnull().sum()
```

```
Unnamed: 0                  0
country                    63
description                 0
designation             37465
points                      0
price                       0
province                   63
region_1                21247
region_2                79460
taster_name             26244
taster_twitter_handle   31213
title                       0
variety                     1
winery                      0
dtype: int64
```

We see that the 'price' column no longer has missing values.

Now, suppose we wanted to make a more accurate imputation.
A good guess would be to replace missing values in the price column with the mean prices within the countries the missing values belong.

For example, if we consider missing wine prices for Italian wine, we can replace these missing values with the mean price of Italian wine.

To proceed, let's look at the distribution in 'country' values. We can use the 'Counter' method from the collections module to do so:

```
from collections import Counter
print(Counter(df['country']))
```

```
Counter({'US': 54504, 'France': 22093, 'Italy': 19540, 'Spain': 6645, 'Portugal': 5691, 'Chile': 4472, 'Argentina': 3800, 'Aust
ria': 3345, 'Australia': 2329, 'Germany': 2165, 'New Zealand': 1419, 'South Africa': 1401, 'Israel': 505, 'Greece': 466, 'Canad
a': 257, 'Hungary': 146, 'Bulgaria': 141, 'Romania': 120, 'Uruguay': 109, 'Turkey': 90, 'Slovenia': 87, 'Georgia': 86, 'Englan
d': 74, 'Croatia': 73, 'Mexico': 70, nan: 63, 'Moldova': 59, 'Brazil': 52, 'Lebanon': 35, 'Morocco': 28, 'Peru': 16, 'Ukraine':
14, 'Czech Republic': 12, 'Serbia': 12, 'Macedonia': 12, 'Cyprus': 11, 'India': 9, 'Switzerland': 7, 'Luxembourg': 6, 'Armeni
a': 2, 'Bosnia and Herzegovina': 2, 'Slovakia': 1, 'China': 1, 'Egypt': 1})
```

Let's take a look at wine made in the 'US'. We can define a data frame containing only 'US' wines:

```
df_US = df[df['country']=='US']
```

Now, let's print the number of missing values:

```
df_US.isnull().sum()
```

```
Unnamed: 0                 0
country                    0
description                0
designation            17596
points                     0
price                    239
province                   0
region_1                 278
region_2                3993
taster_name            16774
taster_twitter_handle  19763
title                      0
variety                    0
winery                     0
dtype: int64
```

We see that there are 239 missing 'price' values in the 'US' wines data. To fill in the missing values with the mean corresponding to the prices in the US we do the following:

```
df_US['price'].fillna(df_US['price'].mean(), inplace = True)
```

Now suppose we wanted to do this for the missing price values in each country. First, let's impute missing country values:

```
df['country'].fillna(df['country'].mode()[0], inplace = True)
```

Next, within a for-loop we can define country-specific data frames:

```
for i in list(set(df['country'])):
   df_country = df[df['country']== country]
```

Next, we can fill the missing values in these country-specific data frames with their respective mean prices:

```
for i in list(set(df['country'])):
   df_country = df[df['country']== country]
   df_country['price'].fillna(df_country['price'].mean(),inplace = True)
```

We then append the result to a list we'll call "frames"

```
frames = []
for i in list(set(df['country'])):
   df_country = df[df['country']== country]
   df_country['price'].fillna(df_country['price'].mean(),inplace = True)
   frames.append(df_country)
```

Finally, we concatenate the resulting list of data frames:

```
frames = []
for i in list(set(df['country'])):
   df_country = df[df['country']== i]
   df_country['price'].fillna(df_country['price'].mean(),inplace = True)
   frames.append(df_country)
   final_df = pd.concat(frames)
```

Now, if we print the number of missing price values before imputation we get:

```
print(df.isnull().sum())
```

```
country                   63
description                0
designation            37465
points                     0
price                   8996
province                  63
region_1               21247
region_2               79460
taster_name            26244
taster_twitter_handle  31213
title                      0
variety                    1
winery                     0
```

And after imputation:

```
print(final_df.isnull().sum())
```

```
country                   0
description               0
designation           37454
points                    0
price                     1
province                  0
region_1              21184
region_2              79397
taster_name           26244
taster_twitter_handle 31213
title                     0
variety                   1
winery                    0
```

We see all but one of the missing values have been imputed. This corresponds to wines in Egypt which has no price data.

We can fix this by checking the length of the data frame within the for loop and only imputing with the country-specific mean if the length is greater than one. If the length is equal to 1 we impute with the mean across all countries:

```
frames = []
for i in list(set(df['country'])):
   df_country = df[df['country']== i]
   if len(df_country) > 1:
      df_country['price'].fillna(df_country['price'].mean(),inplace = True)
   else:
```

```
    df_country['price'].fillna(df['price'].mean(),inplace = True)
  frames.append(df_country)
  final_df = pd.concat(frames)
```

Printing the result we see that all of the values have been imputed for the 'price' column:

```
final_df.isnull().sum()
```

We can define a function that generalizes this logic. Our function will take variables corresponding to a numerical column and categorical column:

```
def impute_numerical(categorical_column, numerical_column):
  frames = []
  for i in list(set(df[categorical_column])):
    df_category = df[df[categorical_column]== i]
    if len(df_category) > 1:
      df_category[numerical_column].fillna(df_category[numerical_column].mean(),inplace = True)
    else:
      df_category[numerical_column].fillna(df[numerical_column].mean(),inplace = True)
    frames.append(df_category)
    final_df = pd.concat(frames)
  return final_df
```

We perform imputation using our function by executing the following:

```
impute_price  = impute_numerical('country', 'price')
impute_price.isnull().sum()
```

Let's also verify that the shapes of the original and imputed data frames match

```
print("Original Shape: ", df.shape)
print("Imputed Shape: ", impute_price.shape)
```

```
Original Shape:  (129971, 13)
Imputed Shape:  (129971, 13)
```

Similarly, we can define a function that imputes categorical values. This function will take two variables corresponding columns with categorical values.

```
def impute_categorical(categorical_column1, categorical_column2):
  cat_frames = []
  for i in list(set(df[categorical_column1])):
    df_category = df[df[categorical_column1]== i]
    if len(df_category) > 1:
      df_category[categorical_column2].fillna(df_category[categorical_column2].mode()[0],inplace = True)
    else:
      df_category[categorical_column2].fillna(df[categorical_column2].mode()[0],inplace = True)
    cat_frames.append(df_category)
    cat_df = pd.concat(cat_frames)
  return cat_df
```

We can impute missing ‘taster_name’ values with the mode in each respective country:

```
impute_taster = impute_categorical('country', 'taster_name')
impute_taster.isnull().sum()
```

We see that the ‘taster_name’ column now has zero missing values. Again, let’s verify that the shape matches with the original data frame:

```
print("Original Shape: ", df.shape)
print("Imputed Shape: ", impute_taster.shape)
```

File Read and Write Support

Using Pandas to Write and Read CSV Files

prepare dataset

```
data = {
   'CHN': {'COUNTRY': 'China', 'POP': 1_398.72, 'AREA': 9_596.96,
        'GDP': 12_234.78, 'CONT': 'Asia'},
   'IND': {'COUNTRY': 'India', 'POP': 1_351.16, 'AREA': 3_287.26,
        'GDP': 2_575.67, 'CONT': 'Asia', 'IND_DAY': '1947-08-15'},
   'USA': {'COUNTRY': 'US', 'POP': 329.74, 'AREA': 9_833.52,
        'GDP': 19_485.39, 'CONT': 'N.America',
        'IND_DAY': '1776-07-04'},
   'IDN': {'COUNTRY': 'Indonesia', 'POP': 268.07, 'AREA': 1_910.93,
        'GDP': 1_015.54, 'CONT': 'Asia', 'IND_DAY': '1945-08-17'},
   'BRA': {'COUNTRY': 'Brazil', 'POP': 210.32, 'AREA': 8_515.77,
        'GDP': 2_055.51, 'CONT': 'S.America', 'IND_DAY': '1822-09-07'},
   'PAK': {'COUNTRY': 'Pakistan', 'POP': 205.71, 'AREA': 881.91,
        'GDP': 302.14, 'CONT': 'Asia', 'IND_DAY': '1947-08-14'},
   'NGA': {'COUNTRY': 'Nigeria', 'POP': 200.96, 'AREA': 923.77,
        'GDP': 375.77, 'CONT': 'Africa', 'IND_DAY': '1960-10-01'},
   'BGD': {'COUNTRY': 'Bangladesh', 'POP': 167.09, 'AREA': 147.57,
        'GDP': 245.63, 'CONT': 'Asia', 'IND_DAY': '1971-03-26'},
   'RUS': {'COUNTRY': 'Russia', 'POP': 146.79, 'AREA': 17_098.25,
        'GDP': 1_530.75, 'IND_DAY': '1992-06-12'},
   'MEX': {'COUNTRY': 'Mexico', 'POP': 126.58, 'AREA': 1_964.38,
        'GDP': 1_158.23, 'CONT': 'N.America', 'IND_DAY': '1810-09-16'},
   'JPN': {'COUNTRY': 'Japan', 'POP': 126.22, 'AREA': 377.97,
        'GDP': 4_872.42, 'CONT': 'Asia'},
   'DEU': {'COUNTRY': 'Germany', 'POP': 83.02, 'AREA': 357.11,
        'GDP': 3_693.20, 'CONT': 'Europe'},
   'FRA': {'COUNTRY': 'France', 'POP': 67.02, 'AREA': 640.68,
        'GDP': 2_582.49, 'CONT': 'Europe', 'IND_DAY': '1789-07-14'},
   'GBR': {'COUNTRY': 'UK', 'POP': 66.44, 'AREA': 242.50,
        'GDP': 2_631.23, 'CONT': 'Europe'},
   'ITA': {'COUNTRY': 'Italy', 'POP': 60.36, 'AREA': 301.34,
        'GDP': 1_943.84, 'CONT': 'Europe'},
   'ARG': {'COUNTRY': 'Argentina', 'POP': 44.94, 'AREA': 2_780.40,
        'GDP': 637.49, 'CONT': 'S.America', 'IND_DAY': '1816-07-09'},
   'DZA': {'COUNTRY': 'Algeria', 'POP': 43.38, 'AREA': 2_381.74,
        'GDP': 167.56, 'CONT': 'Africa', 'IND_DAY': '1962-07-05'},
```

```
  'CAN': {'COUNTRY': 'Canada', 'POP': 37.59, 'AREA': 9_984.67,
       'GDP': 1_647.12, 'CONT': 'N.America', 'IND_DAY': '1867-07-01'},
  'AUS': {'COUNTRY': 'Australia', 'POP': 25.47, 'AREA': 7_692.02,
       'GDP': 1_408.68, 'CONT': 'Oceania'},
  'KAZ': {'COUNTRY': 'Kazakhstan', 'POP': 18.53, 'AREA': 2_724.90,
       'GDP': 159.41, 'CONT': 'Asia', 'IND_DAY': '1991-12-16'}
}

columns = ('COUNTRY', 'POP', 'AREA', 'GDP', 'CONT', 'IND_DAY')
```

```
# import library
import pandas as pd

# data is organized in such a way that the country codes correspond to columns.
# You can reverse the rows and columns of a DataFrame with the property .T:
df = pd.DataFrame(data=data).T
df

# You can save your Pandas DataFrame as a CSV file with .to_csv():
df.to_csv('data.csv')

# Read a CSV File
df = pd.read_csv('data.csv', index_col=0)
df
```

Using Pandas to Write and Read Excel Files

```
# install the following Python packages first:
    ✓ xlwt to write to .xls files
    ✓ openpyxl or XlsxWriter to write to .xlsx files
    ✓ xlrd to read Excel files

\> pip install xlwt openpyxl xlsxwriter xlrd

# Write an Excel File
df.to_excel('data.xlsx')

# Read an Excel File
df = pd.read_excel('data.xlsx', index_col=0)

# You can also use read_excel() with OpenDocument spreadsheets, or .ods files.
```

Pandas Sql Operation

we will use the titanic.csv dataset

```
import Pandas as pd
path = 'https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/'
data = pd.read_csv(path + "titanic.csv")
data.head()
```

Select operation is used to fetch a required piece of information from the given data.
Using the SQL:

```
SELECT Survived, Pclass, Name
FROM data
LIMIT 5;
```

Talking about the Pandas library, we can perform the selection of variables in the following way:

```
data[['Survived', 'Pclass', 'Name']].head()
```

Where is a conditional operation we mostly use to find data values from the data that follows some condition.
Using the SQL we can find data points where the sex variable has the value male in the following way:

```
SELECT *
FROM data
WHERE Sex = 'male'
LIMIT 5
```

#Using Pandas, we can perform the same in the following way:

```
data[data['Sex'] == 'male'].head()
```

OR and **AND** operation. These are also conditional operations that merge two conditions into one.
Using the SQL language, we can find values where variable sex is male and the age of the person is more than five in the following way:

```
SELECT *
FROM data
WHERE Sex = 'Male' AND Age > 5.00;
```

We can perform the same operations using Pandas in the following way:

```
data[(data['Sex'] == 'male') & (data['Age'] > 5.00)]
```

Group by. We perform the group-by operation to make groups of the data values using some categories. With this data, we can make groups of males and females. In SQL we can do this in the following way:

```
SELECT Sex, count(*)
FROM data
GROUP BY Sex;
```

Note: This query will give us the number of records for every Sex.

The same procedure can be performed using Pandas in the following way:

```
data.groupby('Sex').size()
```

Join

Join operations are the most used operations using SQL because it mainly helps in making new data using two or more data using one variable. We join data in different the following manners:

Inner join

This join provides common values from the variable on which we decided to join, using the SQL we can perform this operation in the following manner:

```
SELECT *
FROM df1
INNER JOIN df2
  ON df1.key = df2.key;
```

Where we have two data frames(df1 and df2) and one common variable(key). To perform this operation we are required to have two or more datasets. We can make a data frame in Pandas using the following way:

```
import numpy as np
df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'],
                    'value': np.random.randn(4)})


df2 = pd.DataFrame({'key': ['B', 'D', 'D', 'E'],
                    'value': np.random.randn(4)})
```

Now we can inner join datasets in the following four ways as given below.

```
pd.merge(df1, df2, on='key')
```

Left outer join

This operation helps us join the datasets using a clause. With the help of this, we can preserve the unmatched rows of the left data and join them with a NULL row in the shape of the right table. In SQL we can perform this operation in the following way:

```
SELECT *
FROM df1
LEFT OUTER JOIN df2
  ON df1.key = df2.key;
```

the same operation can be performed using pandas in the following way:

```
pd.merge(df1, df2, on='key', how='left')
```

Right outer join

This operation helps us in joining data using a clause using which we preserve rows from the right data and join them with a null in the shape of the first (left) table. Using the SQL we can perform this operation in the following way:

```
SELECT *
FROM df1
RIGHT OUTER JOIN df2
  ON df1.key = df2.key;
```

The same operation Using the Pandas can be performed in the following way

```
pd.merge(df1, df2, on='key', how='right')
```

Full join

This operation preserves all the rows of every data while joining them. This operation can be performed using the SQL in the following way:

```
SELECT *
FROM df1
FULL OUTER JOIN df2
  ON df1.key = df2.key;
```

#This same operation can be performed using Pandas in the following way

```
pd.merge(df1, df2, on='key', how='outer')
```

Lesson 06 - Scientific computing with Python (Scipy)

Introduction to SciPy

The SciPy library of Python is built to work with NumPy arrays and provides many user-friendly and efficient numerical practices such as routines for numerical integration and optimization. Together, they run on all popular operating systems, are quick to install and are free of charge. NumPy and SciPy are easy to use, but powerful enough to depend on by some of the world's leading scientists and engineers.

```
https://scipy.org/
```

Scipy will often be using python anonymous functions

Anonymous functions

- Anonymous Functions are functions without a name
- Anonymous Functions accept multiple inputs and return single output
- The contain only one single executable statement

There are many anonymous functions in python. Some of the most popular ones are – lambda(), map(), zip()

Lambda Functions

- Lambda function is a small anonymous function.
- It can take any number of arguments but can have only one expression
- The syntax is:

```
Lambda arguments: expression
```

Example:

We want to create a function to compute for the square root of a number;

```
x= lambda a: a*a
print(x(5))
```

we want to create a function that adds to numbers

```
y= lambda a,b: a+b
y(10,30)
```

Filter Function

- The filter function filters the given items in an iteration with the help of a function that helps check each element to be true or not.
- Syntax of the filter function
  filter(function, iterable)

Example:

```
score =[76,23,54,12,65,43]
```

```
def score_check(s):
    if s>50:
        return True
    else:
        return False
```

```
students_passed=filter(score_check, score)
for i in students_passed:
    print(i)
```

Map Function

- The map functions executes a specified function for each item in an iterable. The item is sent to the function as an argument.
- The syntax is:

```
map(function,iterable)
```

Example:

```
total_score=[365,450,290,398,270,430]
def percent(li):
    ans=(li*100)/500
    return ans
```

```
mapped_list=map(percent,total_score)
print(list(mapped_list))
```

Zip Function

- The zip function returns a zip object, which is an iterator of tuples or list, where the item in each passed iterator is paired together, and second item in each passed iterator are paired together

Example:

```
student_names = ['Rohan', 'Amita', 'Shushant', 'Pawan', 'Kiran']
student_marks = [67,34,78,85,65]
zipped_list = zip(student_names, student_marks)
zipped_list = list(zipped_list)
zipped_list
```

[('Rohan', 67), ('Amita', 34), ('Shushant', 78), ('Pawan', 85), ('Kiran', 65)]

```
student_names, student_marks = zip( *zipped_list)
print('names = ',list(student_names))
print('marks = ',list(student_marks))
```

```
names =  ['Rohan', 'Amita', 'Shushant', 'Pawan', 'Kiran']
marks =  [67, 34, 78, 85, 65]
```

SciPy sub package

SciPy is organized into sub-packages covering different scientific computing domains. These are summarized in the following table −

scipy.constants	Physical and mathematical constants
scipy.fftpack	Fourier transform
scipy.integrate	Integration routines
scipy.interpolate	Interpolation
scipy.io	Data input and output
scipy.linalg	Linear algebra routines
scipy.optimize	Optimization
scipy.signal	Signal processing
scipy.sparse	Sparse matrices
scipy.spatial	Spatial data structures and algorithms
scipy.special	Any special mathematical functions
scipy.stats	Statistics

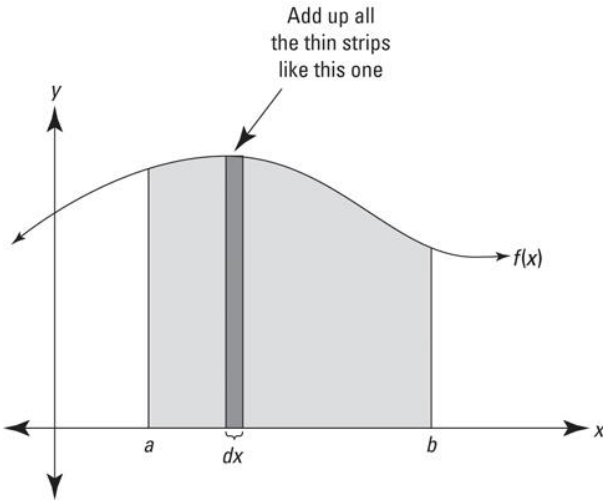
SciPy Sub packages Integrations

The scipy.integrate sub-package provides several integration techniques including an ordinary differential equation integrator.

Numerical Integration is the approximate computation of an integral using numerical techniques. Methods for Integrating function given function object:

quad	– General Purpose Integration
dblquad	– General Purpose Double Integration
nquad	– General Purpose n- fold Integration
fixed_quad	– Gaussian quadrature, order n
quadrature	– Gaussian quadrature to tolerance
romberg	– Romberg integration
trapz	– Trapezoidal rule
cumtrapz	– Trapezoidal rule to cumulatively compute integral
simps	– Simpson’s rule
romb	– Romberg integration
polyint	– Analytical polynomial integration (NumPy)

The most fundamental meaning of integration is to add up. And when you depict integration on a graph, you can see the adding up process as a summing up of thin rectangular strips of area to arrive at the total area under that curve, as shown in this figure.



You can calculate the shaded area in the above figure by using this integral:

$$\int_a^b f(x)\,dx$$

(Note that everything here involves definite integration as opposed to indefinite integration. Definite integration is where the elongated S integration symbol has limits of integration: the two little constants or numbers at the bottom and the top of the symbol. The elongated S without limits of integration indicates an indefinite integral or antiderivative.)

Look at the thin rectangle in the figure. It has a height of f(x) and a width of dx (a little bit of x), so its area (length times width, of course) is given by $f(x) \cdot dx$. The above integral tells you to add up the areas of all the narrow rectangular strips between a and b under the curve f(x). As the strips get narrower and narrower, you get a better and better estimate of the area. The power of integration lies in the fact that it gives you the exact area by sort of adding up an infinite number of infinitely thin rectangles.

Regardless of what the tiny bits are that you're adding up — they could be little bits of distance or volume or energy (or just area) — you can represent the summation as an adding up of the areas of thin rectangular strips under a curve. If the units on both the x and y axes are units of length, say, feet, then each thin rectangle measures so many feet by so many feet, and its area — length times width — is some number of square feet. In this case, the total area of all the rectangles between a and b gives you an area answer (though not necessarily the actual area under the curve because the scale may be different; for example, the actual shaded area in the above figure is a few square inches, but your answer could be a number of square miles if both axes were marked off in miles). The point is that in this case, you add up the areas of all the rectangles, and you get an area answer. Usually, however, even though you add up the areas of rectangles, your answer will not be an area answer.

Demos

quad :

The function quad is provided to integrate a function of one variable between two points.
The points can be +infinite or – infinite to indicate infinite limits.

Example:

```
from scipy.integrate import quad

def f(x):
  return 3.0*x*x + 1.0

I, err = quad(f, 0, 1)
print(I)
print(err)
```

Output :

```
2.0
2.220446049250313e-14
```

dblquad :

This performs Double Integration with 2 arguments.

Example:

```
from scipy.integrate import dblquad

area = dblquad(lambda x, y: x*y, 0, 0.5,
          lambda x: 0, lambda x: 1-2*x)

print(area)
```

Output :

```
(0.010416666666666668, 4.101620128472366e-16)
```

romberg :

With the help of scipy.integrate.romberg() method, we can get the romberg integration of a callable function from limit a to b

Example:

```
import numpy as np
from scipy import integrate

f = lambda x: 3*(np.pi)*x**3

# using scipy.integrate.romberg()
g = integrate.romberg(f, 1, 2, show = True)

print(g)
```

Output:

```
Romberg integration of <function vectorize1.<locals>.vfunc at 0x0000003C1E212790> from [1, 2]
Steps  StepSize   Results
   1  1.000000 42.411501
   2  0.500000 37.110063 35.342917
   4  0.250000 35.784704 35.342917 35.342917

The final result is 35.34291735288517 after 5 function evaluations.
35.34291735288517
```

trapz :

numpy.trapz() function integrate along the given axis using the composite trapezoidal rule.

Example:

```
import numpy as np

b = [2, 4]
a = [6, 8]

f = np.trapz(b, a)

print(f)
```

Output:

```
6.0
```

SciPy Sub packages Optimization

Sample Constrained Optimization

Demo: Box Volume Optimization

Scenario:
Imagine we have a box, and we want to maximize the volume by changing the Length, Width and Height but we want to make sure that the surface a rea of the box stays less than 10 (SA <=10)

1. Find the equation that describes our box
   V = L x W x H

2. The surface area is calculated as
   SA = (2 x L x W) + (2 x L x H) + (2 x W x H)

Solution:

```
import numpy as np
from scipy.optimize import minimize
```

define formula to calculate volume of box

```
def calcVolume(x):
  length = x[0]
  width = x[1]
  height = x[2]
  volume = length * width * height
  return volume
```

define formula to calculate surface area of box

```
def calcSurface(x):
  length = x[0]
  width = x[1]
  height = x[2]
  surfaceArea = 2*length*width + 2*length*height + 2*height*width
  return surfaceArea
```

define objective function for optimization
this is the function that our optimization will try to minimize
in this case we return negative volume, by minimizing the negative volume, we'll maximize the volume of the box

```
def objective(x)
  return -calcVolume(x)
```

create function that define constraints for optimization

```
def constraints(x):
  return 10 – calcSurface(x)
```

scipy also requires that constraints be loaded into a dictionary
here, we are creating a constraint of type: inequality and using a function called constraints

```
cons = ({'type' : 'ineq' , 'fun' : 'constraints'})
```

Set initial guesses

```
lengthGuess=1
widthGuess=1
heightGuess=1
```

load guesses into a numpy array
x0 will be our initial guess

```
x0 = np.array([lengtthGuess,widthGuess,heightGuess])
```

call scipy minimize to solve our optimization problem
sequential least squares programming to solve general nonlinear optimization problems

```
sol = minimize( objective , x0 , method='SLSQP' , constraints=cons , options={'disp' : True} )
```

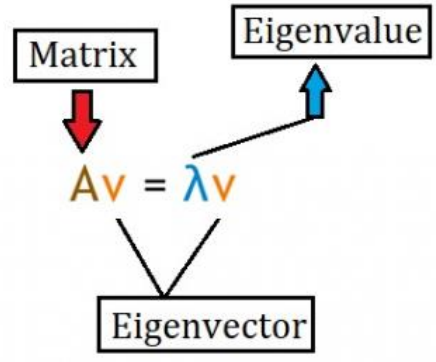
```
# retrieve the optimized box sizing and volume
x0pt = sol.x
volumeOpt = -sol.fun

# calculate surface area with optimized values just to double check
# that we are still within the boundaries of the given surface area
surfaceAreaOpt = calcSurface(xOpt)

# print results
print('length: ' + str(xOpt[0])
print('width: ' + str(xOpt[1])
print('height: ' + str(xOpt[2])
print('volume: ' + str(volumeOpt)
print('surface area: ' + str(surfaceAreaOpt)
```

```
Length: 1.2909944727867015
Width: 1.290994455352175
Height: 1.2909944180130968
Volume: 2.1516574144672 69
Surface Area: 9.999999999713689
```

Demo - Calculate Eigenvalues and Eigenvector

Eigen vectors and Eigen values find their uses in many situations. The word 'Eigen' in German means 'own' or 'typical'. An Eigen vector is also known as a 'characteristic vector'. Suppose we need to perform some transformation on a dataset but the given condition is that the direction of data in the dataset shouldn't change. This is when Eigen vectors and Eigen values can be used.

Given a square matrix (a matrix where the number of rows is equal to the number of columns), an Eigen value and an Eigen vector fulfil the below equation.



Eigen vectors are computed after finding the Eigen values.

Note − Eigen values work well with dimensions 3 or greater as well.

Instead of manually perfroming these mathematical computations, SciPy provides a function in the library called 'eig' which helps compute the Eigenvalue and the Eigenvector.

Syntax of 'eig' function

```
scipy.linalg.eig(matrix)
```

Let us see how the 'eig' function can be used −

Example

```
from scipy import linalg
import numpy as np
my_arr = np.array([[5,7],[11,3]])
eg_val, eg_vect = linalg.eig(my_arr)
print("The Eigenvalues are :")
print(eg_val)
print("The Eigenvectors are :")
print(eg_vect)
```

Output

```
The Eigenvalues are :
[12.83176087+0.j -4.83176087+0.j]
The Eigenvectors are :
[[ 0.66640536 -0.57999285]
[ 0.74558963 0.81462157]]
```

Explanation

- The required libraries are imported.
- A matrix is defined with certain values in it, using the Numpy library.
- The matrix is passed as a parameter to the 'eig' function that computes the eigenvalues and the eigenvectors of the matrix.
- These computed data is stored in two different variables.
- This output is displayed on the console.

Lesson 11 - Web Scraping with BeautifulSoup

There are mainly two ways to extract data from a website:

- Use the API of the website (if it exists). For example, Facebook has the Facebook Graph API which allows retrieval of data posted on Facebook.
- Access the HTML of the webpage and extract useful information/data from it. This technique is called web scraping or web harvesting or web data extraction.

Quick demos:

Searching for an article online

Wikipedia is a great site to gather information about virtually anything, for example, people, places, technology, and what not. If you like to search for something on Wikipedia from your Python script, this recipe is for you.

Download and install Wikipedia's API

```
\>pip install wikipedia
```

Sample code

```
import wikipedia
#print(wikipedia.summary("Cisco"))
#wikipedia.search("Cisco")
print(wikipedia.page("Cisco").content)
```

Reading news feed

If you are developing a social networking website with news and stories, you may be interested to present the news from various world news agencies

Download and import feedparser

```
\>pip install feedparser
```

Sample code

```
import feedparser
import webbrowser

feed = feedparser.parse("https://finance.yahoo.com/rss/")

# feed_title = feed['feed']['title']  # NOT VALID
feed_entries = feed.entries

for entry in feed.entries:

   article_title = entry.title
   article_link = entry.link
   article_published_at = entry.published # Unicode string
   article_published_at_parsed = entry.published_parsed # Time object
   # article_author = entry.author  DOES NOT EXIST
   # content = entry.summary
   # article_tags = entry.tags  DOES NOT EXIST

   print ("{}".format(article_title))
   print ("[{}]".format(article_link))
   print ("Published at {}".format(article_published_at))
   print ()
   # print ("Published by {}".format(article_author))
   # print("Content {}".format(content))
   # print("catagory{}".format(article_tags))
```

Crawling links present in a web page

At times you would like to find a specific keyword present in a web page. In a web browser, you can use the browser's in-page search facility to locate the terms. Some browsers can highlight it. In a complex situation, you would like to dig deep and follow every URL present in a web page and find that specific term. This recipe will automate that task for you.

Sample code #1

```
from bs4 import BeautifulSoup
import requests

url = "https://www.ust.edu.ph/"

page = requests.get(url)
data = page.text
soup = BeautifulSoup(data,"html.parser")

for link in soup.find_all('a'):
   # print(link.get('href'))
   print(link.get('href'),file=open("links.txt","a"))

print("Links saved to file.")
```

Sample code #2

```
import requests
from bs4 import BeautifulSoup

def web(page,WebUrl):
   try:
      if(page>0):
         url = WebUrl
         code = requests.get(url)
         plain = code.text
         s = BeautifulSoup(plain, "html.parser")
         for link in s.findAll('a', {'class':'D_aGc D_ia M_kE'}):
            tet = link.get('title')
            print(tet)
            tet_2 = link.get('href')
            print(url + tet_2)
            print()
   except ValueError as e:
      print(e)

web(1,'https://www.carousell.ph/categories/vehicles-32')
```

Email Web Scraping

Install email-scraper module

```
\> pip install email-scraper
```

```
from email_scraper import scrape_emails
from bs4 import BeautifulSoup
```

```
import requests

url = "https://www.ust.edu.ph/administrative-offices-contact/"

page = requests.get(url)
data = page.text
soup = BeautifulSoup(data,"html.parser")

# check if you got the page code source
# print(soup.decode())

gotemails = scrape_emails(soup.decode())

# check if you got the emails in array
# print(gotemails)

for myemail in gotemails:
   print(myemail)
```

Another demo:

```
# install libraries
pip install requests
pip install html5lib
pip install bs4
```

```
# Accessing the HTML content from webpage
import requests
URL = "https://www.geeksforgeeks.org/data-structures/"
r = requests.get(URL)
print(r.content)
```

```
# Note: Sometimes you may get error “Not accepted” so try adding a browser user agent like below.
# Find your user agent based on device and browser from here https://deviceatlas.com/blog/list-of-user-agent-strings
headers = {'User-Agent': "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/42.0.2311.135 Safari/537.36 Edge/12.246"}
r = requests.get(url=URL, headers=headers)
print(r.content)
```

```
# Parsing the HTML content (not run in online IDE).
# html5lib is the parser being used
import requests
from bs4 import BeautifulSoup

URL = "http://www.values.com/inspirational-quotes"
r = requests.get(URL)

soup = BeautifulSoup(r.content, 'html5lib')        # If this line causes an error, run 'pip install html5lib' or install html5lib
print(soup.prettify())
```

create a program to save those quotes (and all relevant information about them).

```
import requests
from bs4 import BeautifulSoup
import csv

URL = "http://www.values.com/inspirational-quotes"
r = requests.get(URL)

soup = BeautifulSoup(r.content, 'html5lib')
quotes=[] # a list to store quotes
table = soup.find('div', attrs = {'id':'all_quotes'})

for row in table.findAll('div', attrs = {'class':'col-6 col-lg-3 text-center margin-30px-bottom sm-margin-30px-top'}):
        quote = {}
        quote['theme'] = row.h5.text
        quote['url'] = row.a['href']
        quote['img'] = row.img['src']
        quote['lines'] = row.img['alt'].split(" #")[0]
        quote['author'] = row.img['alt'].split(" #")[1]
        quotes.append(quote)

filename = 'inspirational_quotes.csv'
with open(filename, 'w', newline='') as f:
        w = csv.DictWriter(f,['theme','url','img','lines','author'])
        w.writeheader()
        for quote in quotes:
                w.writerow(quote)
```

Lesson 08 - Machine Learning with Scikit–Learn

Scikit-Learn

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python. This library, which is largely written in Python, is built upon NumPy, SciPy and Matplotlib.

Dataset Loading

A collection of data is called dataset. It is having the following two components −

Features − The variables of data are called its features. They are also known as predictors, inputs or attributes.

- Feature matrix − It is the collection of features, in case there are more than one.
- Feature Names − It is the list of all the names of the features.

Response − It is the output variable that basically depends upon the feature variables. They are also known as target, label or output.

- Response Vector − It is used to represent response column. Generally, we have just one response column.
- Target Names − It represent the possible values taken by a response vector.

Scikit-learn have few example datasets like iris and digits for classification and the Boston house prices for regression.

Example

Following is an example to load iris dataset −

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names
print("Feature names:", feature_names)
print("Target names:", target_names)
print("\nFirst 10 rows of X:\n", X[:10])
```

Output

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target names: ['setosa' 'versicolor' 'virginica']
First 10 rows of X:
[
  [5.1 3.5 1.4 0.2]
  [4.9 3. 1.4 0.2]
  [4.7 3.2 1.3 0.2]
  [4.6 3.1 1.5 0.2]
  [5. 3.6 1.4 0.2]
  [5.4 3.9 1.7 0.4]
  [4.6 3.4 1.4 0.3]
  [5. 3.4 1.5 0.2]
  [4.4 2.9 1.4 0.2]
  [4.9 3.1 1.5 0.1]
]
```

Splitting the dataset

To check the accuracy of our model, we can split the dataset into two pieces-a training set and a testing set. Use the training set to train the model and testing set to test the model. After that, we can evaluate how well our model did.

Example

The following example will split the data into 70:30 ratio, i.e. 70% data will be used as training data and 30% will be used as testing data. The dataset is iris dataset as in above example.

```
from sklearn.datasets import load_iris
iris = load_iris()

X = iris.data
y = iris.target

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size = 0.3, random_state = 1
)
```

```
print(X_train.shape)
print(X_test.shape)

print(y_train.shape)
print(y_test.shape)
```

Output

```
(105, 4)
(45, 4)
(105,)
(45,)
```

As seen in the example above, it uses train_test_split() function of scikit-learn to split the dataset. This function has the following arguments −

X, y − Here, X is the feature matrix and y is the response vector, which need to be split.

test_size − This represents the ratio of test data to the total given data. As in the above example, we are setting test_data = 0.3 for 150 rows of X. It will produce test data of 150*0.3 = 45 rows.

random_size − It is used to guarantee that the split will always be the same. This is useful in the situations where you want reproducible results.

Train the Model

Next, we can use our dataset to train some prediction-model. As discussed, scikit-learn has wide range of Machine Learning (ML) algorithms which have a consistent interface for fitting, predicting accuracy, recall etc.

Example

In the example below, we are going to use KNN (K nearest neighbors) classifier. This example is used to make you understand the implementation part only.

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size = 0.4, random_state=1
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
classifier_knn = KNeighborsClassifier(n_neighbors = 3)
classifier_knn.fit(X_train, y_train)
y_pred = classifier_knn.predict(X_test)

# Finding accuracy by comparing actual response values(y_test)with predicted response value(y_pred)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

# Providing sample data and the model will make prediction out of that data
sample = [[5, 5, 3, 2], [2, 4, 3, 5]]
```

```
preds = classifier_knn.predict(sample)
pred_species = [iris.target_names[p] for p in preds] print("Predictions:", pred_species)
```

Output

```
Accuracy: 0.9833333333333333
Predictions: ['versicolor', 'virginica']
```

Model Persistence

Once you train the model, it is desirable that the model should be persist for future use so that we do not need to retrain it again and again. It can be done with the help of dump and load features of joblib package.

Consider the example below in which we will be saving the above trained model (classifier_knn) for future use −

```
from sklearn.externals import joblib
joblib.dump(classifier_knn, 'iris_classifier_knn.joblib')
```

The above code will save the model into file named iris_classifier_knn.joblib. Now, the object can be reloaded from the file with the help of following code −

```
joblib.load('iris_classifier_knn.joblib')
```

Preprocessing the Data

As we are dealing with lots of data and that data is in raw form, before inputting that data to machine learning algorithms, we need to convert it into meaningful data. This process is called preprocessing the data. Scikit-learn has package named preprocessing for this purpose. The preprocessing package has the following techniques −

Binarisation

This preprocessing technique is used when we need to convert our numerical values into Boolean values.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(
  [2.1, -1.9, 5.5],
  [-1.5, 2.4, 3.5],
  [0.5, -7.9, 5.6],
  [5.9, 2.3, -5.8]]
)
data_binarized = preprocessing.Binarizer(threshold=0.5).transform(input_data)
print("\nBinarized data:\n", data_binarized)
```

In the above example, we used threshold value = 0.5 and that is why, all the values above 0.5 would be converted to 1, and all the values below 0.5 would be converted to 0.

Output

```
Binarized data:
[
  [ 1. 0. 1.]
  [ 0. 1. 1.]
  [ 0. 0. 1.]
  [ 1. 1. 0.]
```

```
]
```

Mean Removal

This technique is used to eliminate the mean from feature vector so that every feature centered on zero.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(
  [2.1, -1.9, 5.5],
  [-1.5, 2.4, 3.5],
  [0.5, -7.9, 5.6],
  [5.9, 2.3, -5.8]]
)

#displaying the mean and the standard deviation of the input data
print("Mean =", input_data.mean(axis=0))
print("Stddeviation = ", input_data.std(axis=0))

#Removing the mean and the standard deviation of the input data
data_scaled = preprocessing.scale(input_data)
print("Mean_removed =", data_scaled.mean(axis=0))
print("Stddeviation_removed =", data_scaled.std(axis=0))
```

Output

```
Mean = [ 1.75 -1.275 2.2 ]
Stddeviation = [ 2.71431391 4.20022321 4.69414529]
Mean_removed = [ 1.11022302e-16 0.00000000e+00 0.00000000e+00]
Stddeviation_removed = [ 1. 1. 1.]
```

Scaling

We use this preprocessing technique for scaling the feature vectors. Scaling of feature vectors is important, because the features should not be synthetically large or small.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(
  [
    [2.1, -1.9, 5.5],
    [-1.5, 2.4, 3.5],
    [0.5, -7.9, 5.6],
    [5.9, 2.3, -5.8]
  ]
)
data_scaler_minmax = preprocessing.MinMaxScaler(feature_range=(0,1))
data_scaled_minmax = data_scaler_minmax.fit_transform(input_data)
print ("\nMin max scaled data:\n", data_scaled_minmax)
```

Output

```
Min max scaled data:
[
  [ 0.48648649 0.58252427 0.99122807]
  [ 0. 1. 0.81578947]
  [ 0.27027027 0. 1. ]
  [ 1. 0.99029126 0. ]
]
```

Normalisation

We use this preprocessing technique for modifying the feature vectors. Normalisation of feature vectors is necessary so that the feature vectors can be measured at common scale. There are two types of normalisation as follows −

L1 Normalisation

It is also called Least Absolute Deviations. It modifies the value in such a manner that the sum of the absolute values remains always up to 1 in each row. Following example shows the implementation of L1 normalisation on input data.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(
  [
    [2.1, -1.9, 5.5],
    [-1.5, 2.4, 3.5],
    [0.5, -7.9, 5.6],
    [5.9, 2.3, -5.8]
  ]
)
data_normalized_l1 = preprocessing.normalize(input_data, norm='l1')
print("\nL1 normalized data:\n", data_normalized_l1)
```

Output

```
L1 normalized data:
[
  [ 0.22105263 -0.2 0.57894737]
  [-0.2027027 0.32432432 0.47297297]
  [ 0.03571429 -0.56428571 0.4 ]
  [ 0.42142857 0.16428571 -0.41428571]
]
```

L2 Normalisation

Also called Least Squares. It modifies the value in such a manner that the sum of the squares remains always up to 1 in each row. Following example shows the implementation of L2 normalisation on input data.

Example

```
import numpy as np
from sklearn import preprocessing
Input_data = np.array(
  [
```

```
    [2.1, -1.9, 5.5],
    [-1.5, 2.4, 3.5],
    [0.5, -7.9, 5.6],
    [5.9, 2.3, -5.8]
  ]
)
data_normalized_l2 = preprocessing.normalize(input_data, norm='l2')
print("\nL1 normalized data:\n", data_normalized_l2)
```

Output

```
L2 normalized data:
[
  [ 0.33946114 -0.30713151 0.88906489]
  [-0.33325106 0.53320169 0.7775858 ]
  [ 0.05156558 -0.81473612 0.57753446]
  [ 0.68706914 0.26784051 -0.6754239 ]
]
```

Lesson 09 - Natural Language Processing with Scikit Learn

NLP Overview

Simply and in short, natural language processing (NLP) is about developing applications and services that can understand human languages.

We are talking here about practical examples of natural language processing (NLP) like speech recognition, speech translation, understanding complete sentences, understanding synonyms of matching words, and writing complete grammatically correct sentences and paragraphs.

This is not everything; you can think about the industrial implementations about these ideas and their benefits.

NLP Applications

As all of you know, there are millions of gigabytes every day are generated by blogs, social websites, and web pages.

Many companies are gathering all of these data for understanding users and their passions and give reports to the companies to adjust their plans.

These data could show that the people of Brazil are happy with product A which could be a movie or anything while the people of the US are happy with product B. And this could be instant (real-time result). Like what search engines do, they give the appropriate results to the right people at the right time.

You know what, search engines are not the only implementation of natural language processing (NLP), and there are a lot of awesome implementations out there.

These are some of the successful implementations of Natural Language Processing (NLP):

Search engines like Google, Yahoo, etc. Google search engine understands that you are a tech guy, so it shows you results related to you.

Social websites feeds like Facebook news feed. The news feed algorithm understands your interests using natural language processing and shows you related Ads and posts more likely than other posts.
Speech engines like Apple Siri.
Spam filters like Google spam filters. It's not just about the usual spam filtering, now spam filters understand what's inside the email content and see if it's a spam or not.

NLP Libraries-Scikit

There are many open-source Natural Language Processing (NLP) libraries, and these are some of them:

- Natural language toolkit (NLTK).
- Apache OpenNLP.
- Stanford NLP suite.
- Gate NLP library.

Natural language toolkit (NLTK) is the most popular library for natural language processing (NLP) which is written in Python and has a big community behind it.

Some Demo

install nltk

```
\> pip install nltk
```

Once you've installed NLTK, you should install all the NLTK packages by running the following code:

```
import nltk
nltk.download()
```

Tokenize text using pure Python

First, we will grab a web page content then we will analyze the text to see what the page is about.
We will use the urllib module to crawl the web page:

```
import urllib.request
response = urllib.request.urlopen('http://php.net/')
html = response.read()
print (html)
```

As you can see from the printed output, the result contains a lot of HTML tags that need to be cleaned.
We can use BeautifulSoup to clean the grabbed text like this:

```
from bs4 import BeautifulSoup
import urllib.request
response = urllib.request.urlopen('http://php.net/')
html = response.read()
soup = BeautifulSoup(html,"html5lib")
text = soup.get_text(strip=True)
print (text)
```

Now we have a clean text from the crawled web page.

Finally, let's convert that text into tokens by splitting the text like this:

```
from bs4 import BeautifulSoup
import urllib.request
response = urllib.request.urlopen('http://php.net/')
html = response.read()
soup = BeautifulSoup(html,"html5lib")
text = soup.get_text(strip=True)
tokens = [t for t in text.split()]
print (tokens)
```

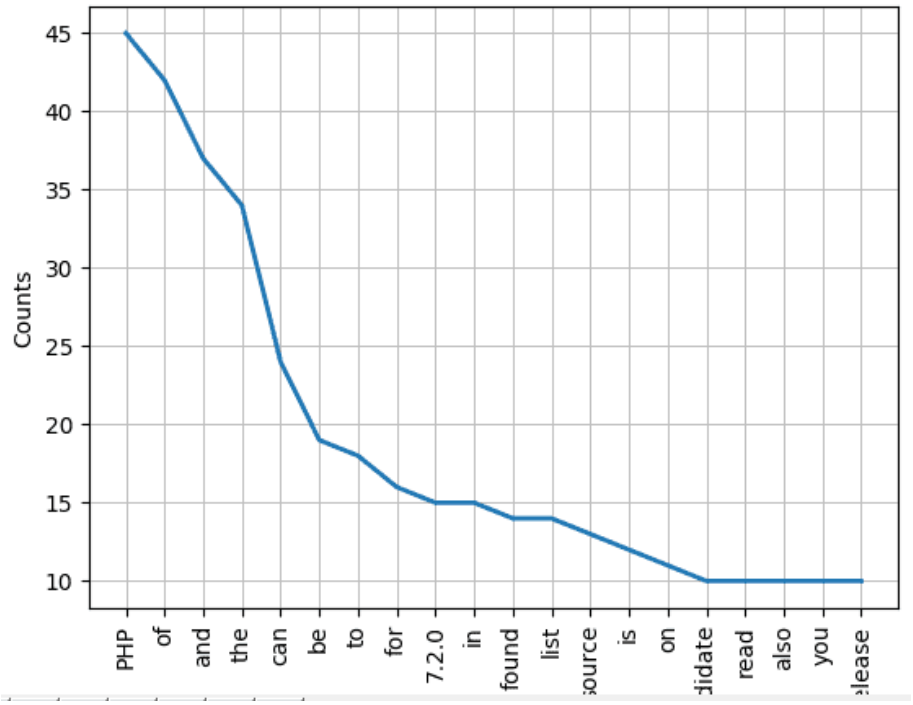
Count word frequency

The text is much better now. Let's calculate the frequency distribution of those tokens using Python NLTK.
There is a function in NLTK called FreqDist() does the job:

```
from bs4 import BeautifulSoup
import urllib.request
import nltk
response = urllib.request.urlopen('http://php.net/')
html = response.read()
soup = BeautifulSoup(html,"html5lib")
text = soup.get_text(strip=True)
tokens = [t for t in text.split()]
freq = nltk.FreqDist(tokens)
for key,val in freq.items():
   print (str(key) + ':' + str(val))
```

If you search the output, you'll find that the most frequent token is PHP.
You can plot a graph for those tokens using plot function like this:

```
freq.plot(20, cumulative=False)
```



Remove stop words using NLTK

NLTK comes with stop words lists for most languages. To get English stop words, you can use this code:

```
from nltk.corpus import stopwords
stopwords.words('english')
```

Now, let's modify our code and clean the tokens before plotting the graph.
First, we will make a copy of the list; then we will iterate over the tokens and remove the stop words:

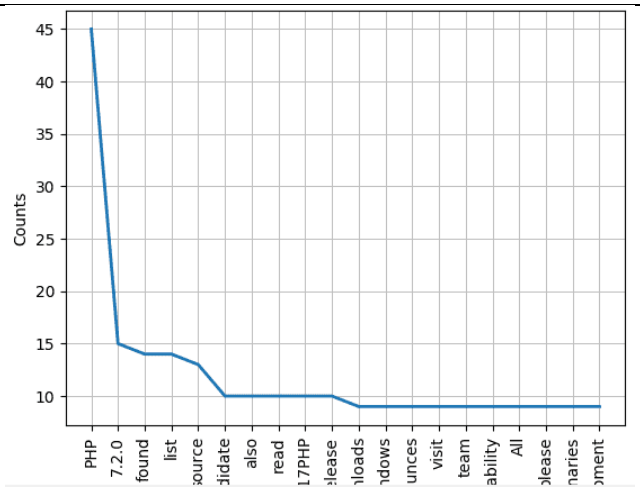
```
clean_tokens = tokens[:]
sr = stopwords.words('english')
for token in tokens:
    if token in stopwords.words('english'):
        clean_tokens.remove(token)
```

You can review the Python list functions to know how to process lists.
So the final code should be like this:

```
from bs4 import BeautifulSoup
import urllib.request
import nltk
from nltk.corpus import stopwords
response = urllib.request.urlopen('http://php.net/')
html = response.read()
soup = BeautifulSoup(html,"html5lib")
text = soup.get_text(strip=True)
tokens = [t for t in text.split()]
clean_tokens = tokens[:]
sr = stopwords.words('english')
for token in tokens:
    if token in stopwords.words('english'):
        clean_tokens.remove(token)
freq = nltk.FreqDist(clean_tokens)
for key,val in freq.items():
    print (str(key) + ':' + str(val))
```

If you check the graph now, it's better than before since no stop words on the count.

```
freq.plot(20,cumulative=False)
```



Lesson 10 - Data Visualization in Python using matplotlib

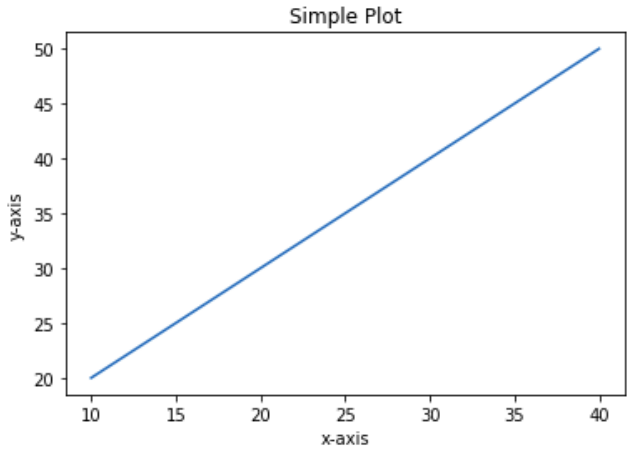
Matplotlib is one of the most popular Python packages used for data visualization. It is a cross-platform library for making 2D plots from data in arrays.

to install

```
\>pip install matplotlib
```

let's create a simple plot. We will be plotting two lists containing the X, Y coordinates for the plot.

```
import matplotlib.pyplot as plt

# initializing the data
x = [10, 20, 30, 40]
y = [20, 30, 40, 50]

# plotting the data
plt.plot(x, y)

# Adding the title
plt.title("Simple Plot")

# Adding the labels
plt.ylabel("y-axis")
plt.xlabel("x-axis")
plt.show()
```



Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

install seaborn

```
\> pip install seaborn
```

Univariate vs Bivariate vs Multivariate Analysis

- Univariate: Visualize and analyze distribution of a single variable only
- Bivariate: Visualize and analyze distribution of 2 variables, tries to check the relationship between these variables to see if it has associations and the strength of this associations.
- Multivariate: Visualize and analyze distribution of more than 2 variables

Univariate Analysis

#Importing the important libraries

```
import pandas as pd
import seaborn as sns
```

#Reading the dataset

```
data = pd.read_csv('employee.csv')
data.shape
```

out: (1470, 35)

lets check the head of the dataset

```
data.head()
```

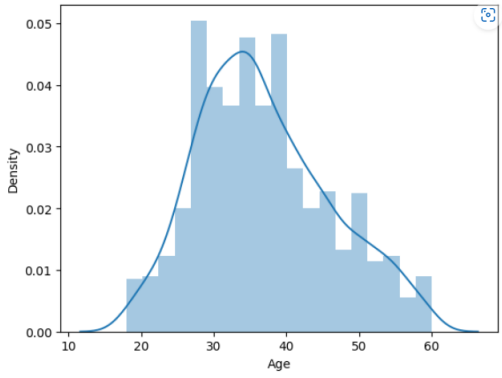
	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educatic
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life S
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life S
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life S
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 35 columns

Univariate Analysis on Numerical Variables

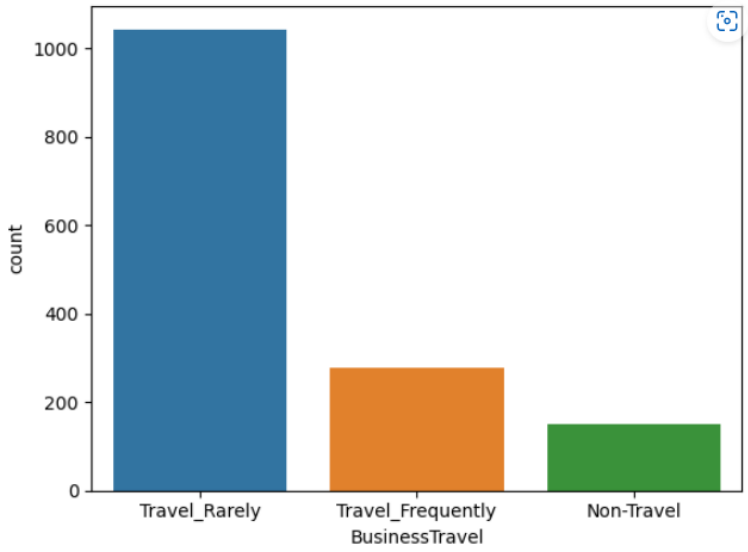
Distribution of Age variable

```
sns.distplot(data['Age'])
```



Univariate Analysis on Categorical Variables

```
sns.countplot(data['BusinessTravel'])
```



Bivariate Analysis

#Importing the important libraries

```
import pandas as pd
import seaborn as sns
```

#Reading the dataset

```
data = pd.read_csv('employee.csv')
data.shape
```

out: (1470, 35)

lets check the head of the dataset

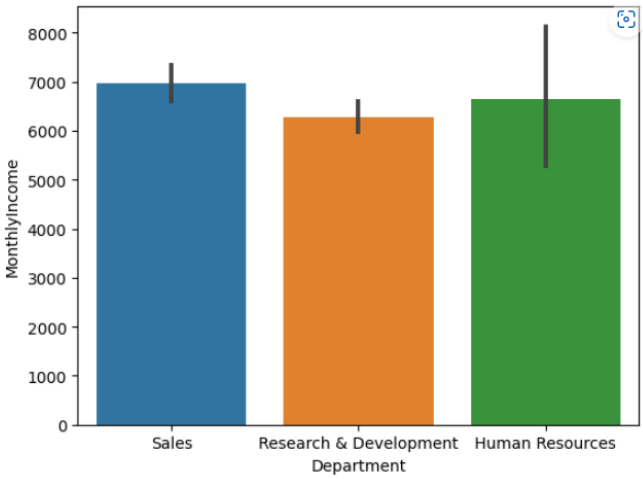
```
data.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educati
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life S
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life S
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life S
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 35 columns

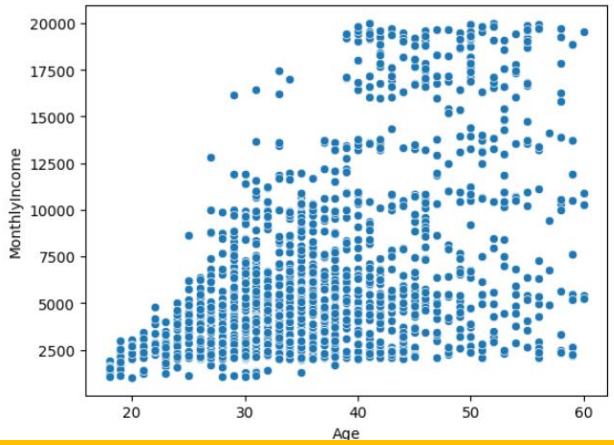
Categorical vs Continuous Variables

```
sns.barplot(x=data['Department'], y=data['MonthlyIncome'])
```



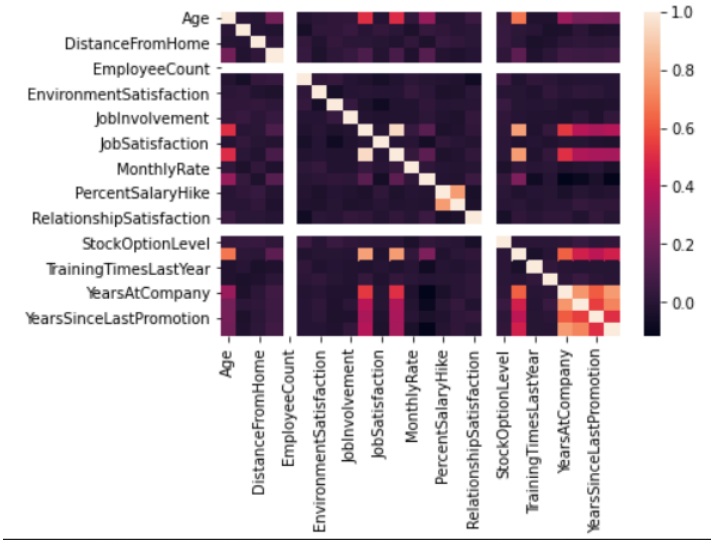
Continous vs Continuous Variables

```
sns.scatterplot(x=data['Age'], y=data['MonthlyIncome'])
```



Multivariate Analysis

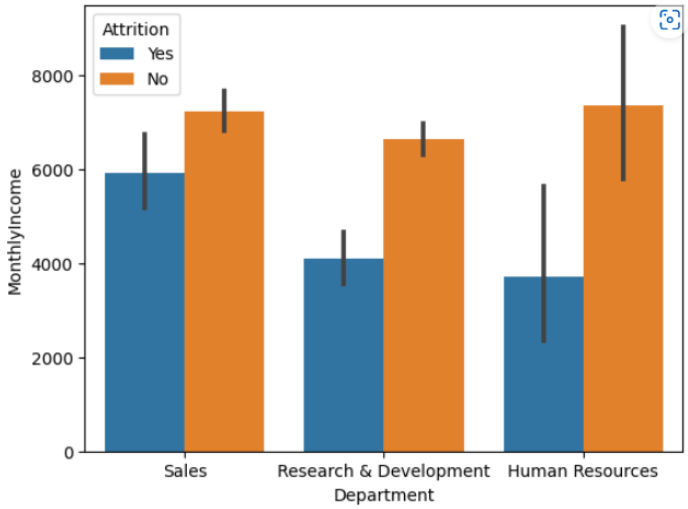
```
sns.heatmap(data.corr())
```



Bar plot with Extra Variable

hue adds a legend

```
sns.barplot(x=data['Department'], y=data['MonthlyIncome'], hue=data['Attrition'])
```



Scatter Plots

Install some modules

```
\> pip install plotly statsmodels
```

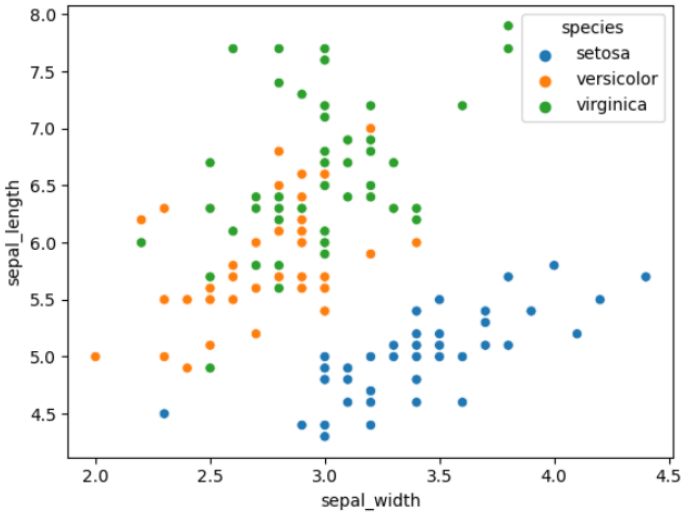
import libs

```
import pandas as pd
import seaborn as sns
import plotly.express as px
```

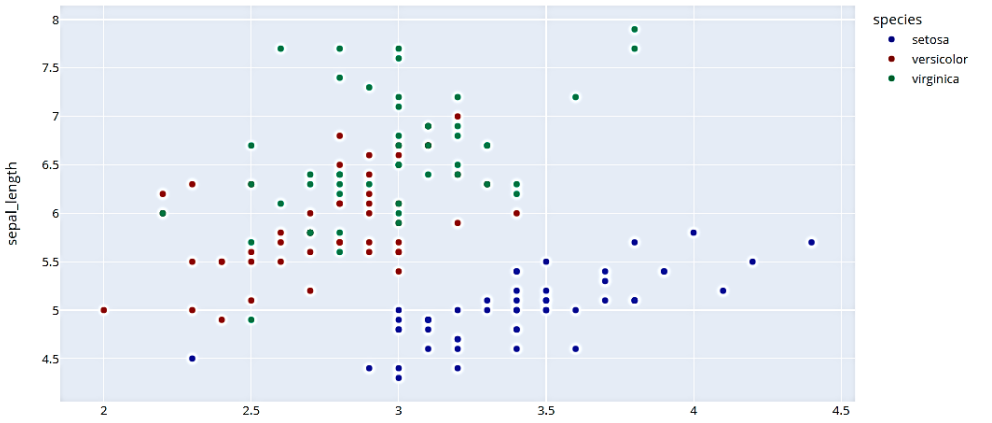
reading the dataset

```
data = px.data.iris()       # iris is a built-in dataset from the plotly data package
data.head()
```

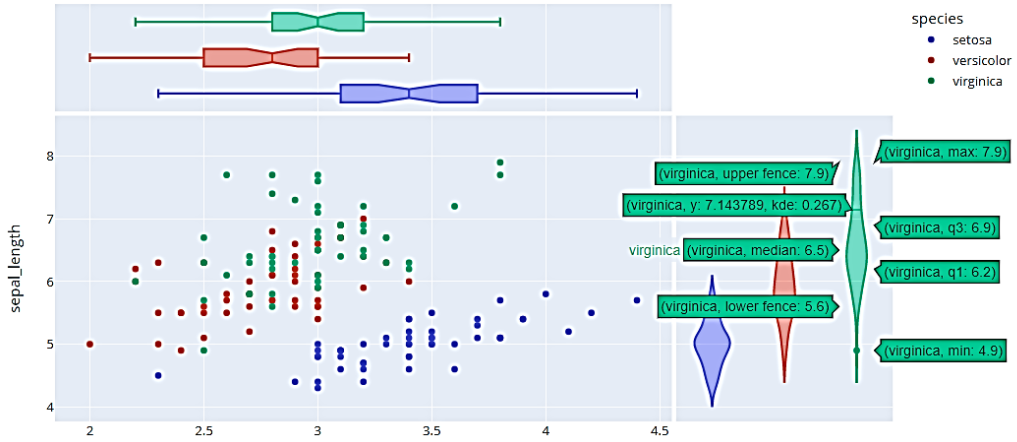
```
sns.scatterplot(data['sepal_width'], data['sepal_length'], hue = data['species'])
```



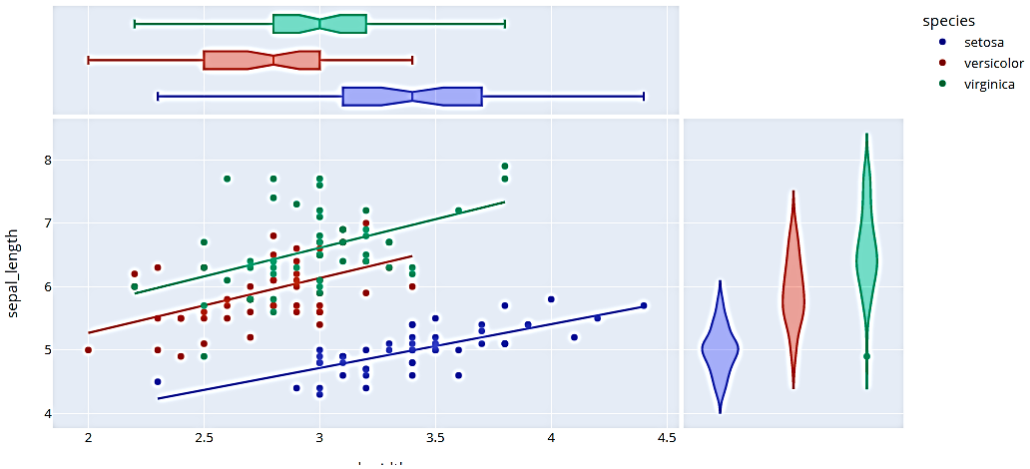
```
px.scatter(data, x="sepal_width", y="sepal_length", color="species")
```




```
px.scatter(data, x="sepal_width", y="sepal_length", color="species", marginal_y="violin", marginal_x="box")
```




```
px.scatter(data, x="sepal_width", y="sepal_length", color="species", marginal_y="violin", marginal_x="box",
trendline="ols")
```



Charts with colorscale

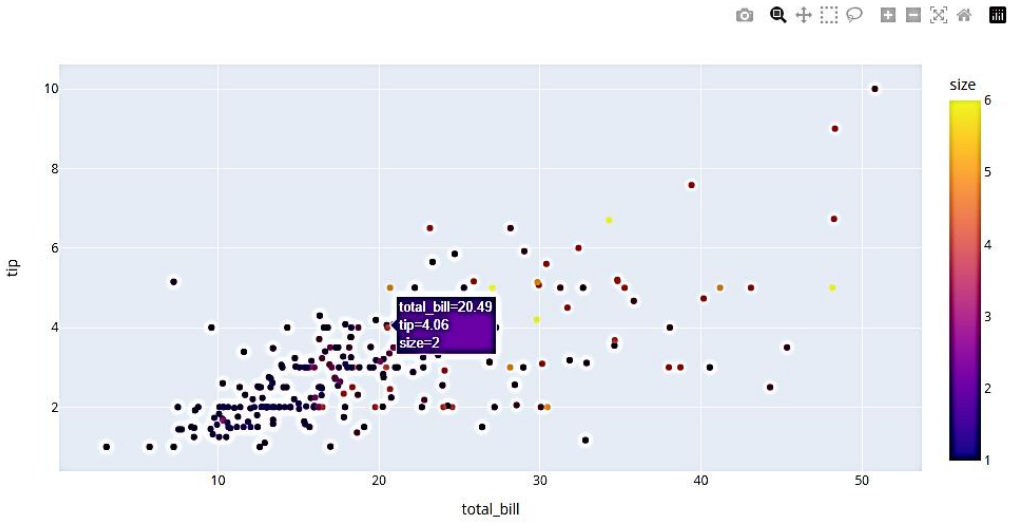
```
import pandas as pd
import plotly.express as px
```

reading the dataset

```
data = px.data.tips()      # tips is a built-in dataset from the plotly data package
data.head()
```

```
px.scatter(data, x="total_bill", y="tip")
```

```
px.scatter(data, x="total_bill", y="tip", color="size")
```



Bar and Line Charts

```
import pandas as pd
import seaborn as sns
import plotly.express as px

data = px.data.gapminder()
data.head()
```

```
sns.lineplot(x=data['year'], y=data['lifeExp'])
```

```
px.line(data, x="year", y="lifeExp")
```

```
px.line(data, x="year", y="lifeExp", color="continent")
```

```
px.line(data, x="year", y="lifeExp", color="continent", line_shape="hv")
```

```
px.line(data, x="year", y="lifeExp", color="continent", line_shape="vh")
```

```
px.area(data, x="year", y="pop", color="continent", line_group="country")
```

```
px.bar(data, x='continent', y='pop')
```

lets read the tips data also

```
df = px.data.tips()
df.head()
```

```
px.bar(df, x="sex", y="total_bill", color="smoker", barmode="group")
```

```
px.bar(df, x="sex", y="total_bill", color="smoker", barmode="stack")
```

Facet Grids

```
import pandas as pd
import plotly.express as px
```

reading the dataset

```
data = px.data.tips()
data.head()
```

```
px.bar(data, x="sex", y="total_bill", color="smoker", barmode="group")
```

```
px.bar(data, x="sex", y="total_bill", color="smoker",
    barmode="group",
    facet_row="time",
    facet_col="day",
    category_orders={"day": ["Thur", "Fri", "Sat", "Sun"], "time": ["Lunch", "Dinner"]})
```

```
px.scatter(data, x="tip", y="total_bill", color="smoker",
    facet_row="time",
    facet_col="day",
    category_orders={"day": ["Thur", "Fri", "Sat", "Sun"], "time": ["Lunch", "Dinner"]})
```

3D Charts

```
import pandas as pd
import plotly.express as px
```

reading the dataset

```
data = px.data.iris()
data.head()
```

```
px.scatter_3d(data, x='sepal_length', y='sepal_width', z='petal_width')
```

```
px.scatter_3d(data, x='sepal_length', y='sepal_width', z='petal_width', color='species')
```

```
px.scatter_3d(data, x='sepal_length', y='sepal_width', z='petal_width', color='petal_length', symbol='species')
```

Maps

```
import plotly.express as px
```

```
df = px.data.carshare()
df.head()
```

Map based on Latitude and Longitude

```
px.scatter_mapbox(df, lat="centroid_lat",
                  lon="centroid_lon",
                  color="peak_hour",
                  size="car_hours",
                  size_max=15, zoom=10,
                  mapbox_style="carto-positron")
```

Scatter Map

```
df = px.data.gapminder()
df.head()
```

```
px.scatter_geo(df, locations="iso_alpha",
               color="continent",
               hover_name="country",
               size="pop",
               projection="natural earth")
```

Animations

 # Animated bubbleplots

```
import pandas as pd
import plotly.express as px
import plotly.offline as py
from bubbly.bubbly import bubbleplot
```

```
data = px.data.gapminder()
data.head()

import warnings
warnings.filterwarnings('ignore')

figure = bubbleplot(dataset = data,
                    y_column = 'gdpPercap',
                    x_column = 'lifeExp',
                    bubble_column = 'continent',
                    time_column = 'year',
                    size_column = 'pop',
                    color_column = 'continent',
    title = 'Life Expectancy vs GDP',
    y_logscale = True,
    scale_bubble = 5,
    height = 650)
```

```
py.iplot(figure, config={'scrollzoom': True})
```

Animation with facets

```
import plotly.express as px
```

```
df = px.data.gapminder()
df.head()
```

```
px.scatter(df, x="gdpPercap", y="lifeExp",
        animation_frame="year",
        animation_group="country",
        size="pop",
        color="continent",
        hover_name="country",
        log_x=True, size_max=45, range_x=[100,100000], range_y=[25,90])
```

```
px.scatter(df, x="gdpPercap", y="lifeExp",
        animation_frame="year",
        animation_group="country",
        size="pop",
        color="continent",
        hover_name="country",
        facet_col="continent",
        log_x=True, size_max=45, range_x=[100,100000], range_y=[25,90])
```

Animation with Scattermaps

```
import plotly.express as px
```

```
df = px.data.gapminder()
df.head()
```

```
px.scatter_geo(df, locations="iso_alpha",
            color="continent",
            hover_name="country",
            size="pop",
            animation_frame="year",
            projection="natural earth")
```

Animation with Chloropleth maps

```
import plotly.express as px
df = px.data.gapminder()
df.head()
```

```
px.choropleth(df, locations="iso_alpha",
            color="lifeExp",
            hover_name="country",
            animation_frame="year",
            range_color=[20,80])
```

Lesson 12 - Python integration with Hadoop MapReduce and Spark

Why Big Data Solutions are Provided for Python

As we all know, Big Data is the most valuable commodity in the modern era. The amount of data generated by companies is increasing at a rapid pace. By 2025, IDC says the worldwide data will reach 175 zettabytes. A zettabyte is equivalent to a trillion gigabytes. Now multiply that 175 times. Then imagine how fast data is exploding.

Choosing a programming language for the Big Data field is very project-specific and depends on its goal. And whatever may be the project goals, Python is the perfect programming language for Big Data because of its easy readability and statistical analysis capacity.

Python is a fast-growing programming language, and a combination of Python and Big Data is the most preferred choice for developers due to less coding and tremendous library support.

1) Simple coding
2) Open-source and easy to learn
3) Python supports multiple libraries
4) Python provides high compatibility with Hadoop
5) Python has a high processing speed
6) Scope
7) Python has data processing support
8) Python is portable
9) Python has large community support
10) Scalability

Hadoop Core Components

Apache Hadoop is an excellent open-source big data technology platform that allows the use of computer networks to perform complex processing and come up with results that are always available even when a few nodes are not available for functional processing. There are a few important Hadoop core components that govern the way it can perform through various cloud-based platforms.

The core components are often termed as modules:

The Distributed File System

It allows the platform to access spread out storage devices and use the basic tools to read the available data and perform the required analysis.

MapReduce

MapReduce is another of Hadoop core components that combines two separate functions, which are required for performing smart big data operations. The first function is reading the data from a database and putting it in a suitable format for performing the required analysis. This is the function that we know more as a mapping activity. It essentially allows a platform to prepare the data for the analytical needs in a common format to allow any computer to do the next step.

The next step in this core component is that of a mathematical operation. This operation is termed as reduction, because it usually reduces the available map to a set of well-defined values that describe an important statistical value.

Hadoop Common

It is also one of the Hadoop core components and it brings that tools which allow any computer to become part of the Hadoop network regardless of the operating system or the present hardware. This module uses Java tools and parts that create a system like the virtual machine and allow the Hadoop platform to store data under its specific file system.

YARN

The fourth of the Hadoop core components is YARN. It is the component which manages all the information sources that store the data and then run the required analysis. It is a system which manages the available resources in a network cluster, as well as schedule the processing tasks to come up with a smart solution, for every big data need on the system.

Python Integration with HDFS using Hadoop Streaming
Demo 01 - Using Hadoop Streaming for Calculating Word Count

Hadoop streaming is a utility that comes with the Hadoop distribution. This utility allows you to create and run Map/Reduce jobs with any executable or script as the mapper and/or the reducer.

Download Hadoop for Linux: https://hadoop.apache.org/releases.html
Install Hadoop on windows: https://medium.com/analytics-vidhya/hadoop-how-to-install-in-5-steps-in-windows-10-61b0e67342f8
Activity Steps here: https://www.geeksforgeeks.org/hadoop-streaming-using-python-word-count-problem/

Python Integration with Spark using PySpark
Demo 02 - Using PySpark to Determine Word Count

Apache Spark is written in Scala programming language. To support Python with Spark, Apache Spark community released a tool, PySpark. Using PySpark, you can work with RDDs in Python programming language also. It is because of a library called Py4j that they are able to achieve this

to install

```
\> pip install pyspark
```

example file

```
import sys
from pyspark import SparkContext, SparkConf

if __name__ == "__main__":

        # create Spark context with necessary configuration
        sc = SparkContext("local","PySpark Word Count Exmaple")

        # read data from text file and split each line into words
        words = sc.textFile("D:/workspace/spark/input.txt").flatMap(lambda line: line.split(" "))

        # count the occurrence of each word
        wordCounts = words.map(lambda word: (word, 1)).reduceByKey(lambda a,b:a +b)

        # save the counts to output
        wordCounts.saveAsTextFile("D:/workspace/spark/output/")
```

Run this Python Spark Application.

```
> spark-submit pyspark_example.py
```

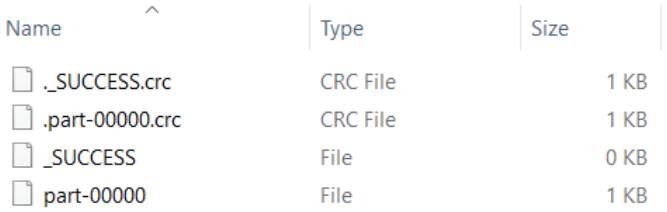
Analyse the Input and Output of PySpark Word Count

Let us analyse the input and output of this Example.

We have provided the following data in the input text file.

```
Python Lists allow us to hold items of heterogeneous types. In this article, we will learn how to create a list in Python; access the list items; find the number of items in the list, how to add an item to list; how to remove an item from the list; loop through list items; sorting a list, reversing a list; and many more transformation and aggregation actions on Python Lists.
```

In the output folder, you would notice the following list of files.



Open part-00000. The contents would be as shown below (image trimmed):

```
('Python', 2)
('Lists', 1)
('allow', 1)
('us', 1)
('to', 5)
('hold', 1)
('items', 2)
('of', 2)
('heterogeneous', 1)
('types.', 1)
```

What have we done in PySpark Word Count?

We created a SparkContext to connect connect the Driver that runs locally.

```
sc = SparkContext("local","PySpark Word Count Exmaple")
```

Next, we read the input text file using SparkContext variable and created a flatmap of words. words is of type PythonRDD.

```
words = sc.textFile("D:/workspace/spark/input.txt").flatMap(lambda line: line.split(" "))
```

we have split the words using single space as separator.

Then we will map each word to a key:value pair of word:1, 1 being the number of occurrences.

```
words.map(lambda word: (word, 1))
```

The result is then reduced by key, which is the word, and the values are added.

```
reduceByKey(lambda a,b:a +b)
```

The result is saved to a text file.

```
wordCounts.saveAsTextFile("D:/workspace/spark/output/")
```